

《计算机组成原理》

（第六讲）

厦门大学信息学院软件工程系

2026年5月

目录

- 第1章 计算机系统概论
- 第2章 数据信息的表示
- 第3章 运算方法与运算器
- 第4章 存储系统
- 第5章 指令系统
- 第6章 中央处理器
- 第7章 指令流水线
- 第8章 总线系统
- 第9章 输入输出系统



第6章 中央处理器

- 6.1 中央处理器概述
- 6.2 指令周期
- 6.3 数据通路及指令操作流程
- 6.4 时序与控制
- 6.5 硬布线控制器（略）
- 6.6 微程序控制器
- 6.7 异常与中断处理（并入第9章 I/O）

6.1 中央处理器概述

• 6.1.1 中央处理器的功能

- 中央处理器（**CPU**: Central Processing Unit）由**运算器**和**控制器**组成，是计算机的核心。
- **CPU**的主要功能是**执行程序**；计算机上电后（或者按Reset键后），**CPU**即开始周而复始地进行取指令和执行指令的工作。
- 为了保证程序的正确执行，**CPU**应该具有以下功能：
 - ① **程序控制**：控制程序中的指令执行顺序（包括顺序执行指令、转移指令、子程序调用指令、子程序返回指令等），即**CPU**必须能够正确地确定下条指令的地址。
 - ② **操作控制**：产生指令执行过程中需要的操作控制信号；例如执行加法指令时，**CPU**必须生成运算器的运算选择控制信号，以保证其进行加法操作。
 - ③ **时序控制**：对每个操作控制信号进行定时，以便按规定的顺序执行各操作，控制各功能部件。
 - ④ **数据加工**：对数据进行算术、逻辑运算，或将数据在各部件之间传送。
 - ⑤ **中断处理**：**CPU**应能及时响应内部异常和外部中断请求。

6.1.2 中央处理器的组成

- 早期计算机的CPU主要包括运算器和控制器；现代计算机的CPU还增加了cache（L1 cache，内部cache）、MMU（存储器管理单元）、浮点运算器等单元；图6.1：CPU的基本组成。
- 运算器**是执行部件，由ALU和各种寄存器组成，负责数据加工。
- 控制器**的主要功能包括取指令、计算下一条指令的地址、对指令进行译码、生成指令对应的操作控制信号序列、控制指令执行的步骤和数据流动的方向。

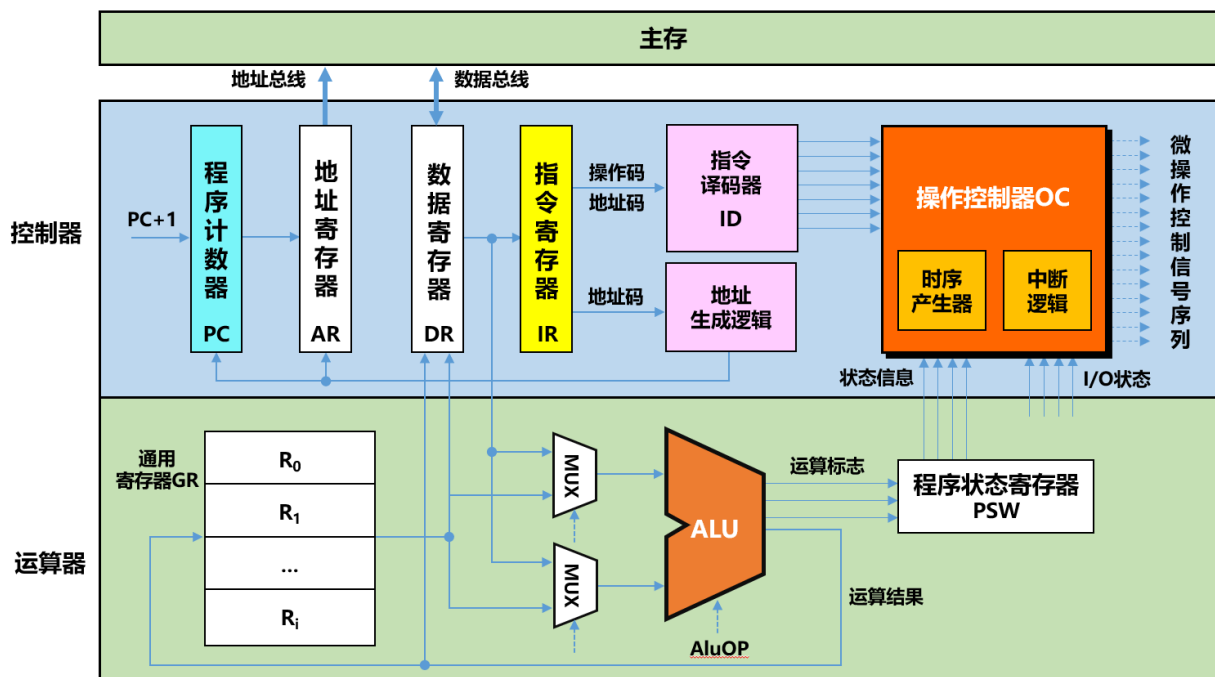


图6.1 CPU的基本组成

6.1.2 中央处理器的组成

- 1、CPU中的主要寄存器

- ① **程序计数器**：PC（Program Counter），Intel x86称为IP（Instruction Pointer），用于保存将要执行的指令的字节地址。
- ② **存储器地址寄存器**：AR（Address Register），或者MAR（Memory Address Register），用来保存CPU访问主存的单元地址。
- ③ **存储器数据寄存器**：DR（Data Register），或者MDR（Memory Data Register），用于存放从主存中读出的数据或准备写入主存的数据。
- ④ **指令寄存器**：IR（Instruction Register），用于保存当前正在执行的指令；指令中的操作码、地址码送**指令译码器**（ID，Instruction Decoder），指令中的地址码送地址生成逻辑。
- ⑤ **通用寄存器组**：GR（General Register），也称寄存器堆（寄存器文件），这些通用寄存器可以作为ALU的累加器、变址寄存器、基址寄存器、地址指针、数据缓冲器，用来存放操作数、中间结果以及各种地址信息等。
- ⑥ **程序状态字寄存器**：PSW/PSR（Program Status Word/Register），Intel x86称为EFLAGS，用于保存由算术运算指令、逻辑运算指令、测试指令等建立的各种条件标志，常见的标志有：**C**（进位标志）、**V**（溢出标志）、**S**（结果为负数标志）、**Z**（结果为零标志）；还用于保存中断和系统工作的状态信息。

6.1.2 中央处理器的组成

- 2、操作控制器

- 操作控制器接收指令译码器送来的指令译码信号，与时序信号、条件及状态信息进行组合，形成各种具有严格时间先后顺序的操作控制信号（微操作控制信号序列）。
- CPU执行指令的过程就是CPU控制信息流的过程，操作控制器产生的微操作控制信号序列就是控制流。

- 3、时序产生器

- 指令执行过程中的所有操作都必须按照一定的次序执行，因此需要引入时序的概念，也就是要对完成指令而执行的微操作控制信号进行时间调制，严格规定各信号的产生时间和持续时间（6.4小节）。
- 根据时序调制方法的不同，操作控制器分为硬布线控制器（6.5小节）和微程序控制器（6.6小节）两种。
- 硬布线控制器采用时序逻辑技术实现，是一种硬时序；微程序控制器采用程序存储逻辑技术实现，是一种软时序。

6.2 指令周期

• 6.2.1 指令执行的一般流程

— 图6.2：控制器**执行指令**的一般流程。

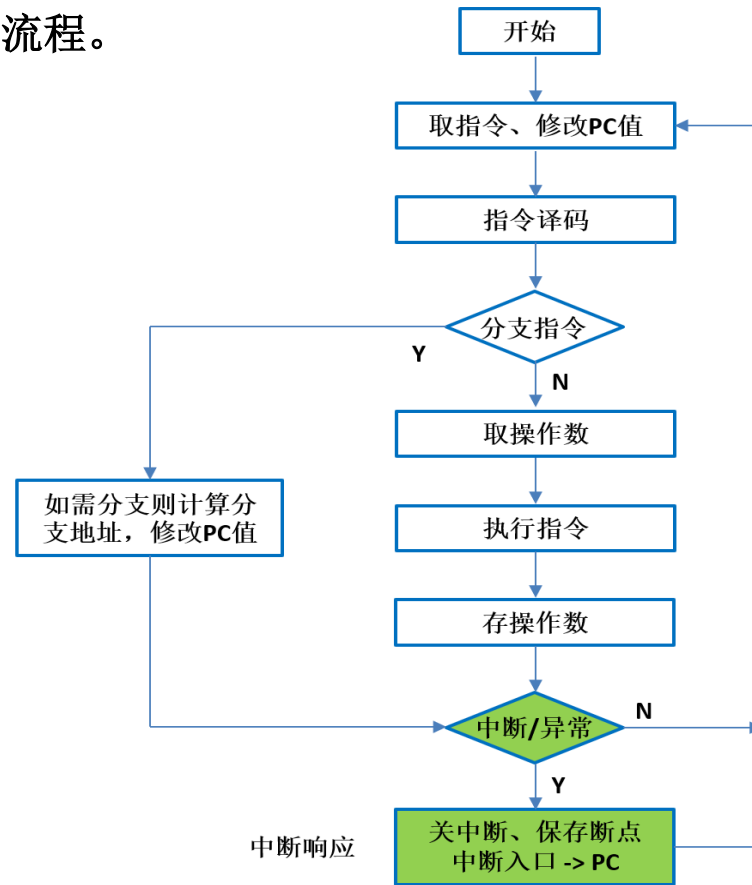


图6.2 指令执行的一般流程

6.2.2 指令周期的基本概念

- **指令周期**：将一条指令从取出到执行完成所需要的时间称为指令周期。
- 指令周期粗分为：**取指周期**和**执行周期**。
- 指令周期细分为：取指周期、**译码/取操作数周期**、执行周期、**写回周期**（存操作数周期）。
- **1、取指周期**
 - 取指周期中，CPU以程序计数器PC的内容为地址从主存取出指令，并计算后续指令的地址（PC的新值）。
 - 顺序执行指令时，PC的新值为PC加当前指令的字节长度（例如，PC+4）。
 - 当执行分支指令（转移指令、子程序调用指令、子程序返回指令等）时，需要根据寻址方式、分支条件、分支目标地址等内容，计算PC的新值。

6.2.2 指令周期的基本概念

– 2、译码/取操作数周期

- 译码/取操作数周期中，对指令寄存器IR中的指令字进行**译码**，识别指令类型。
- 根据指令地址码生成操作数的有效地址，然后访问相应的寄存器或主存单元（**取操作数**）。
- 当寻址方式为间接寻址时，则需要加入访存周期才能得到操作数的地址，这个访存周期又称为**间址周期**：

例：MOV EAX, @2008H

假设主存地址2008H中的内容为A0A0F000H，而主存地址A0A0F000H中的内容为5000H，则操作数S=5000H；即执行指令“MOV EAX, @2008H”后，EAX=5000H。

访问地址为2008H主存单元的访存周期称为**间址周期**；而访问地址为A0A0F000H主存单元的访存周期为取操作数周期。

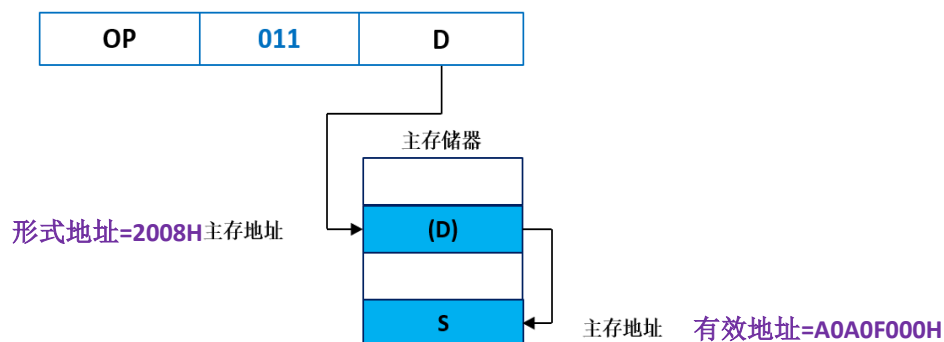


图5.7 间接寻址

6.2.2 指令周期的基本概念

– 3、执行周期

- 控制器向**ALU**（运算器）及数据通路中的其他相关部件发出操作控制命令，对已取出的操作数进行加工处理，并将处理的状态信息记录在**PSW**（程序状态字寄存器）中。
- 当程序出现分支时，执行周期还要计算分支的目标地址。

– 4、写回周期

- 将运算结果写回到目的寄存器或存储器中。

– 除了上述4个周期外，指令周期还包括：

- **中断周期**：当指令执行完毕后，如果出现异常或外部中断请求，**CPU**将进入中断响应周期（中断周期）；利用硬件逻辑保存断点，然后将中断处理程序入口地址送入程序计数器**PC**，实现当前程序与中断处理程序的切换。
- **总线周期**：用于完成总线操作及总线控制权的转移。
- **I/O周期**：用于完成输入输出操作。

6.2.2 指令周期的基本概念

- 可以将指令周期划分为若干个**机器周期**（又称为**CPU周期**，通常将从主存取出一个存储字所需的最短时间定义为机器周期）；而每个机器周期又可包含若干个**时钟周期**（图6.3）。

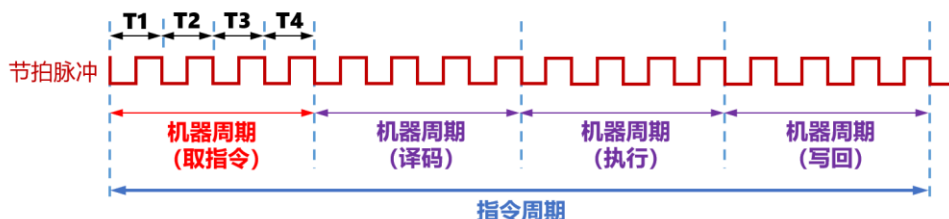


图6.3 指令周期、机器周期与时钟周期的关系

- **定长指令周期**：所有指令的指令周期相同、机器周期数固定、机器周期所包含的节拍数（时钟周期数）也是固定的；如早期的**三级时序系统**（图6.3）：
 - 第一级：指令周期
 - 第二级：机器周期
 - 第三级：时钟周期（节拍）
- **变长指令周期**：现代计算机普遍采用以**时钟信号**进行定时的变长指令周期，不同指令的机器周期数可变，按时钟周期同步；一条指令的执行需要多少个时钟周期就安排多少个时钟周期，机器周期的概念也逐渐消失。

6.2.3 寄存器传送语言

- **寄存器传送语言**：RTL，Register Transfer Language；用来表示指令执行过程中的操作。
- 每条指令的执行过程都可以分解为一组操作序列，进而可分解为一组微操作序列；**操作**是指功能部件级的动作；**微操作**是指令序列中最基本的、不可再分割的动作。
- 图6.4：指令执行过程中的操作与微操作（将寄存器A中的信息传送到寄存器B中）
 - 微操作 A_{out} ：用于控制三态门将寄存器A的输出数据送入寄存器B的输入端口；**三态门**： $A_{out}=0$ 时，输出为**高阻态**； $A_{out}=1$ 时，输出=输入（**0状态**，**1状态**）。
 - 微操作 B_{in} ：用于打开寄存器B的写使能控制信号，通过时钟信号的配合最终将数据锁存到寄存器B中。

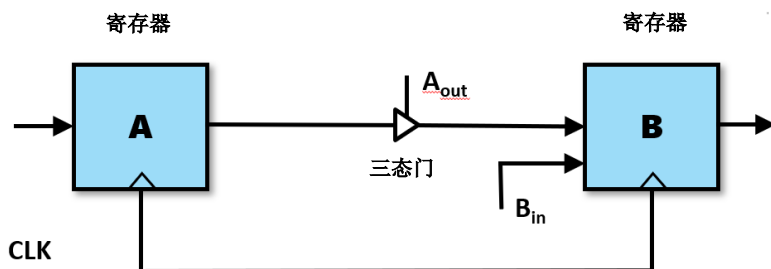


图6.4 指令执行过程的操作和微操作

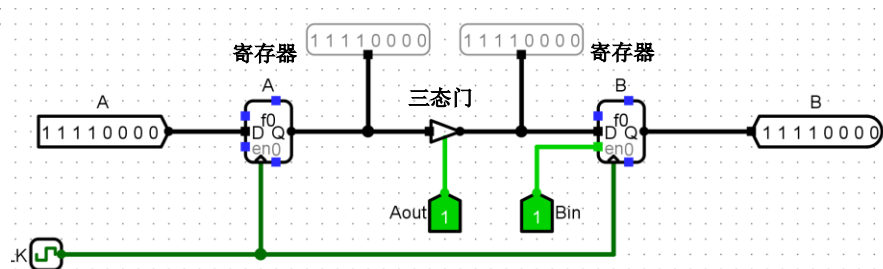


图6.4 Logisim中的电路

6.2.3 寄存器传送语言

– RTL（寄存器传送语言）描述规则如下：

- ① **M[addr]**: 表示主存单元或内容；**addr**为主存单元的地址，作为源操作数使用时表示主存内容，作为目的操作数使用时表示主存单元。
- ② **R[i]**: 表示通用寄存器组中的第*i*号寄存器或内容；*i*为寄存器编号，作为源操作数使用时表示寄存器内容，作为目的操作数使用时表示寄存器。
- ③ 用 “**B <- A**” 或 “**A -> B**” 表示数据传送；其中**A**为源操作数，**B**为目的操作数。
- ④ **X_{y:z}**: 表示寄存器**X**的第**y**位到第**z**位的数据字段；例如，**R_{0:7}**。
- ⑤ **SignExt(X)**: 表示将**X**带符号扩展到32位；例如，**X=0123H**，**SignExt(X)=0000 0123H**；**X=8123H**，**SignExt(X)=FFFF 8123H**。
- ⑥ **{X,Y}**: 表示将**X**、**Y**的比特位连接在一起；例如，**{10,11,011}=1011011**。

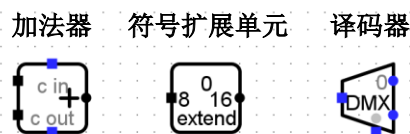
– 根据以上规则：

- **M[R[i]]**: 表示以第*i*号寄存器内容作为地址的主存单元或主存内容（寄存器间接寻址）。
- **M[PC]**: 表示程序计数器**PC**所指向的主存单元的内容。

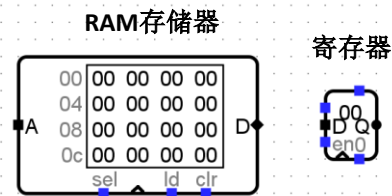
6.3 数据通路及指令操作流程

• 6.3.1 数据通路模型与定时

- 数据在各功能部件之间传送的路径称为**数据通路**；数据通路可用**总线**或**专用通路**两种方式构建。
- 构成数据通路的功能部件根据性质可分为**数据处理单元**和**状态存储单元**。
- **数据处理单元**由组合逻辑电路构成，其输出只与当前的输入有关，负责对数据进行加工和处理，如**ALU**、符号扩展单元、译码器等。
- **状态存储单元**（状态单元）是指带有存储功能的单元，如存储器和寄存器等；状态单元一般包含输入端、写使能控制端、时钟输入端**CLK**、输出端。



Logisim中的数据处理单元



Logisim中的状态存储单元

6.3.1 数据通路模型与定时

- **数据通路模型**：一个数据处理单元连接两个状态单元（图6.5a）。
- **简化的数据通路模型**：只有一个数据处理单元和一个状态单元（图6.5b）。
- 图6.5a中，各状态单元只有在写使能信号（Write1、Write2）有效，且时钟触发到来时，**才能将输入端的数据写入**。
- 并且这个写入过程还必须**遵循较为严格的时序约定**，才能将状态单元1的数据经过数据处理单元处理后，正确写入状态单元2。

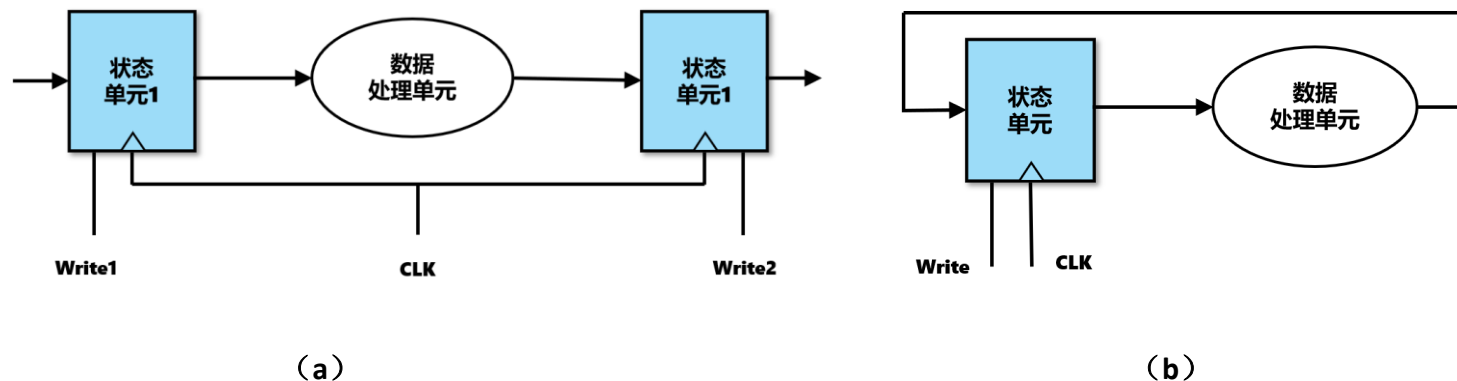


图6.5 数据通路模型

6.3.1 数据通路模型与定时

– D触发器定时模型，有3个时间：

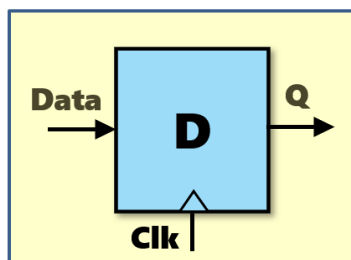
① 建立时间（**Setup Time**）：时钟Clk触发前（上跳沿），输入数据Data须稳定一段时间（即教材中的**寄存器建立时间 T_{setup}** ），便于数据能够被时钟正确的采样。

② 保持时间（**Hold Time**）：时钟Clk触发后（上跳沿），输入数据Data还须稳定一段时间（即教材中的**寄存器保持时间 T_{hold}** ），以便于数据能够被电路准确的传输。

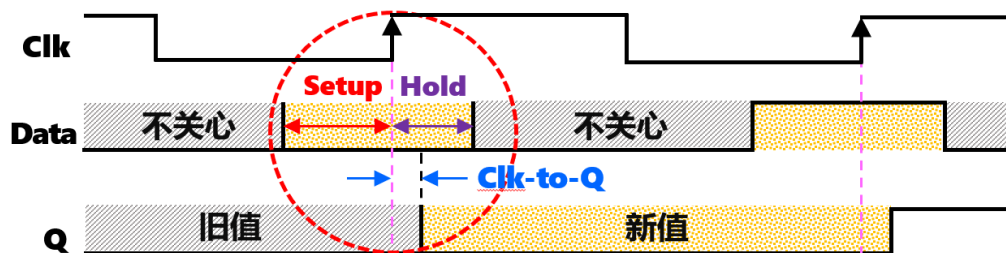
可理解为：时钟到来之前，数据需要提前准备好，时钟到来之后，数据还要稳定一段时间，才能传输

③ 触发器延迟 **Clk_to_Q**：从时钟Clk触发（上跳沿），到输出Q稳定的时间（即教材中的**寄存器延迟 $T_{clk_to_q}$** ，教材图6.6中标记该时间有误）。

– 存储器、寄存器等状态单元的定时模型与D触发器相同。



D触发器



D触发器定时模型

注：D触发器是在CP正跳沿前接受输入信号，正跳沿时触发翻转，正跳沿后输入即被封锁，三步都是在正跳沿后完成，所以有**边沿触发器**之称

6.3.1 数据通路模型与定时

— 数据通路最小时钟周期（图6.6）：

- 图6.6中的寄存器A、寄存器B、组合逻辑，分别对应图6.5a中的状态单元1、状态单元2、数据处理单元。
- 假设数据处理单元（组合逻辑）对寄存器A的输出数据进行处理加工时有一个**关键路径延迟** T_{max} ，即数据处理单元所有输出信号稳定的最大延迟时间。
- 必须满足：数据通路最小时钟周期 $> T_{clk_to_q} + T_{max} + T_{setup}$
- 否则，将导致数据处理单元（组合逻辑）的输出来不及送到寄存器B

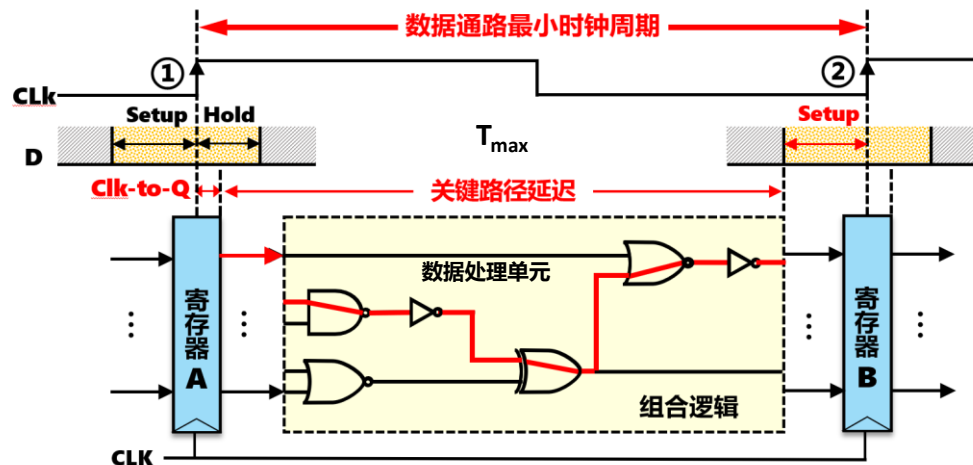


图6.6 数据通路与时钟周期

– 数据通路的保持时间违例（图6.7）：

- 假设数据处理单元（组合逻辑）的**最短路径延迟**为 $T_{\min}$ ，即数据处理单元所有输出信号稳定的最小延迟时间。
- 要使寄存器B正确锁存，时钟上跳沿②之后的输入数据还需要保持一段稳定时间 T_{hold} ；与此同时，时钟上跳沿②也使寄存器A触发，经过 $T_{\text{clk_to_q}} + T_{\min}$ 时间后，数据处理单元（组合逻辑）的输出就会到达寄存器B的输入端。
- 此时，如果寄存器B仍然处于保持时间段（ T_{hold} 时间段），即如果： $T_{\text{hold}} \geq T_{\text{clk_to_q}} + T_{\min}$ ，将导致新的数据（数据处理单元（组合逻辑）的输出）又要进入寄存器B的建立时间段（ T_{setup} 时间段），从而发生**保持时间违例**（即：旧的数据在寄存器B中还没有写入完成，新的数据又要写入寄存器B中）。
- 因此，必须满足： $T_{\text{hold}} < T_{\text{clk_to_q}} + T_{\min}$ （即：控制**最短路径延迟** $T_{\min}$ 不能太小）即：时钟周期不能太短（主频不能太高）

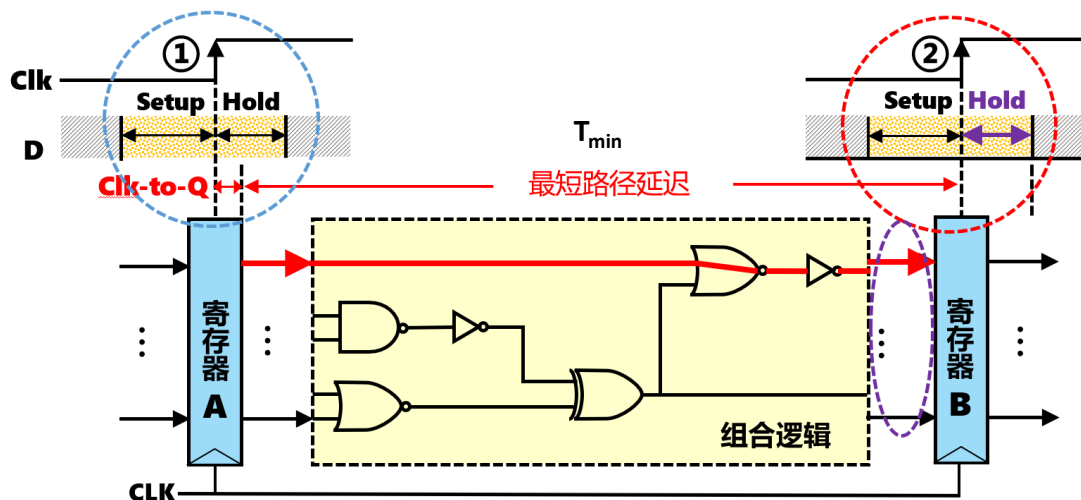


图6.7 数据通路的保持时间违例

6.3.2 单总线结构的数据通路

- 图6.8：单总线结构的计算机框图；CPU中的运算器、控制器、寄存器堆通过一条内总线（也称为**CPU内总线**）连接；图中所有寄存器、存储器的数据位宽均为**32**位。
- 连接CPU、存储器和输入输出设备的总线称为**系统总线**（外总线）。

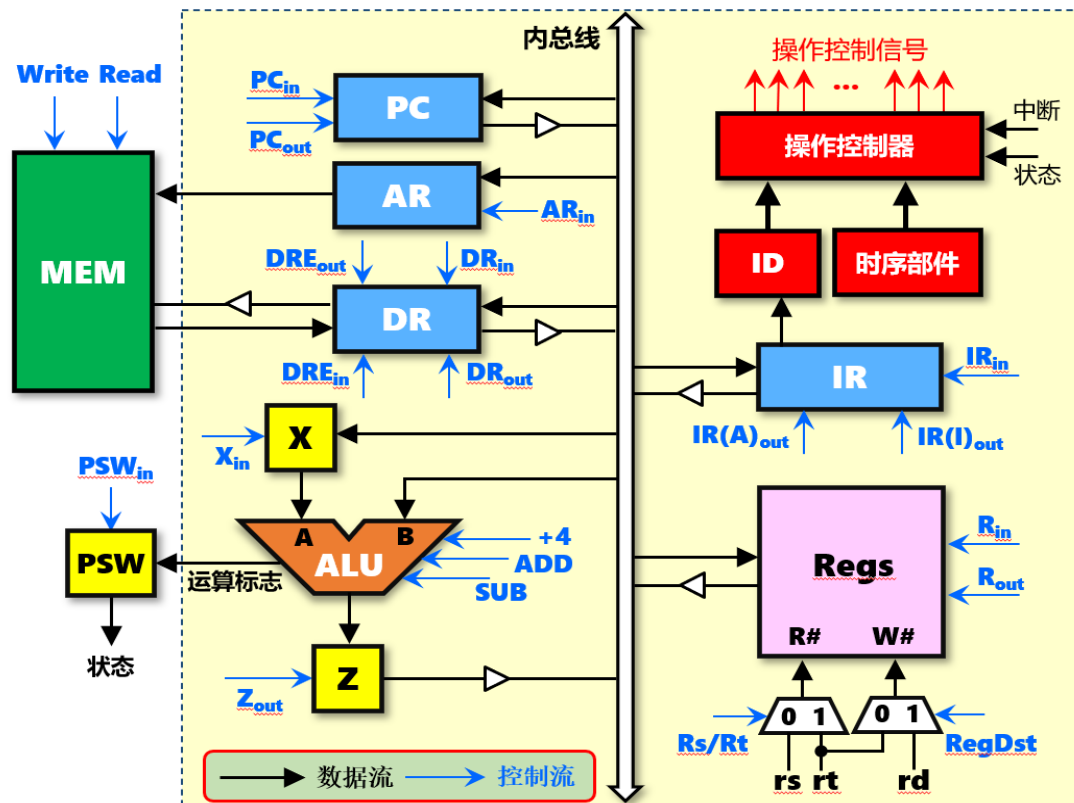


图6.8 单总线结构的计算机框图

— 图6.8中的功能模块：

- ① **PC**：程序计数器；输入来自内总线，输出送到内总线。
- ② **AR**：存储器地址寄存器；输入来自内总线，输出送到存储器**MEM**。
- ③ **DR**：存储器数据寄存器；输入来自内总线或存储器，输出送到内总线或存储器。
- ④ **X**：暂存器；输入来自内总线，输出送到**ALU**。
- ⑤ **Z**：暂存器；输入来自**ALU**，输出送到内总线。
- ⑥ **ALU**：算术逻辑单元；输入**A**来自暂存器**X**，输入**B**来自内总线，输出送到暂存器**Z**。
- ⑦ **PSW**：程序状态字寄存器；输入来自**ALU**，输出送到操作控制器。
- ⑧ **IR**：指令寄存器；输入来自内总线，输出送到内总线和指令译码器**ID**。
- ⑨ **ID**：指令译码器；输入来自**IR**，输出送到操作控制器。
- ⑩ **Regs**：寄存器堆；输入来自内总线，输出送到内总线；**R#**为读出的寄存器编号，**W#**为写入的寄存器编号，分别由两个2路选择器提供。
- ⑪ **MEM**：存储器；地址由**AR**提供，数据来自或送到**DR**。

- 单总线结构的计算机，总线上连接很多功能部件；**为了防止总线上出现数据冲突**，某一时刻只允许一个功能部件向总线发送数据；因此，每个功能部件的输出端都采用**三态门**（图中的三角形标志）进行输出控制。

– 表6.1：图6.8中的控制信号及其功能。

表6.1 控制信号及其功能

所有涉及寄存器输入的写操作，都需要时钟控制

所有向内总线/系统总线输出的操作，都需要设计三态门，防止数据冲突

序号	控制信号	功能说明
1	PC _{in}	控制PC接收来自内总线的数据，需配合时钟控制
2	PC _{out}	控制PC向内总线输出数据
3	AR _{in}	控制AR接收来自内总线的数据，需配合时钟控制
4	DR _{in}	控制DR接收来自内总线的数据，需配合时钟控制
5	DR _{out}	控制DR向内总线输出数据
6	DRE _{in}	控制DR接收从主存读出的数据，需配合时钟控制
7	DRE _{out}	控制DR向主存输出数据，以便最后将该数据写入主存
8	X _{in}	控制暂存器X接收来自内总线的数据，需配合时钟控制
9	+4	将ALU的A端口的数据加4输出
10	ADD	控制ALU执行加法，实现A端口和B端口的两个数相加
11	SUB	控制ALU执行加法，实现A端口和B端口的两个数相减
12	PSW _{in}	控制程序状态字寄存器PSW接收ALU的运算状态，需配合时钟控制
13	Z _{out}	控制暂存器Z向内总线输出数据，注意暂存器Z没有输入控制，每个时钟都会锁存新值
14	IR _{in}	控制IR接收来自内总线的数据，需配合时钟控制
15	IR(A) _{out}	控制IR中的分支目标地址输出到内总线，指令字中的立即数要转换成目标地址，需要相应逻辑
16	IR(I) _{out}	控制IR中的立即数输出到内总线，指令字中的立即数符号扩展为32位才能输出
17	Write	存储器写命令，需配合时钟控制
18	Read	存储器读命令
19	R _{in}	控制寄存器堆接收来自内总线的数据，写入W#端口对应的寄存器中，需配合时钟控制
20	R _{out}	控制寄存器堆输出指定编号R#寄存器的数据，该寄存器堆为单端口输出
21	Rs/Rt	控制多路选择器选择送入R#的寄存器编号，为0时送入指令字中的rs字段，为1时送入rt
22	RegDst	控制多路选择器选择送入W#的寄存器编号，为0时送入指令字中的rt字段，为1时送入rd

- 表6.2：7条典型MIPS 32指令的汇编代码形式及功能说明。
- 使用这7条指令，可以编写通过冒泡法进行内存数据排序的程序。

表6.2 典型MIPS 32指令

序号	指令	汇编代码	指令类型	RTL功能说明	指令功能
1	lw	lw rt,imm(rs)	I型	$R[rt] \leftarrow M[R[rs] + \text{SignExt}(imm)]$	取数指令：寄存器 $\leftarrow$ 存储器
2	sw	sw rt,imm(rs)	I型	$M[R[rs] + \text{SignExt}(imm)] \leftarrow R[rt]$	存取指令：存储器 $\leftarrow$ 寄存器
3	beq	beq rs,rt,imm	I型	If($R[rs] == R[rt]$) PC $\leftarrow$ PC + 4 + SignExt(imm) <<2	条件分支指令：如果rs = rt，则跳转
4	addi	addi rt,rs,imm	I型	$R[rt] \leftarrow R[rs] + \text{SignExt}(imm)$	加法指令：寄存器 $\leftarrow$ 寄存器 + 立即数
5	add	add rd,rs,rt	R型	$R[rd] \leftarrow R[rs] + R[rt]$	加法指令：寄存器 $\leftarrow$ 寄存器 + 寄存器
6	slt	slt rd,rs,rt	R型	$R[rd] \leftarrow (R[rs] < R[rt]) ? 1:0$	比较指令：如果rs < rt，则置rd=1；否则，置rd=0
7	j	j imm26	J型	PC $\leftarrow \{(PC+4)_{31:28}, \text{imm26} <<2\}$	无条件转移指令

- 任何指令的指令周期的第一个机器周期都是取指周期，**取指周期**要完成以下的工作：
 - ① **M[PC] -> IR**: 以程序计数器PC值为地址从存储器中取出指令，并送入指令寄存器IR中保存。
 - ② **PC + 指令长度 -> PC**: 计算顺序指令的地址，修改PC的值，因为MIPS 32是**32位定长**指令，因此这里是：**PC + 4 -> PC**。
 - ③ 指令译码：以确定指令在执行阶段将要进行何种操作。
- **执行周期**中CPU应根据取指令阶段对操作码的译码或测试，进行指令所要求的操作，不同功能的指令具有不同的操作。

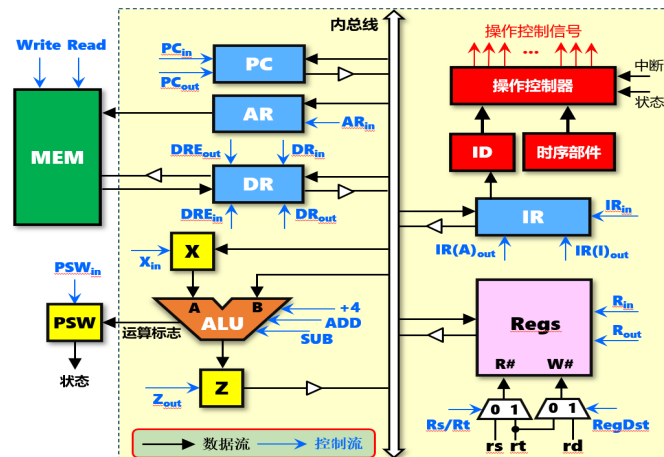


图6.8 单总线结构的计算机框图

– 1、lw指令的执行流程

- lw指令（**取数指令**）属于I型指令，汇编代码为：lw rt,imm(rs)，其指令格式如图6.9所示；lw指令的RTL功能描述是： $R[rt] \leftarrow M[R[rs] + \text{SignExt}(imm)]$

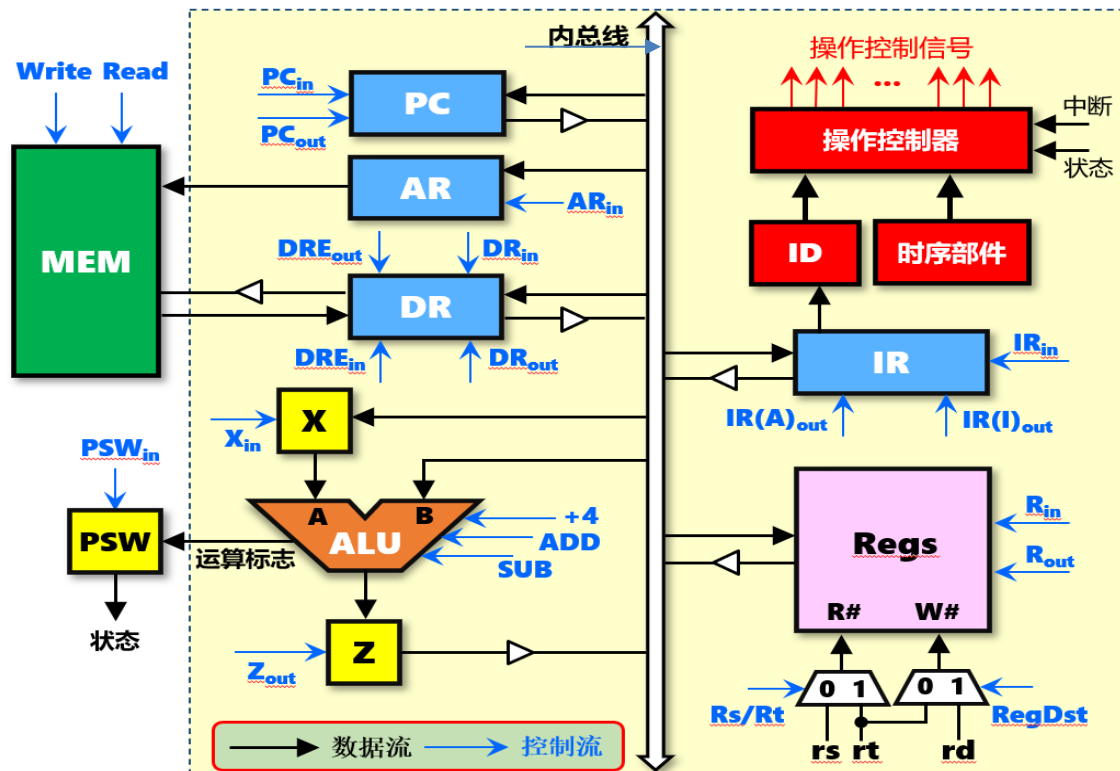


图6.9 lw指令格式

- lw指令的执行共需要3个机器周期，其指令操作流程及控制信号如表6.3所示：
 - ① 第一个机器周期：**取指周期**。
 - ② 第二个机器周期：**计算周期**，用于计算访存地址。
 - ③ 第三个机器周期：**执行周期**，用于实现存储器读取操作。

表6.3 lw指令操作流程及控制信号（仅给出值为1的控制信号）

周期	节拍	操作	功能说明	控制信号
取指周期 M_{if}	T_1	PC $\rightarrow$ AR ; PC $\rightarrow$ X	将程序计数器PC内容送AR，同时送入暂存器X	$PC_{out}=AR_{in}=X_{in}=1$
	T_2	X+4 $\rightarrow$ Z	将PC值加4并送入暂存器Z	+4=1
	T_3	Z $\rightarrow$ PC ; M[AR] $\rightarrow$ DR	将暂存器Z内容回送到PC 同时读AR内容对应主存单元的值并送入DR	$Z_{out}=PC_{in}=1$ Read=DRE _{in} =1
	T_4	DR $\rightarrow$ IR	将DR内容送入IR，完成取指令	DR _{out} =IR _{in} =1
计算周期 M_{cal}	T_1	R[rs] $\rightarrow$ X	将rs寄存器内容送入暂存器X，准备计算访存地址	$R_{out}=X_{in}=1$
	T_2	IR(I) + X $\rightarrow$ Z	将IR中的立即数符号扩展为32位并送入ALU做加法运算	IR(I) _{out} =ADD=1
执行周期 M_{ex}	T_1	Z $\rightarrow$ AR	将Z中暂存的访存地址送入AR	$Z_{out}=AR_{in}=1$
	T_2	M[AR] $\rightarrow$ DR	读AR内容对应主存单元的值并送入DR	Read=DRE _{in} =1
	T_3	DR $\rightarrow$ R[rt]	将DR内容送入寄存器rt	DR _{out} =R _{in} =1



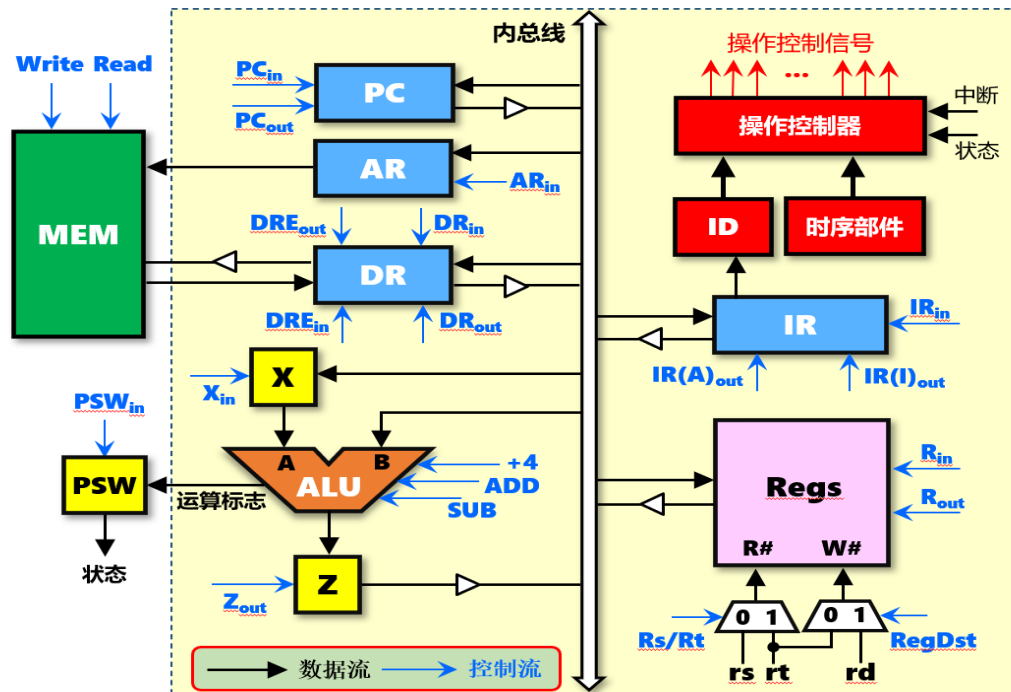
lw指令取指周期使用的两条数据通道：

① PC -> AR -> MEM -> DR -> IR

；以PC为地址访存，取指令，并送入IR

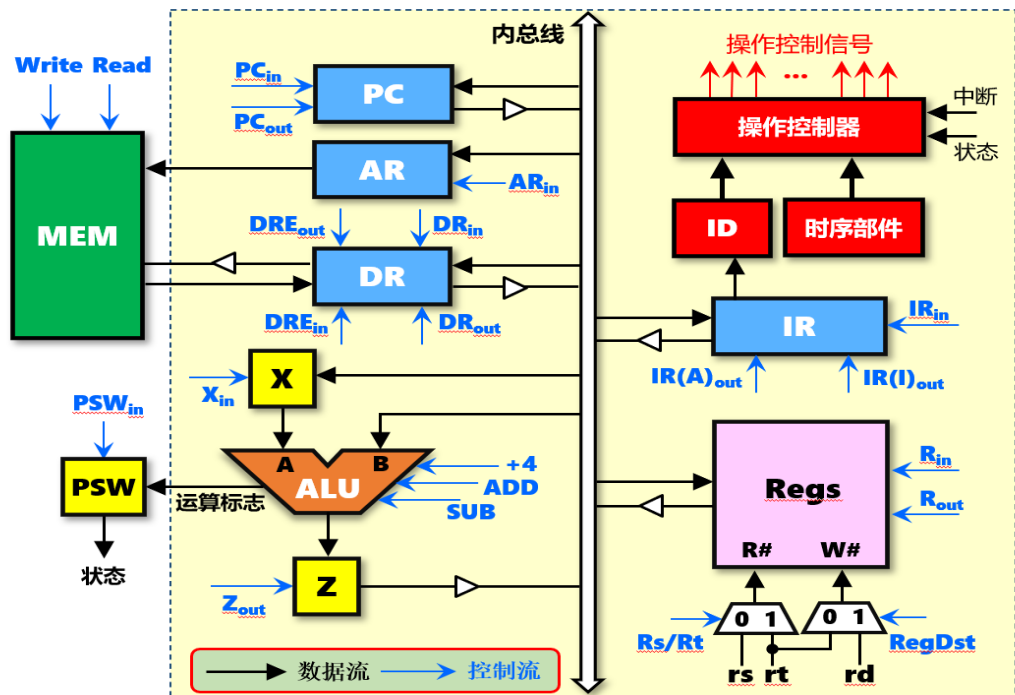
② PC -> X -> ALU -> Z -> PC

；修改PC的值，为取下一条指令做准备



lw指令计算周期使用的数据通道:

$R[rs] \rightarrow X \rightarrow \text{ALU}$; $\text{IR}(I) \rightarrow \text{ALU} \rightarrow Z$; 计算访存地址 $R[rs] + \text{imm}$, 并送入暂存器Z



周期	节拍	操作	功能说明	控制信号
----	----	----	------	------

lw指令执行周期使用的数据通道:

Z -> AR -> MEM -> DR -> R[rt] ;从主存中取32位存储字，并送入寄存器rt

执行周期 M_{ex}	T_1	Z -> AR	将Z中暂存的访存地址送入AR	$Z_{out}=AR_{in}=1$
	T_2	M[AR] -> DR	读AR内容对应主存单元的值并送入DR	Read=DRE _{in} =1
	T_3	DR -> R[rt]	将DR内容送入寄存器rt	DR _{out} =R _{in} =1

周期	节拍	操作	功能说明	控制信号
取指周期 M_{if}	T_1	$PC \rightarrow AR ; PC \rightarrow X$	将程序计数器PC内容送AR，同时送入暂存器X	$PC_{out}=AR_{in}=X_{in}=1$
	T_2	$X+4 \rightarrow Z$	将PC值加4并送入暂存器Z	$+4=1$
	T_3	$Z \rightarrow PC ; M[AR] \rightarrow DR$	将暂存器Z内容回送到PC 同时读AR内容对应主存单元的值并送入DR	$Z_{out}=PC_{in}=1$ $Read=DRE_{in}=1$
	T_4	$DR \rightarrow IR$	将DR内容送入IR，完成取指令	$DR_{out}=IR_{in}=1$
计算周期 M_{cal}	T_1	$R[rs] \rightarrow X$	将rs寄存器内容送入暂存器X，准备计算访存地址	$R_{out}=X_{in}=1$
	T_2	$IR(I) + X \rightarrow Z$	将IR中的立即数符号扩展为32位并送入ALU做加法运算	$IR(I)_{out}=ADD=1$
执行周期 M_{ex}	T_1	$Z \rightarrow AR$	将Z中暂存的访存地址送入AR	$Z_{out}=AR_{in}=1$
	T_2	$M[AR] \rightarrow DR$	读AR内容对应主存单元的值并送入DR	$Read=DRE_{in}=1$
	T_3	$DR \rightarrow R[rt]$	将DR内容送入寄存器rt	$DR_{out}=R_{in}=1$

取指周期中 $M[AR] \rightarrow DR$ ，可以放在节拍 T_3 （上面），也可以放在节拍 T_2 （下面）；两者功能等价

周期	节拍	操作	功能说明	控制信号
取指周期 M_{if}	T_1	$PC \rightarrow AR ; PC \rightarrow X$	将程序计数器PC内容送AR，同时送入暂存器X	$PC_{out}=AR_{in}=X_{in}=1$
	T_2	$X+4 \rightarrow Z ; M[AR] \rightarrow DR$	将PC值加4并送入暂存器Z 同时读AR内容对应主存单元的值并送入DR	$+4=1$ $Read=DRE_{in}=1$
	T_3	$Z \rightarrow PC$	将暂存器Z内容回送到PC	$Z_{out}=PC_{in}=1$
	T_4	$DR \rightarrow IR$	将DR内容送入IR，完成取指令	$DR_{out}=IR_{in}=1$
计算周期 M_{cal}	T_1	$R[rs] \rightarrow X$	将rs寄存器内容送入暂存器X，准备计算访存地址	$R_{out}=X_{in}=1$
	T_2	$IR(I) + X \rightarrow Z$	将IR中的立即数符号扩展为32位并送入ALU做加法运算	$IR(I)_{out}=ADD=1$
执行周期 M_{ex}	T_1	$Z \rightarrow AR$	将Z中暂存的访存地址送入AR	$Z_{out}=AR_{in}=1$
	T_2	$M[AR] \rightarrow DR$	读AR内容对应主存单元的值并送入DR	$Read=DRE_{in}=1$
	T_3	$DR \rightarrow R[rt]$	将DR内容送入寄存器rt	$DR_{out}=R_{in}=1$

周期	节拍	操作	功能说明	控制信号
取指周期 M_{if}	T_1	PC $\rightarrow$ AR ; PC $\rightarrow$ X	将程序计数器PC内容送AR，同时送入暂存器X	$PC_{out}=AR_{in}=X_{in}=1$
	T_2	X+4 $\rightarrow$ Z	将PC值加4并送入暂存器Z	+4=1
	T_3	Z $\rightarrow$ PC ; M[AR] $\rightarrow$ DR	将暂存器Z内容回送到PC 同时读AR内容对应主存单元的值并送入DR	$Z_{out}=PC_{in}=1$ Read=DRE _{in} =1
	T_4	DR $\rightarrow$ IR	将DR内容送入IR，完成取指令	DR _{out} =IR _{in} =1
计算周期 M_{cal}	T_1	R[rs] $\rightarrow$ X	将rs寄存器内容送入暂存器X，准备计算访存地址	$R_{out}=X_{in}=1$
	T_2	IR(I) + X $\rightarrow$ Z	将IR中的立即数符号扩展为32位并送入ALU做加法运算	IR(I) _{out} =ADD=1
执行周期 M_{ex}	T_1	Z $\rightarrow$ AR	将Z中暂存的访存地址送入AR	$Z_{out}=AR_{in}=1$
	T_2	M[AR] $\rightarrow$ DR	读AR内容对应主存单元的值并送入DR	Read=DRE _{in} =1
	T_3	DR $\rightarrow$ R[rt]	将DR内容送入寄存器rt	DR _{out} =R _{in} =1

计算周期中，暂存器X可以是暂存rs寄存器的值（上面），也可以是暂存符号扩展后的立即数的值（下面）；两者结果一样

周期	节拍	操作	功能说明	控制信号
取指周期 M_{if}	T_1	PC $\rightarrow$ AR ; PC $\rightarrow$ X	将程序计数器PC内容送AR，同时送入暂存器X	$PC_{out}=AR_{in}=X_{in}=1$
	T_2	X+4 $\rightarrow$ Z	将PC值加4并送入暂存器Z	+4=1
	T_3	Z $\rightarrow$ PC ; M[AR] $\rightarrow$ DR	将暂存器Z内容回送到PC 同时读AR内容对应主存单元的值并送入DR	$Z_{out}=PC_{in}=1$ Read=DRE _{in} =1
	T_4	DR $\rightarrow$ IR	将DR内容送入IR，完成取指令	DR _{out} =IR _{in} =1
计算周期 M_{cal}	T_1	IR(I) $\rightarrow$ X	将IR中的立即数符号扩展为32位并送入暂存器X	IR(I) _{out} =X _{in} =1
	T_2	R(rs) + X $\rightarrow$ Z	将rs寄存器内容送入ALU做加法运算	$R_{out}=ADD=1$
执行周期 M_{ex}	T_1	Z $\rightarrow$ AR	将Z中暂存的访存地址送入AR	$Z_{out}=AR_{in}=1$
	T_2	M[AR] $\rightarrow$ DR	读AR内容对应主存单元的值并送入DR	Read=DRE _{in} =1
	T_3	DR $\rightarrow$ R[rt]	将DR内容送入寄存器rt	DR _{out} =R _{in} =1

– 2、sw指令的执行流程

- sw指令（**存数指令**）也属于I型指令，汇编代码为：sw rt,imm(rs)，其指令格式如图6.10所示；sw指令的RTL功能描述是： $M[R[rs] + \text{SignExt}(\text{imm})] \leftarrow R[rt]$

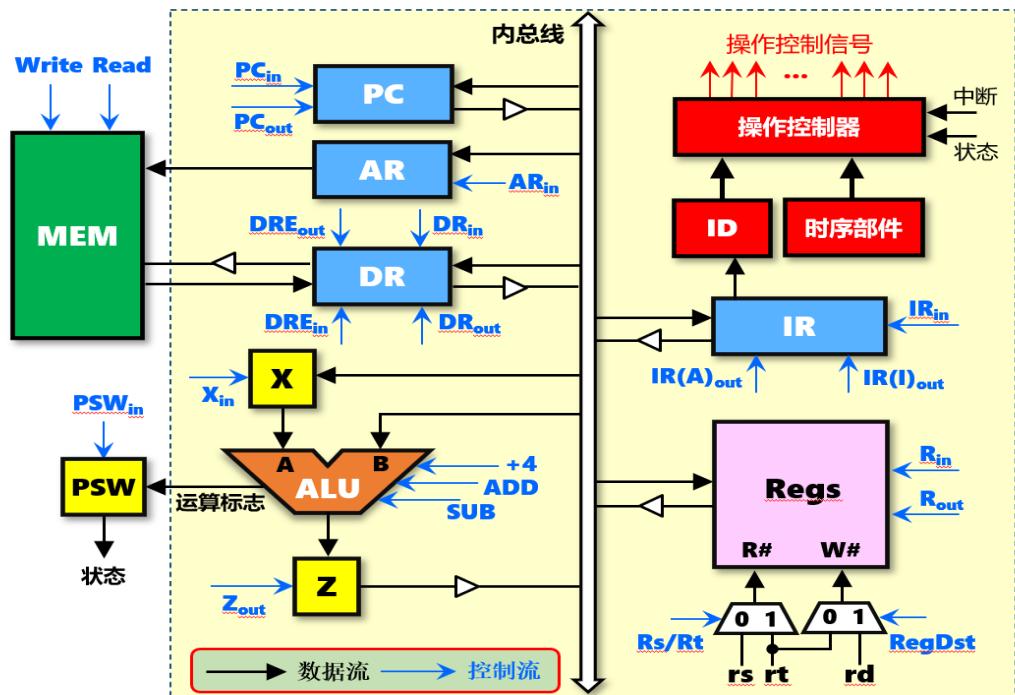


图6.10 sw指令格式

- sw指令的执行也需要3个机器周期，其指令操作流程及控制信号如表6.4所示：
 - ① 第一个机器周期：**取指周期**。
 - ② 第二个机器周期：**计算周期**，用于计算访存地址。
 - ③ 第三个机器周期：**执行周期**，用于实现存储器写入操作。

表6.4 sw指令操作流程及控制信号（仅给出值为1的控制信号）

周期	节拍	操作	功能说明	控制信号
取指周期 M_{if}	T_1	PC $\rightarrow$ AR ; PC $\rightarrow$ X	将程序计数器PC内容送AR，同时送入暂存器X	$PC_{out}=AR_{in}=X_{in}=1$
	T_2	X+4 $\rightarrow$ Z	将PC值加4并送入暂存器Z	+4=1
	T_3	Z $\rightarrow$ PC ; M[AR] $\rightarrow$ DR	将暂存器Z内容回送到PC 同时读AR内容对应主存单元的值并送入DR	$Z_{out}=PC_{in}=1$ $Read=DRE_{in}=1$
	T_4	DR $\rightarrow$ IR	将DR内容送入IR，完成取指令	$DR_{out}=IR_{in}=1$
计算周期 M_{cal}	T_1	R[rs] $\rightarrow$ X	将rs寄存器内容送入暂存器X，准备计算访存地址	$R_{out}=X_{in}=1$
	T_2	IR(I) + X $\rightarrow$ Z	将IR中的立即数符号扩展为32位并送入ALU做加法运算	$IR(I)_{out}=ADD=1$
执行周期 M_{ex}	T_1	Z $\rightarrow$ AR	将Z中暂存的访存地址送入AR	$Z_{out}=AR_{in}=1$
	T_2	R[rt] $\rightarrow$ DR	将rt寄存器内容送入DR	$R_{out}=Rs/Rt=DR_{in}=1$
	T_3	DR $\rightarrow$ M[AR]	将DR内容写入AR内容所指的主存单元	$DRE_{out}=Write=1$

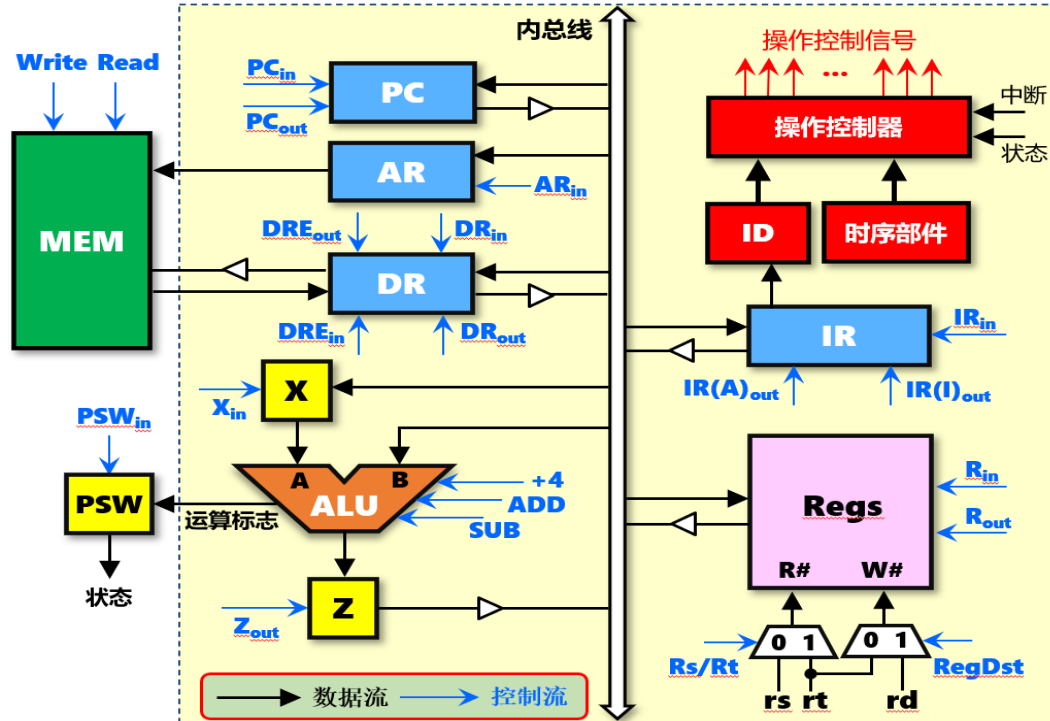


周期	节拍	操作	功能说明	控制信号
取指周期 M_{if}	T_1	$PC \rightarrow AR ; PC \rightarrow X$	将程序计数器PC内容送AR, 同时送入暂寄存器X	$PC_{out}=AR_{in}=X_{in}=1$
	T_2	$X+4 \rightarrow Z$	将PC值加4并送入暂寄存器Z	$+4=1$
	T_3	$Z \rightarrow PC ; M[AR] \rightarrow DR$	将暂寄存器Z内容回送到PC 同时读AR内容对应主存单元的值并送入DR	$Z_{out}=PC_{in}=1$ $Read=DRE_{in}=1$
	T_4	$DR \rightarrow IR$	将DR内容送入IR, 完成取指令	$DR_{out}=IR_{in}=1$

sw指令取指周期使用的两条数据通道（与lw指令相同）：

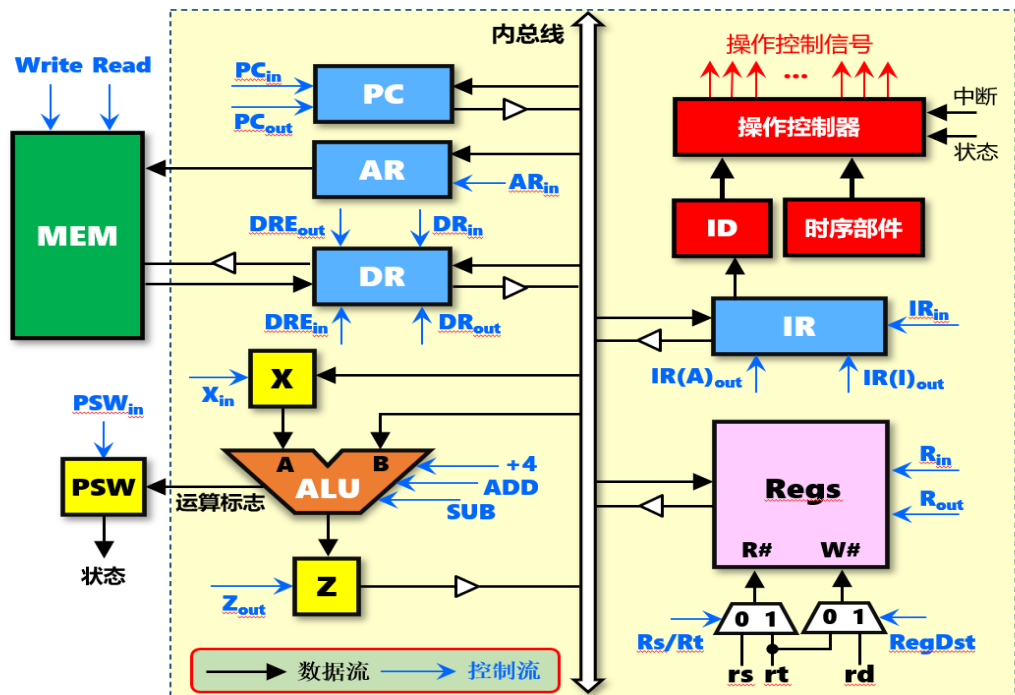
① $PC \rightarrow AR \rightarrow MEM \rightarrow DR \rightarrow IR$;以PC为地址访存, 取指令, 并送入IR

② $PC \rightarrow X \rightarrow ALU \rightarrow Z \rightarrow PC$;修改PC的值, 为取下一条指令做准备



sw指令计算周期使用的数据通道（与lw指令相同）：

$R[rs] \rightarrow X \rightarrow ALU$; $IR(I) \rightarrow ALU \rightarrow Z$; 计算访存地址 $R[rs] + imm$ ，并送入暂存器Z



周期	节拍	操作	功能说明	控制信号
----	----	----	------	------

sw指令执行周期使用的数据通道:

Z -> AR; R[rt] -> DR -> MEM ;将寄存器rt的内容，写入主存单元

执行周期 M_{ex}	T_1	Z -> AR	将Z中暂存的访存地址送入AR	$Z_{out}=AR_{in}=1$
	T_2	R[rt] -> DR	将rt寄存器内容送入DR	$R_{out}=Rs/Rt=DR_{in}=1$
	T_3	DR -> M[AR]	将DR内容写入AR内容所指的主存单元	$DRE_{out}=Write=1$

– 3、beq指令的执行流程

- beq指令（**条件分支指令**）也属于I型指令，汇编代码为：beq rs,rt,imm，其指令格式如图6.11所示；beq指令的RTL功能描述是：If(R[rs] == R[rt]) PC <- PC + 4 + SignExt(imm) <<2



图6.11 beq指令格式

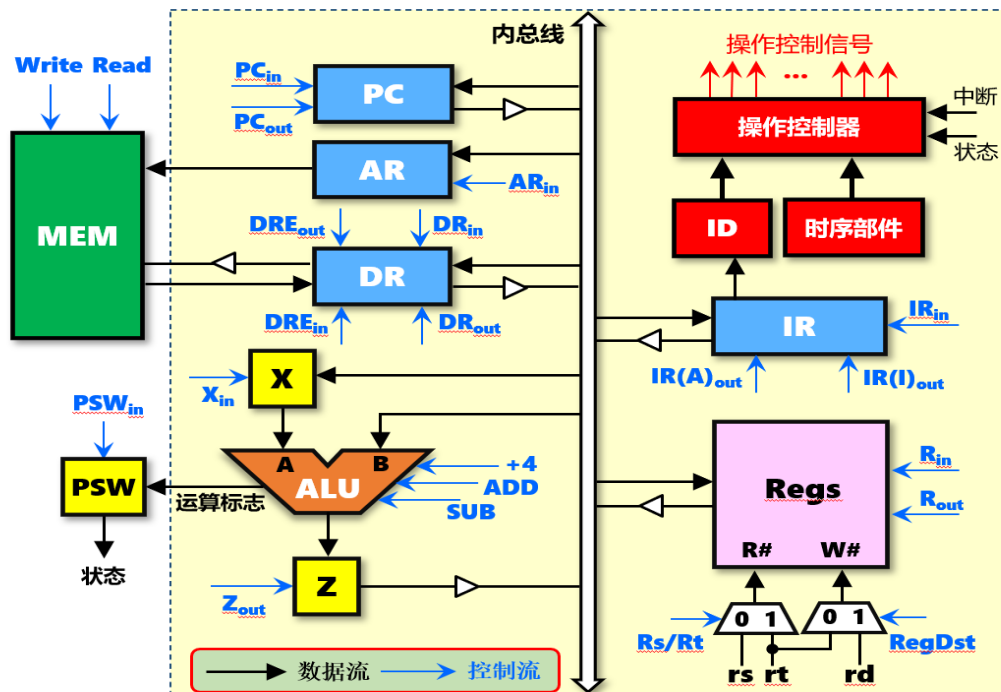
- beq指令的执行也需要3个机器周期，其指令操作流程及控制信号如表6.5所示：
 - ① 第一个机器周期：**取指周期**。
 - ② 第二个机器周期：**计算周期**，用于**比较两个寄存器的值**，并产生用于条件分支的标志位。
 - ③ 第三个机器周期：**执行周期**，负责计算分支目标地址，并根据计算周期生成的标志位，决定是否分支跳转。

表6.5 beq指令操作流程及控制信号（仅给出值为1的控制信号）

周期	节拍	操作	功能说明	控制信号
取指周期 M_{if}	T_1	PC → AR ; PC → X	将程序计数器PC内容送AR，同时送入暂存器X	$PC_{out}=AR_{in}=X_{in}=1$
	T_2	$X+4 \rightarrow Z$	将PC值加4并送入暂存器Z	$+4=1$
	T_3	$Z \rightarrow PC$; $M[AR] \rightarrow DR$	将暂存器Z内容回送到PC 同时读AR内容对应主存单元的值并送入DR	$Z_{out}=PC_{in}=1$ $Read=DRE_{in}=1$
	T_4	DR → IR	将DR内容送入IR，完成取指令	$DR_{out}=IR_{in}=1$
计算周期 M_{cal}	T_1	$R[rs] \rightarrow X$	将rs寄存器内容送入暂存器X，准备进行比较	$R_{out}=X_{in}=1$
	T_2	$X - R[rt] \rightarrow PSW$	将rt寄存器内容送入ALU做减法，可产生结果为零的标志位	$R_{out}=Rs/Rt=SUB=PSW_{in}=1$
执行周期 M_{ex}	T_1	PC → X	将PC内容送入暂存器X	$PC_{out}=X_{in}=1$
	T_2	$IR(A) + X \rightarrow Z$	将IR中的立即数符号扩展为32位后，左移两位送入ALU做加法，计算出分支目标地址	$IR(A)_{out}=ADD=1$
	T_3	if(PSW.equal) Z → PC	如果equal标志位1，将分支目标地址送入PC	$Z_{out}=1$ $PC_{in}=PSW.equal$

否则PC值
维持Mif,T3
: $PC+4 \rightarrow PC$

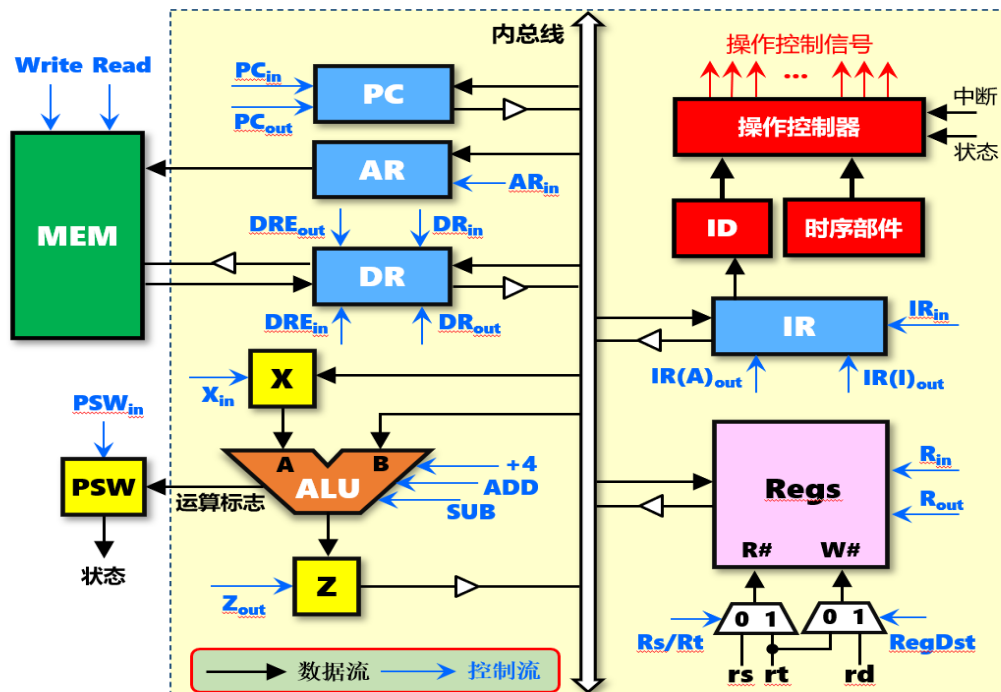
Pcin控制信
号受到equal
控制



周期	节拍	操作	功能说明	控制信号
取指周期 M_{if}	T_1	PC $\rightarrow$ AR ; PC $\rightarrow$ X	将程序计数器PC内容送AR, 同时送入暂存器X	$PC_{out}=AR_{in}=X_{in}=1$
	T_2	$X+4 \rightarrow Z$	将PC值加4并送入暂存器Z	$+4=1$
	T_3	$Z \rightarrow PC$; $M[AR] \rightarrow DR$	将暂存器Z内容回送到PC 同时读AR内容对应主存单元的值并送入DR	$Z_{out}=PC_{in}=1$ $Read=DRE_{in}=1$
	T_4	$DR \rightarrow IR$	将DR内容送入IR, 完成取指令	$DR_{out}=IR_{in}=1$

beq指令取指周期使用的两条数据通道（与lw指令相同）：

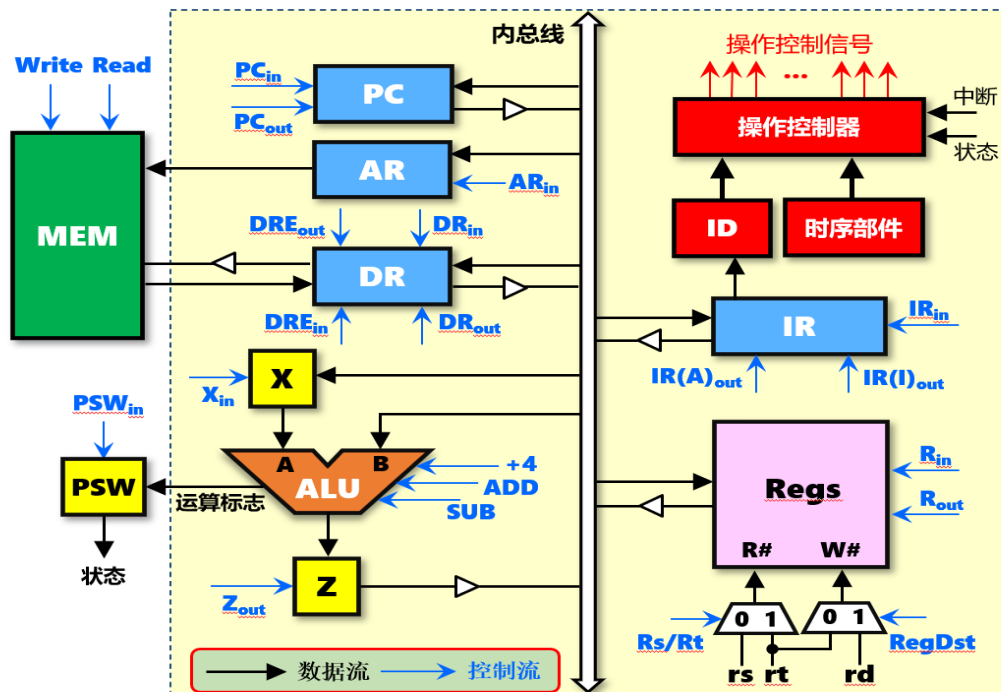
- ① PC $\rightarrow$ AR $\rightarrow$ MEM $\rightarrow$ DR $\rightarrow$ IR ;以PC为地址访存, 取指令, 并送入IR
- ② PC $\rightarrow$ X $\rightarrow$ ALU $\rightarrow$ Z $\rightarrow$ PC ;修改PC的值, 为取下一条指令做准备



beq指令计算周期使用的数据通道：

$R[rs] \rightarrow X \rightarrow ALU$; $R[rt] \rightarrow ALU \rightarrow PSW$; 比较两个寄存器的值，生成相等标志位送入 PSW

计算周期 M_{cal}	T_1	$R[rs] \rightarrow X$	将rs寄存器内容送入暂存器X，准备进行比较	$R_{out}=X_{in}=1$
	T_2	$X - R[rt] \rightarrow PSW$	将rt寄存器内容送入ALU做减法，可产生结果为零的标志位	$R_{out}=Rs/Rt=SUB=PSW_{in}=1$



周期	节拍	操作	功能说明	控制信号
----	----	----	------	------

beq指令执行周期使用的数据通道:

PC -> X -> ALU; IR(A) -> ALU -> Z ; if(PSW.equal) Z -> PC ;计算分支地址，根据标志位进行分支跳转

执行周期 M_{ex}	T_1	PC -> X	将PC内容送入暂存器X	$PC_{out}=X_{in}=1$
	T_2	IR(A) + X -> Z	将IR中的立即数符号扩展为32位后，左移两位送入ALU做加法，计算出分支目标地址	$IR(A)_{out}=ADD=1$ ，控制将IR中的分支目标地址（立即数）送入内总线
	T_3	if(PSW.equal) Z -> PC	如果equal标志位=1，将分支目标地址送入PC	$Z_{out}=1$ $PC_{in}=PSW.equal$

– 4、addi指令的执行流程

- **addi指令（立即数加法指令）**也属于I型指令，汇编代码为：**addi rt,rs,imm**，其指令格式如图6.12所示；**addi指令的RTL功能描述是： $R[rt] \leftarrow R[rs] + \text{SignExt}(imm)$**

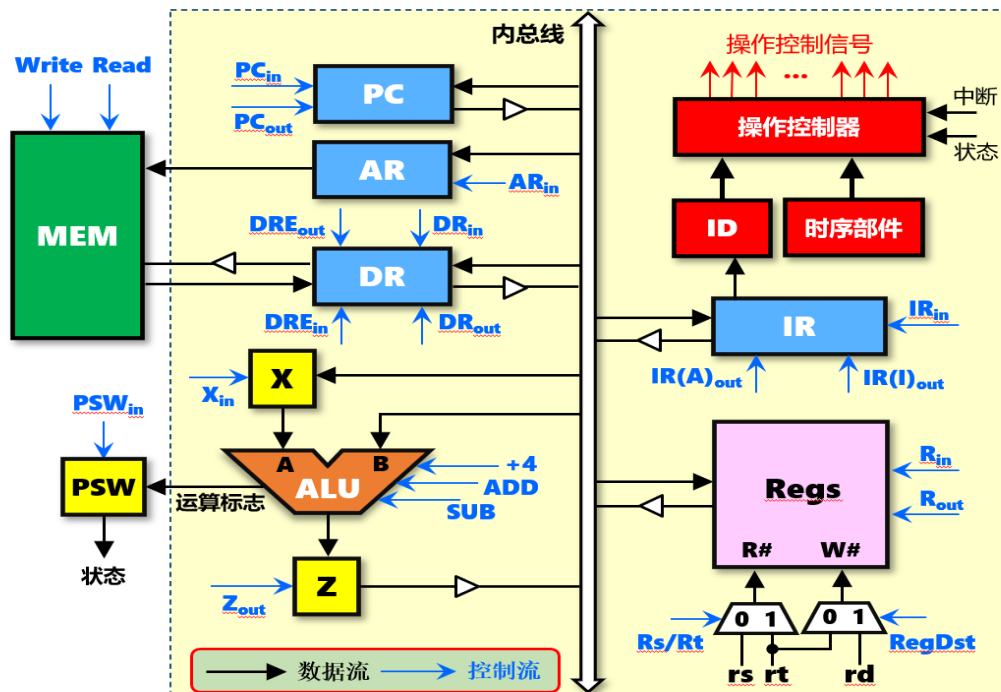


图6.12 addi指令格式

- **addi指令的执行只需要2个机器周期**，其指令操作流程及控制信号如表6.6所示：
 - ① 第一个机器周期：**取指周期**。
 - ② 第二个机器周期：**执行周期**，将寄存器rs和立即数相加，结果送入寄存器rt中。

表6.6 addi指令操作流程及控制信号（仅给出值为1的控制信号）

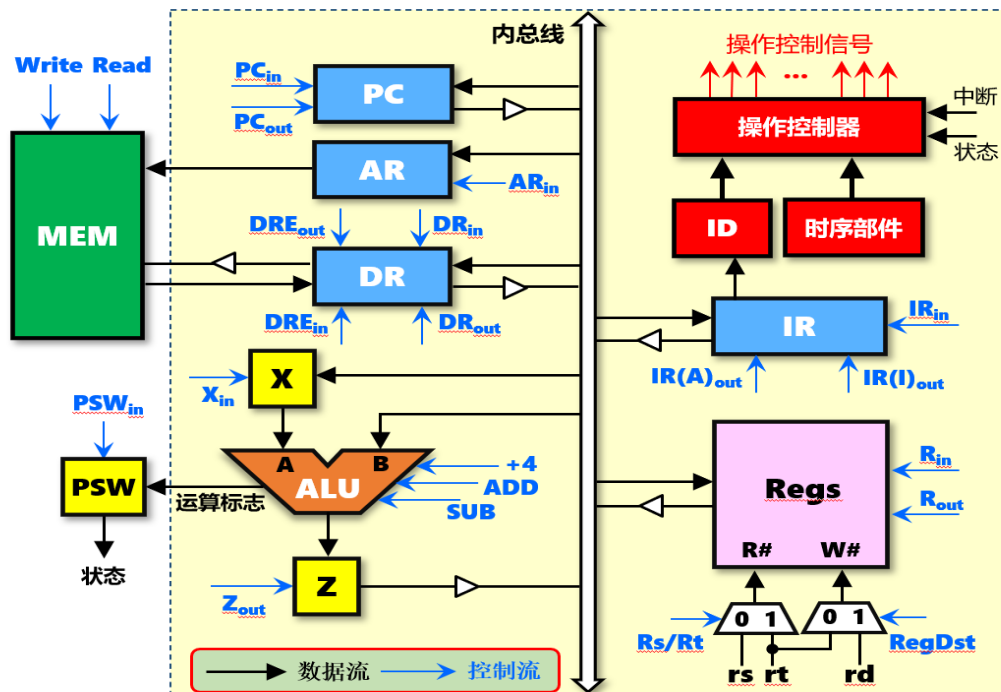
周期	节拍	操作	功能说明	控制信号
取指周期 M_{if}	T_1	$PC \rightarrow AR ; PC \rightarrow X$	将程序计数器PC内容送AR，同时送入暂存器X	$PC_{out}=AR_{in}=X_{in}=1$
	T_2	$X+4 \rightarrow Z$	将PC值加4并送入暂存器Z	$+4=1$
	T_3	$Z \rightarrow PC ; M[AR] \rightarrow DR$	将暂存器Z内容回送到PC 同时读AR内容对应主存单元的值并送入DR	$Z_{out}=PC_{in}=1$ $Read=DRE_{in}=1$
	T_4	$DR \rightarrow IR$	将DR内容送入IR，完成取指令	$DR_{out}=IR_{in}=1$
执行周期 M_{ex}	T_1	$R[rs] \rightarrow X$	将rs寄存器内容送入暂存器X；准备进行加法运算	$R_{out}=X_{in}=1$
	T_2	$IR(I) + X \rightarrow Z$	将IR中的立即数符号扩展为32位后，送入ALU做加法运算	$IR(I)_{out}=ADD=1$
	T_3	$Z \rightarrow R[rt]$	将暂存器Z的结果送入目的寄存器rt	$Z_{out}=R_{in}=1$



周期	节拍	操作	功能说明	控制信号
取指周期 M_{if}	T_1	$PC \rightarrow AR$; $PC \rightarrow X$	将程序计数器PC内容送AR, 同时送入暂存器X	$PC_{out}=AR_{in}=X_{in}=1$
	T_2	$X+4 \rightarrow Z$	将PC值加4并送入暂存器Z	$+4=1$
	T_3	$Z \rightarrow PC$; $M[AR] \rightarrow DR$	将暂存器Z内容回送到PC 同时读AR内容对应主存单元的值并送入DR	$Z_{out}=PC_{in}=1$ $Read=DRE_{in}=1$

addi指令取指周期使用的两条数据通道（与lw指令相同）：

- ① $PC \rightarrow AR \rightarrow MEM \rightarrow DR \rightarrow IR$;以PC为地址访存，取指令，并送入IR
- ② $PC \rightarrow X \rightarrow ALU \rightarrow Z \rightarrow PC$;修改PC的值，为取下一条指令做准备



周期	节拍	操作	功能说明	控制信号
----	----	----	------	------

addi指令执行周期使用的数据通道:

$R[rs] \rightarrow X$; $IR(I) \rightarrow ALU \rightarrow Z \rightarrow R[rt]$; 计算rs寄存器与立即数的和, 并送入rt寄存器

执行周期 M_{ex}	T_2	$IR(I) + X \rightarrow Z$	将IR中的立即数符号扩展为32位后, 送入ALU做加法运算	$IR(I)_{out} = ADD = 1$
	T_3	$Z \rightarrow R[rt]$	将暂存器Z的结果送入目的寄存器rt	$Z_{out} = R_{in} = 1$

– 5、add指令的执行流程

- add指令（**寄存器加法指令**）属于R型指令，汇编代码为：add rd,rt,rs，其指令格式如图6.13所示；add指令的RTL功能描述是： $R[rd] \leftarrow R[rs] + R[rt]$

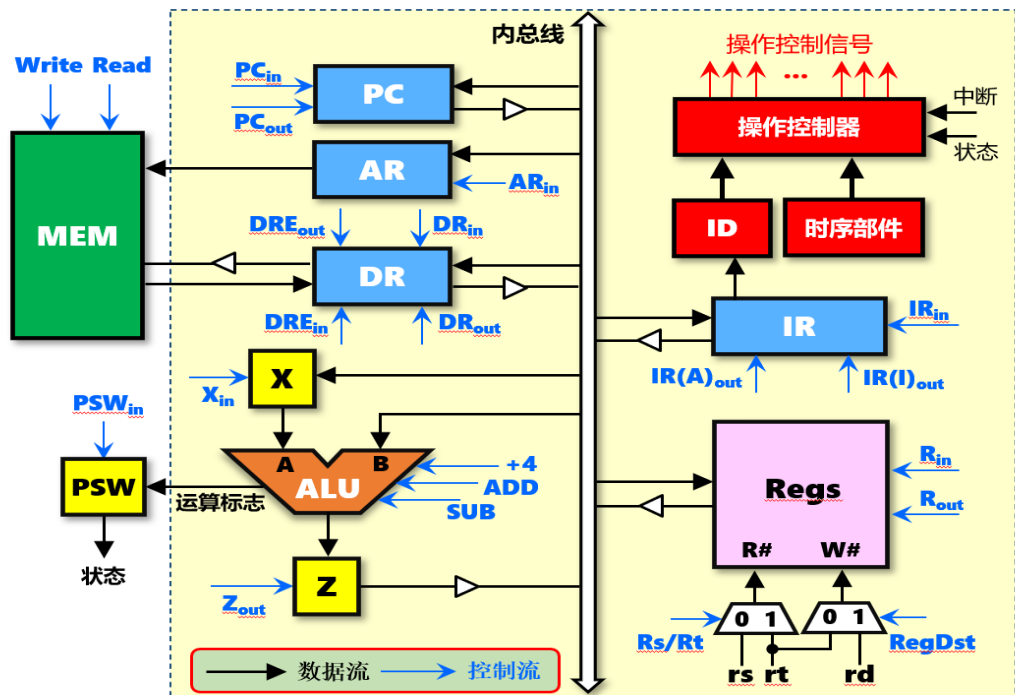


图6.13 add指令格式

- add指令的执行只需要2个机器周期，其指令操作流程及控制信号如表6.7所示：
 - ① 第一个机器周期：取指周期。
 - ② 第二个机器周期：执行周期，将寄存器rs和寄存器rt相加，结果送入寄存器rd中。

表6.7 add指令操作流程及控制信号（仅给出值为1的控制信号）

周期	节拍	操作	功能说明	控制信号
取指周期 M_{if}	T_1	PC $\rightarrow$ AR ; PC $\rightarrow$ X	将程序计数器PC内容送AR，同时送入暂存器X	$PC_{out}=AR_{in}=X_{in}=1$
	T_2	X+4 $\rightarrow$ Z	将PC值加4并送入暂存器Z	+4=1
	T_3	Z $\rightarrow$ PC ; M[AR] $\rightarrow$ DR	将暂存器Z内容回送到PC 同时读AR内容对应主存单元的值并送入DR	$Z_{out}=PC_{in}=1$ $Read=DRE_{in}=1$
	T_4	DR $\rightarrow$ IR	将DR内容送入IR，完成取指令	$DR_{out}=IR_{in}=1$
执行周期 M_{ex}	T_1	$R[rs] \rightarrow X$	将rs寄存器内容送入暂存器X；准备进行加法运算	$R_{out}=X_{in}=1$
	T_2	$R[rt] + X \rightarrow Z$	将rt寄存器内容送入ALU做加法运算，结果送入Z	$R_{out}=Rs/Rt=ADD=1$
	T_3	Z $\rightarrow R[rd]$	将暂存器Z的结果送入目的寄存器rd	$Z_{out}=RegDst=R_{in}=1$

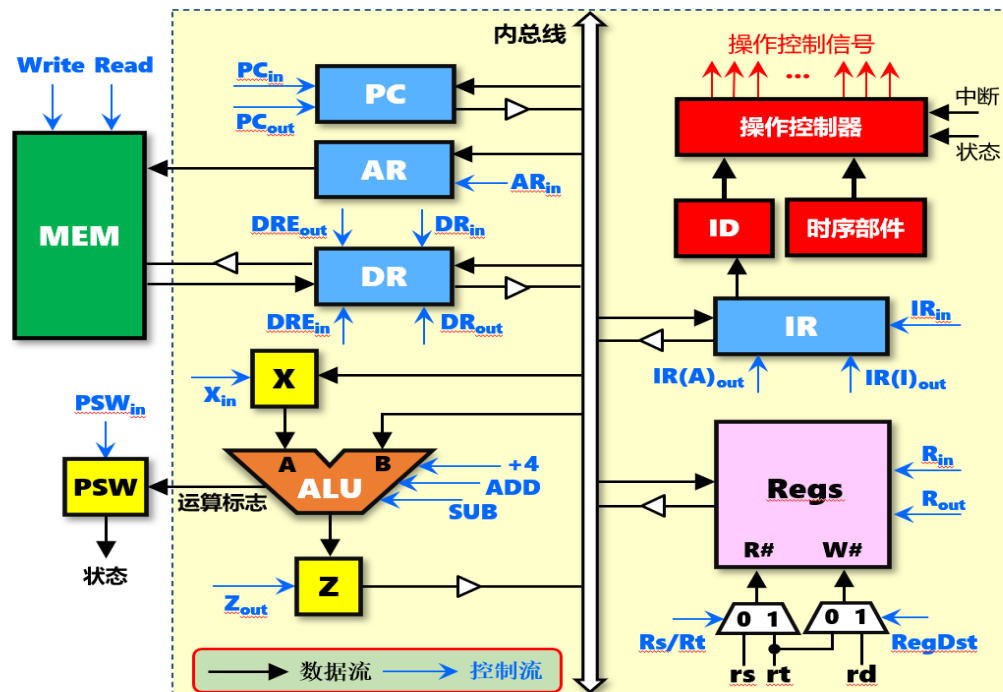


周期	节拍	操作	功能说明	控制信号
取指周期 M_{if}	T_1	PC $\rightarrow$ AR ; PC $\rightarrow$ X	将程序计数器PC内容送AR, 同时送入暂存器X	$PC_{out}=AR_{in}=X_{in}=1$
	T_2	$X+4 \rightarrow Z$	将PC值加4并送入暂存器Z	$+4=1$
	T_3	$Z \rightarrow PC$; $M[AR] \rightarrow DR$	将暂存器Z内容回送到PC 同时读AR内容对应主存单元的值并送入DR	$Z_{out}=PC_{in}=1$ $Read=DRE_{in}=1$
	T_4	$DR \rightarrow IR$	将DR内容送入IR, 完成取指令	$DR_{out}=IR_{in}=1$

add指令取指周期使用的两条数据通道（与lw指令相同）：

① PC $\rightarrow$ AR $\rightarrow$ MEM $\rightarrow$ DR $\rightarrow$ IR ;以PC为地址访存，取指令，并送入IR

② PC $\rightarrow$ X $\rightarrow$ ALU $\rightarrow$ Z $\rightarrow$ PC ;修改PC的值，为取下一条指令做准备



周期	节拍	操作	功能说明	控制信号
----	----	----	------	------

add指令执行周期使用的数据通道:

$R[rs] \rightarrow X \rightarrow ALU$; $R[rt] \rightarrow ALU \rightarrow Z \rightarrow R[rd]$; 计算rs寄存器与rt寄存器的和, 并送入rd寄存器

执行周期 M_{ex}	T_1	$R[rs] \rightarrow X$	将rs寄存器内容送入暂存器X; 准备进行加法运算	$R_{out}=X_{in}=1$
	T_2	$R[rt] + X \rightarrow Z$	将rt寄存器内容送入ALU做加法运算, 结果送入Z	$R_{out}=Rs/Rt=ADD=1$
	T_3	$Z \rightarrow R[rd]$	将暂存器Z的结果送入目的寄存器rd	$Z_{out}=RegDst=R_{in}=1$

– 6、单总线结构性能分析

- 图6.14：单总线结构计算机5条指令的执行流程；每个方框代表一个**时钟周期**（节拍），方框中的内容为该时钟周期内的**操作**。
- 指令执行完毕后，首先判断是否有**中断/异常**（或DMA请求，图6.14中没有画出）；如有中断/异常，将进入**中断周期**（包括：保存断点、修改PC、关中断；具体见第9章）；否则进入下一条指令的取指周期。
- 单总线结构计算机的最小时钟周期（最大时钟频率），取决于不同机器周期、不同时钟节拍中**最慢**的关键路径时间延迟；通常，**访存通路（M[AR]→DR；DR→M[AR]）和运算通路（ALU→Z）时间延迟最长（最慢）**；因此最小时钟周期应为：

$$T_{\min_clk} = T_{clk_to_q} + \max(T_{alu}, T_{mem}) + T_{setup}$$

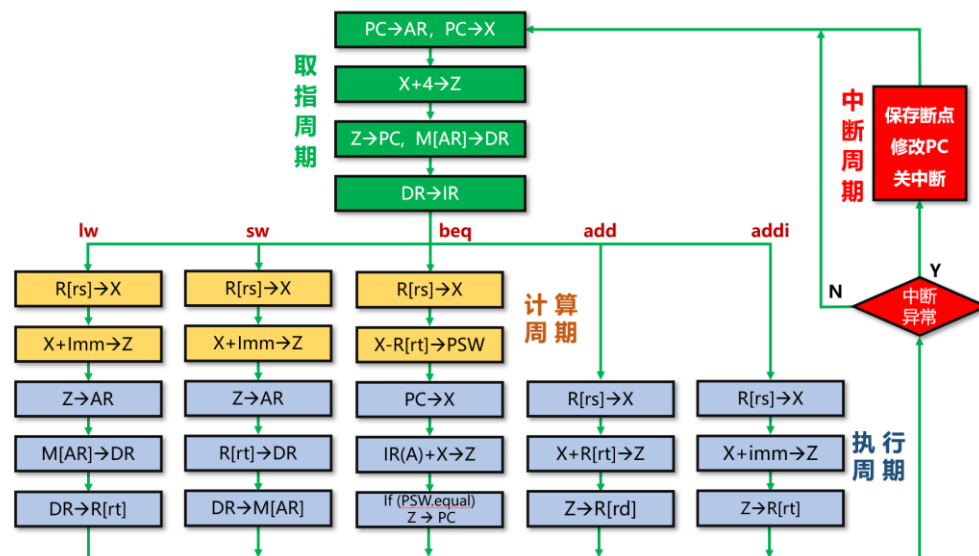


图6.14 单总线结构数据通路指令方框图

6.3.2 单总线结构的数据通路

- 例6.1: SPECINT2000基准测试程序包含的取数据指令、存数据指令、条件分支指令、跳转指令、R型算术逻辑运算指令比例分别为: 25%、10%、11%、2%、52%。求此基准测试程序在图6.8所示的单总线结构计算机上运行的CPI。假设SPECINT2000基准测试程序的指令数目为1000亿条。如果CPU采用65nm CMOS工艺实现, 各功能部件的时间延迟如表6.8所示, 求该计算机最大时钟频率, 以及基准测试程序的执行时间。

表6.8 各功能部件的时间延迟

功能部件	参数	延迟	功能部件	参数	延迟
寄存器延迟	$T_{\text{clk_to_q}}$	30ps	运算器ALU	T_{alu}	200ps
存储器读	T_{mem}	250ps	多路选择器	T_{mux}	25ps
寄存器堆读	$T_{\text{RF_read}}$	150ps	寄存器建立时间	T_{setup}	20ps

- 例6.1：求此基准测试程序在图6.8所示的单总线结构计算机上运行的CPI。假设SPECINT2000基准测试程序的指令数目为1000亿条。如果CPU采用65nm CMOS工艺实现，各功能部件的时间延迟如表6.8所示，求该计算机最大时钟频率，以及基准测试程序的执行时间。

表6.8 各功能部件的时间延迟

功能部件	参数	延迟	功能部件	参数	延迟
寄存器延迟	$T_{\text{clk_to_q}}$	30ps	运算器ALU	T_{alu}	200ps
存储器读	T_{mem}	250ps	多路选择器	T_{mux}	25ps
寄存器堆读	$T_{\text{RF_read}}$	150ps	寄存器建立时间	T_{setup}	20ps

- 解：1ps=10¹²
 - 由图6.14可知，执行取数据指令、存数据指令、条件分支指令、R型算术逻辑运算指令的时钟周期数分别为：9、9、9、7；跳转指令属于无条件转移指令，无须计算分支条件，即没有计算周期，因此，执行跳转指令的时钟周期数为7。

J address;
PC <- {(PC+4)_{31:28}, address, 00}

- CPI即指令执行的时钟周期数，因此该测试程序的CPI为：

$$\text{CPI} = 0.25 \times 9 + 0.1 \times 9 + 0.11 \times 9 + 0.02 \times 7 + 0.52 \times 7 = 7.92$$

- 根据表6.8，可以计算最小时钟周期：

$$T_{\text{min_clk}} = T_{\text{clk_to_q}} + \max(T_{\text{alu}}, T_{\text{mem}}) + T_{\text{setup}} = 30 + \max(200, 250) + 20 = 300\text{ps}$$

- 因此，最大时钟频率为：

$$T_{\text{max_freq}} = 1/300\text{ps} = 3.33\text{GHz}$$

- 基准测试程序的执行时间为：

$$T_{\text{total}} = \text{指令条数} \times \text{CPI} \times T_{\text{min_clk}} = 1000 \times 10^8 \times 7.92 \times 300 \times 10^{-12} = 237.6\text{s}$$

6.3.3 专用通路结构的数据通路

- 采用专用通路的CPU中，各寄存器之间或寄存器与ALU之间，均基于专用的数据传输通路连接，而非基于总线方式共享连接，各通路中的数据可并行传输，控制起来比总线结构的CPU要简单。
- 图6.8：单总线结构的计算机框图
- 图6.25：单周期MIPS处理器的数据通路（采用专用通路结构）

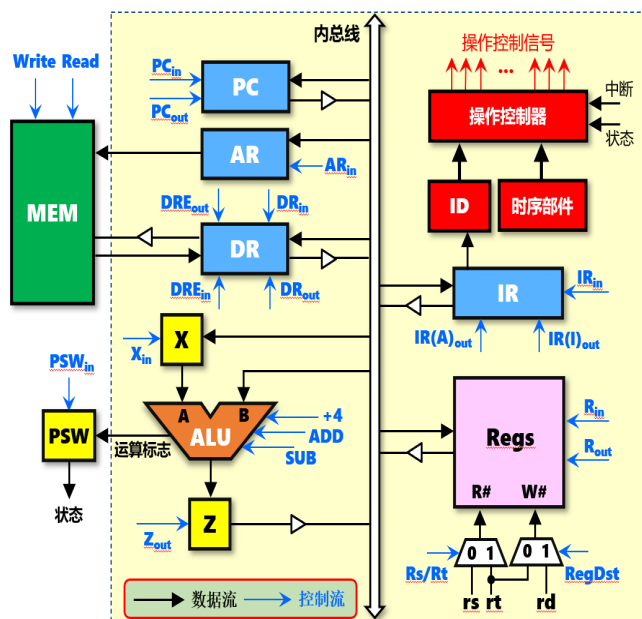


图6.8 单总线结构的计算机框图

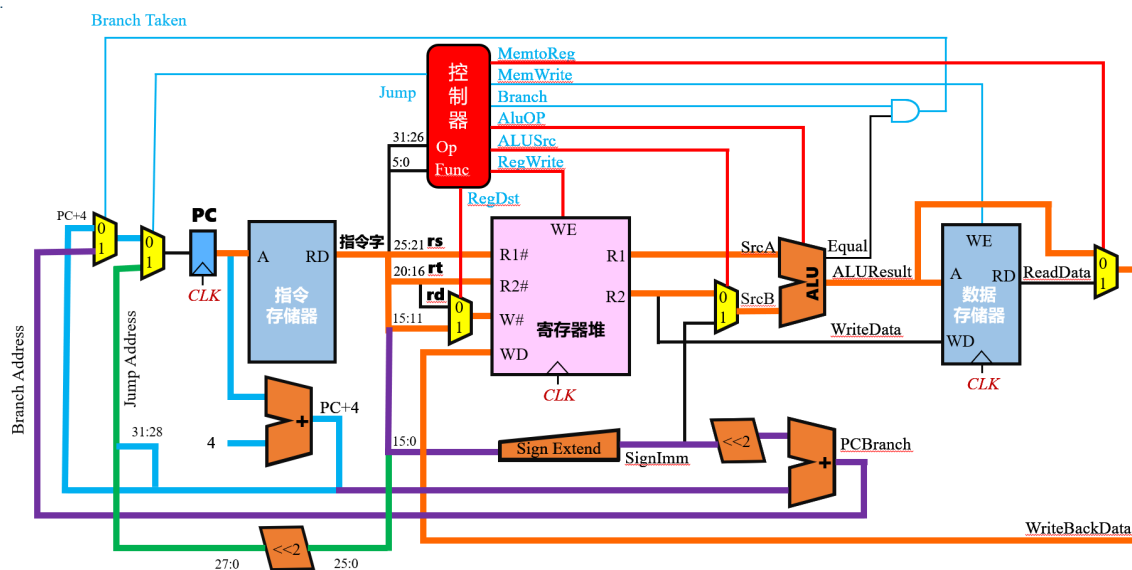
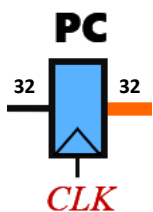


图6.25 单周期MIPS处理器的数据通路

– 1、数据通路基本功能部件

- (1) 程序计数器PC
 - 采用32位寄存器实现；顺序执行时： $PC=PC+4$
- (2) 指令存储器
 - 采用ROM实现；输入为32位地址，输出为32位MIPS指令字
- (3) 数据存储器
 - 采用RAM实现（分离模式）；输入为32位地址、32位数据，输出为32位数据，WE为读写控制信号（WE=0，读；WE=1，写），CLK为时钟信号
- (4) 寄存器堆
 - 32个32位寄存器，有2个读出口、1个写入口；输入为2个读出口的寄存器编号（均为5位）、1个写入口的寄存器编号（5位）、写入数据（32位），输出为2个读出口数据（均为32位），WE为读写控制信号（WE=0，读；WE=1，写），CLK为时钟信号



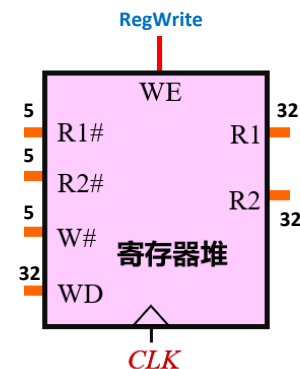
(a) 程序计数器



(b) 指令存储器



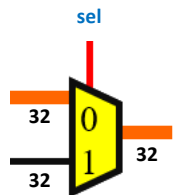
(c) 数据存储器



(d) 寄存器堆

图6.15 数据通路中的存储部件

- (5) 多路选择器
 - 2路选择器；sel=0，输出=左上角的输入；sel=1，输出=左下角的输入；还有4路选择器、8路选择器等等
- (6) 符号扩展单元
 - 将16位的数扩展为32位的数；例如：输入=1234H，输出=00001234H；输入=8234H，输出=FFFF8234H
- (7) 移位运算器、加法器和ALU
 - “<<2”表示左移2位运算；例如：输入=12345678H=0001 0010 0011 0100 0101 0110 0111 1000，输出=0100 1000 1101 0001 0101 1001 1110 0000=48D1 59E0H
 - 加法器：如顺序指令地址的PC+4计算，分支目标地址的计算
 - ALU：实现指令的算术逻辑运算功能，具体什么运算由AluOp决定；如果两个输入相等，则equal=1



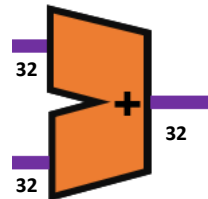
(a) 多路选择器



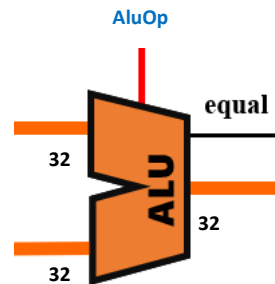
(b) 符号扩展器



(c) 移位运算器



(d) 加法器



(e) ALU

图6.16 数据通路中的组合逻辑部件

– 2、单周期处理器典型数据通路

- 图6.25为采用专用数据通路的单周期MIPS处理器结构；单周期处理器是指每条指令在一个时钟周期内执行完成，即一个指令周期只能执行一条指令。
- 由于不同的指令，其执行时间是不同的（如图6.14中的5条指令，执行时间分别为9、9、9、7、7个节拍）；根据木桶原理，单周期处理的时钟周期取决于执行速度最慢的指令。
- 此外，由于是在一个时钟周期内完成指令的取出和执行操作，指令执行过程中数据通路的任何资源都不能被重复使用，都应该是专用的数据通路；需要被多次使用的资源（如加法器，图6.25中有2个）都需要设置多个；取指令和取操作数都需要访问存储器，因此将指令和数据分别存放在两个存储器中（指令存储器、数据存储器）。

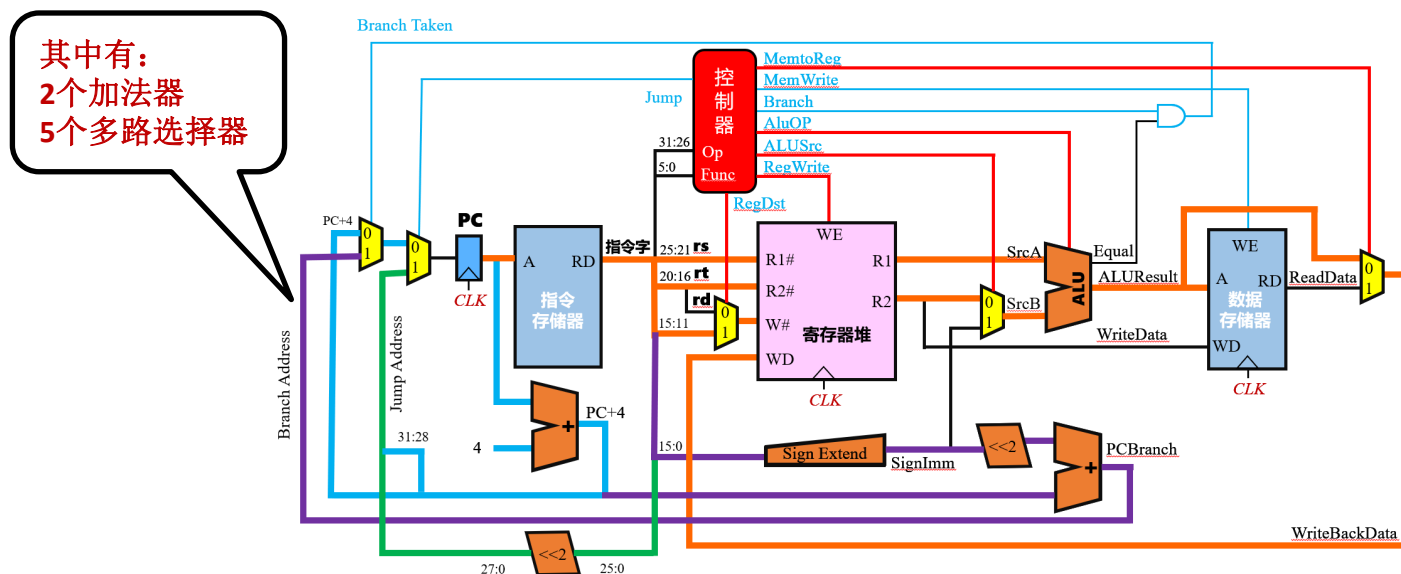


图6.25 单周期MIPS处理器的数据通路

- (1) 取指令的数据通路

- 包括程序计数器PC、加法器、指令存储器（图6.17）。
- 加法器用于完成PC+4（因为MIPS 32每条指令的长度都是32位），即计算下一条指令的地址。
- 程序计数器PC的写入受时钟信号CLK的控制（上跳沿）。

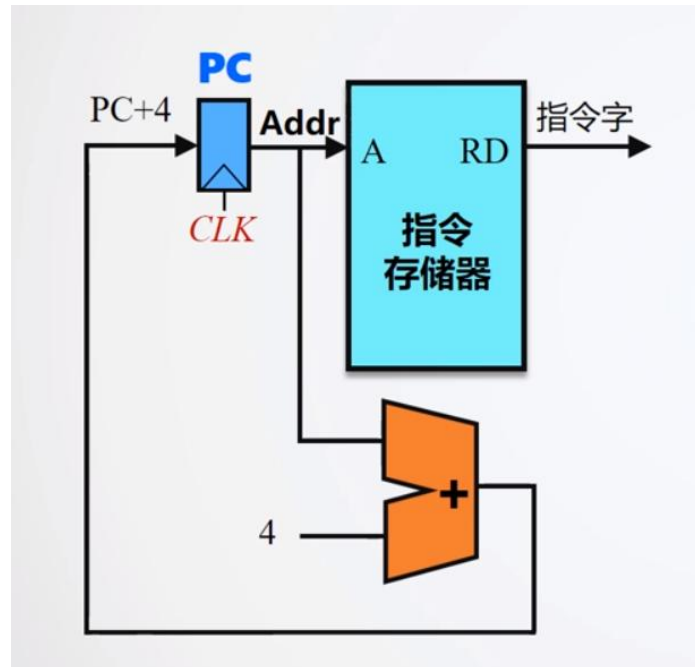


图6.17 取指令数据通路

- (2) R型运算类指令数据通路

- 例如加法指令就属于R型运算类指令：**add rd,rs,rt**；数据通路包括**程序计数器PC**、**指令存储器**、**寄存器堆**（寄存器组）和**ALU**等功能部件（图6.18）。



- 将指令字中的源寄存器字段rs、rt分别送寄存器堆的R1#和R2#，将目的寄存器字段rd送寄存器堆的W#；从寄存器堆读出的R1和R2送到ALU；指令字中的**funct**字段决定ALU进行加法运算（发出**AluOP**控制信号，加法运算）；运算结果送寄存器堆的输入数据口WD（发出**RegWrite**控制信号，寄存器写，受时钟信号CLK控制（上跳沿）），从而完成“ $R[rs]+R[rt] \rightarrow R[rd]$ ”的功能。

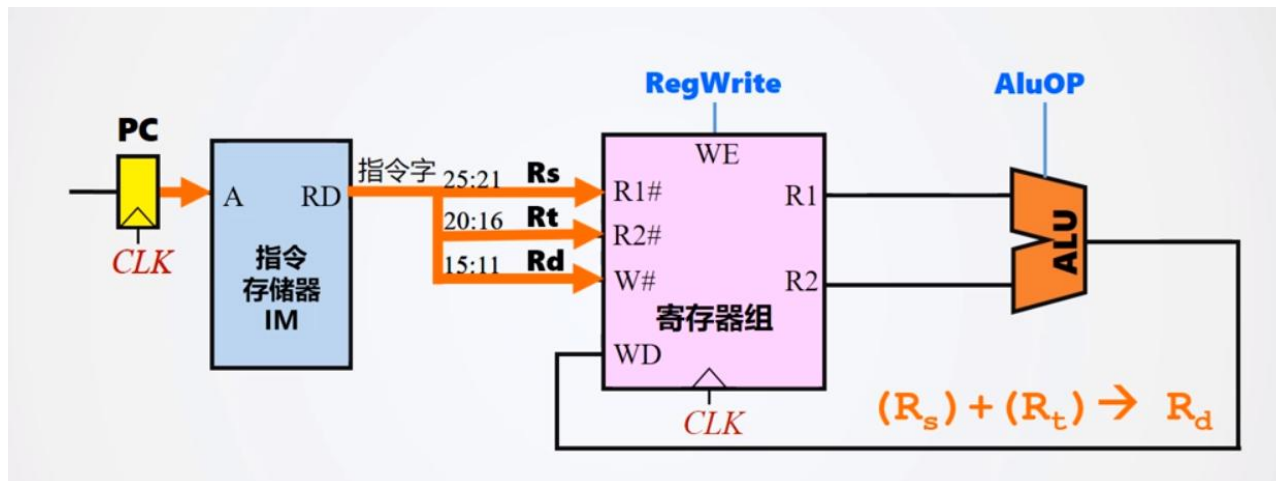


图6.18 R型运算类指令的数据通路

- **(3) I型运算类指令数据通路**

- 例如取数指令lw（访存指令）就属于I型指令：lw rt,imm16(rs)；数据通路包括程序计数器PC、指令存储器、寄存器堆（寄存器组）、符号扩展器、ALU、数据存储器等功能部件（图6.19）。



- 将指令字中的源寄存器字段rs送寄存器堆的R1#，目的寄存器字段rt送寄存器堆的W#，立即数字段imm16送符号扩展器（16位扩展为32位）；从寄存器堆读出的R1，与符号扩展器的输出，一起送到ALU，相加后（发出AluOP控制信号，加法运算）形成数据存储器地址；从数据存储器中读取数据，送到寄存器堆的WD口（发出RegWrite控制信号，寄存器写，受时钟信号CLK控制（上跳沿）），从而完成“M[R[rs] + SignExt(imm16)] -> R[rt]”的功能。

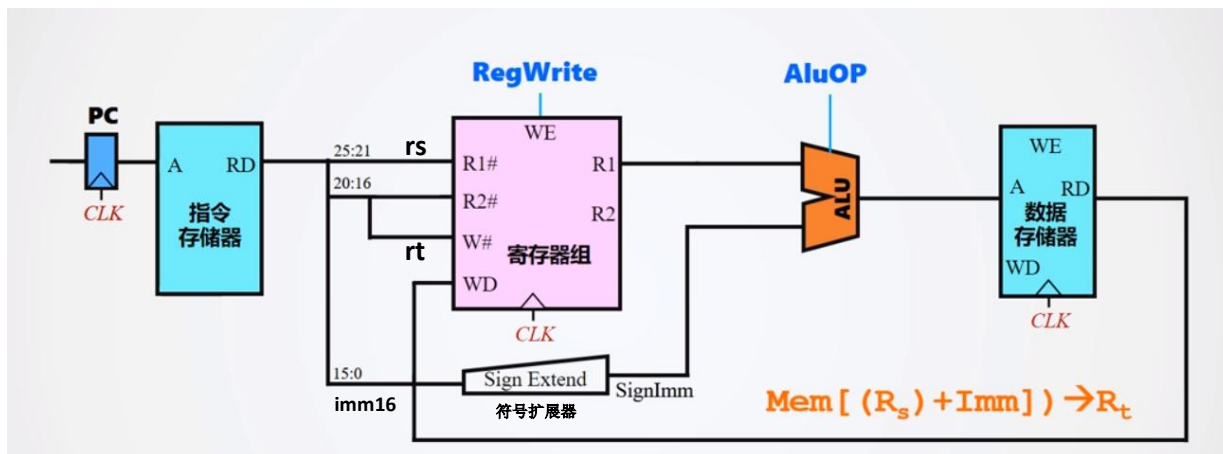


图6.19 lw指令的数据通路

- 例如存数指令`sw`（访存指令）就属于I型指令：`sw rt,imm16(rs)`；数据通路包括程序计数器PC、指令存储器、寄存器堆（寄存器组）、符号扩展器、ALU、数据存储器等功能部件（图6.20）。



- 将指令字中的源寄存器字段`rs`、`rt`分别送寄存器堆的 $R1\#$ 、 $R2\#$ ，立即数字段`imm16`送符号扩展器（16位扩展为32位）；从寄存器堆读出的 $R1$ ，与符号扩展器的输出，一起送到ALU，相加后（发出`AluOP`控制信号，加法运算）形成数据存储器的地址；从寄存器堆中读出 $R2$ ，送数据存储器的`WD`（发出`MemWrite`控制信号，数据存储器写，受时钟信号`CLK`控制（上跳沿）），从而完成“ $R[rt] \rightarrow M[R[rs] + \text{SignExt}(imm16)]$ ”的功能。

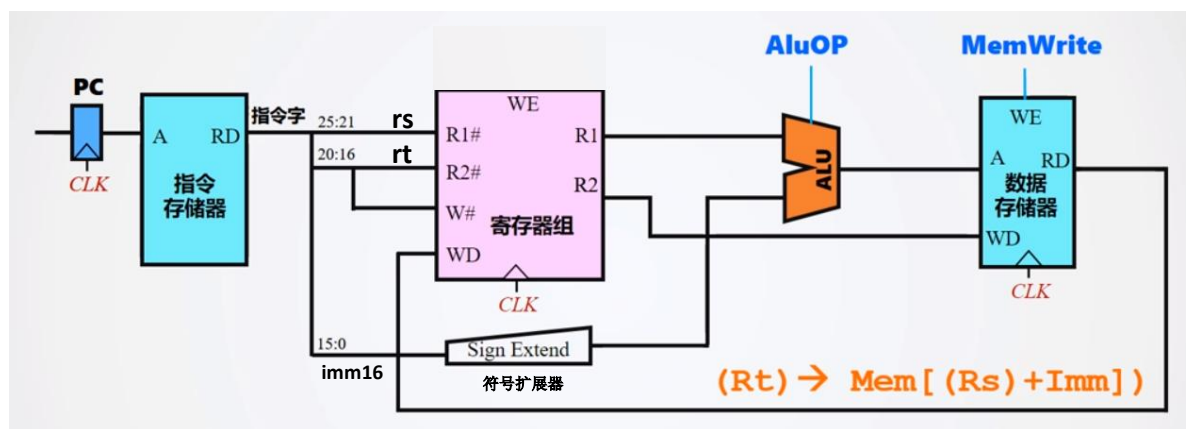


图6.20 `sw`指令的数据通路

- 可以将R型运算类指令和访存指令的数据通路结合起来，构成支持R型运算类指令和访存指令的混合数据通路（图6.21）。
- 该数据通路（图6.21）既可以完成R型运算类指令（图6.18），也可以完成取数指令lw（图6.19）和存数指令sw（图6.20）。
- 图6.21中增加了3个多路选择器（二路选择器）：
 - » 第一个多路选择器：RegDst=0，W#=rt，用于lw指令；RegDst=1，W#=rd，用于R型运算类指令。
 - » 第二个多路选择器：AluSrc=0，SrcB=R2，用于R型运算类指令；AluSrc=1，SrcB=符号扩展器的输出SignImm，用于lw指令或sw指令。
 - » 第三个多路选择器：MemToReg=0，寄存器堆的WD=ALUResult，用于R型运算类指令；MemToReg=1，寄存器堆的WD=数据存储器输出ReadData，用于lw指令。

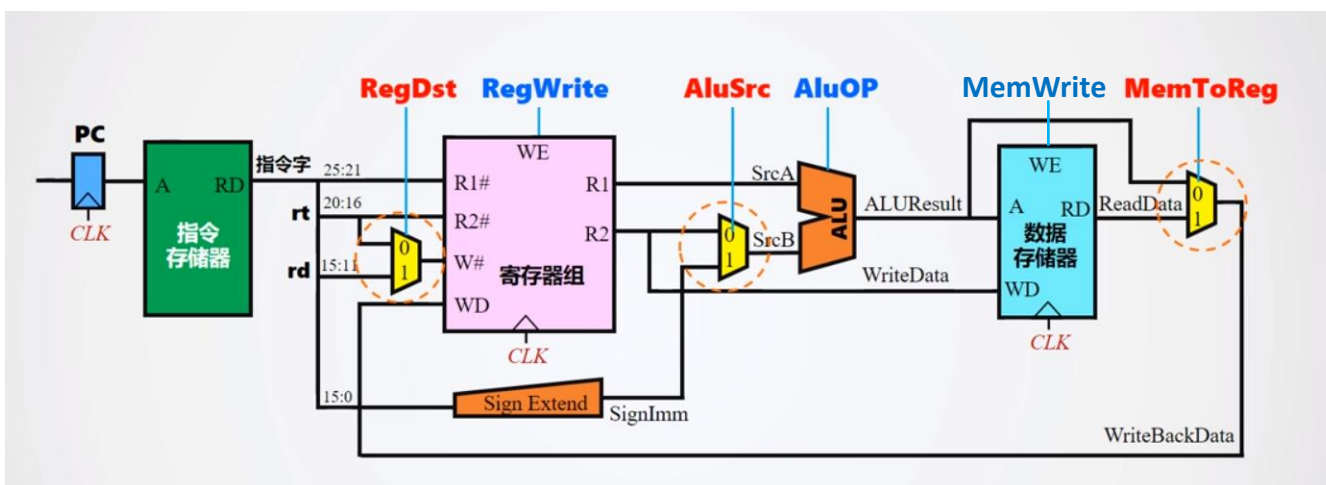


图6.21 支持R型运算类指令和访存指令的混合数据通路

- **(4) 条件分支指令数据通路**

- 条件分支指令也属于I型指令，例如beq指令：**beq rs,rt,imm16**；数据通路包括**程序计数器PC、指令存储器、寄存器堆（寄存器组）、符号扩展器、移位运算器、ALU、数据存储器、2个加法器、4个多路选择器、与门**等功能部件（图6.22）。



- 将立即数字段imm16送符号扩展器，扩展为32位，然后通过移位运算器左移2位，再经过加法器后，得到条件成立后的分支目标地址（ $PC + 4 + \text{SignExt}(\text{imm}) \ll 2$ ）；将指令字中的寄存器字段rs、rt送寄存器堆的R1#、R2#，寄存器堆的输出R1、R2送ALU，进行比较运算；如果rs=rt，则Equal=1；Branch为beq的译码信号，故Branch=1，与门的输出Branch Taken=1；因此，将 $PC + 4 + \text{SignExt}(\text{imm}) \ll 2$ 送程序计数器PC；如果rs≠rt，则Equal=0，Branch Taken=0，将PC+4送程序计数器PC；从而完成“if(R[rs] == R[rt]) PC + 4 + SignExt(imm16) << 2 -> PC”的功能。

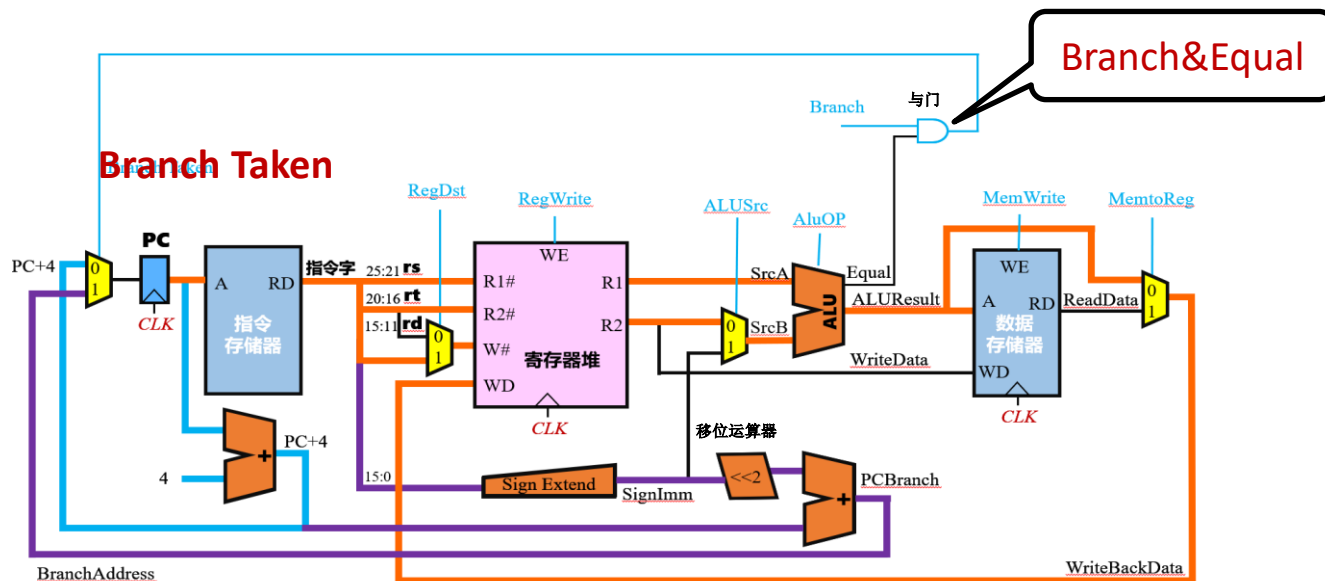


图6.22 beq指令的数据通路

- **(5) 无条件分支指令数据通路**

- 无条件分支指令属于J型指令，例如j指令：**j address**；数据通路包括**程序计数器PC**、**指令存储器**、**寄存器堆（寄存器组）**、**符号扩展器**、**2个移位运算器**、**ALU**、**数据存储器**、**2个加法器**、**5个多路选择器**、**与门**等功能部件。



- 将指令字中的26位地址address字段通过移位运算器左移2位，得到28位地址；再与PC+4的高4位（31:28位）拼接成32位跳转地址（Jump Address），送多路选择器；j指令经过译码，使Jump=1，多路选择器将Jump Address送PC；从而完成“ $\{(PC+4)_{31:28}, address \ll 2\} \rightarrow PC$ ”的功能。

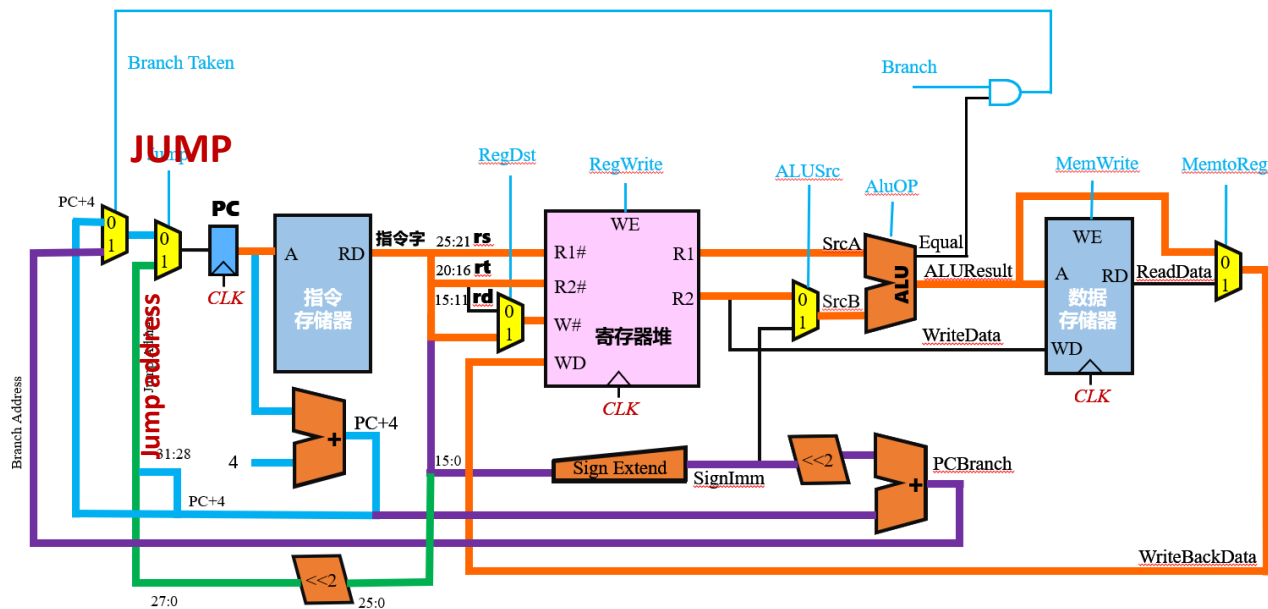


图6.24 j指令的数据通路

• (6) 单周期数据通路的操作控制

- 将前面的各类指令数据通路合并，并增加必要的控制线路（**控制器**），就可以得到能支持MIPS指令系统子集（部分指令）的完整数据通路（图6.25）。

– 图6.25中：

- ① 程序计数器PC、寄存器堆、数据存储寄存器受统一时钟信号**CLK**控制，上跳沿有效。
- ② 指令存储器为ROM，不用设置指令寄存器IR。
- ③ ALU的操作控制信号**AluOP**为4位，可以有16种运算；**AluOP**的值由指令字的**OP**字段和**funct**字段经译码产生。
- ④ 单周期MIPS处理器的操作控制器（控制器）是**组合逻辑电路**；控制器的输入为指令的**OP**字段和**funct**字段，控制器的输出为8个控制信息，具体见表6.9。

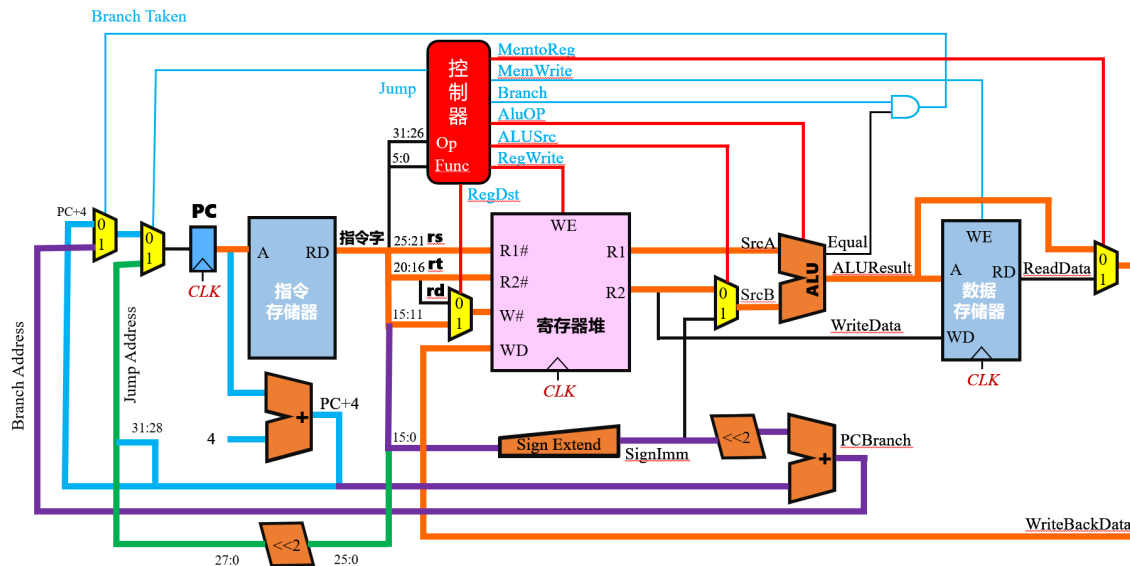


图6.25 单周期MIPS处理器的数据通路

表6.9 单周期MIPS处理器的控制信号及功能

控制信号	信号分类	功能说明
Jump	指令译码信号	j、jal等无条件分支指令时生成
Branch	指令译码信号	分支指令译码信号，beq、bne指令需要产生类似的译码信号
RegDst	多路选择控制	写入目的寄存器的选择，为1时写入rd寄存器，为0时写入rt寄存器
AluSrc	多路选择控制	控制ALU的第二输入，为0时输入R2（rt寄存器），为1时输入扩展后的立即数
MenToReg	多路选择控制	为1时将从数据存储器读出的数据写回寄存器堆，为0时将ALU的运算结果写回寄存器堆
RegWrite	功能部件控制	控制寄存器堆写操作，为1时数据需要写入指定的寄存器，受时钟驱动
AluOp	功能部件控制	控制ALU进行不同的运算，具体取指和位宽（如AluOP为4位）与ALU的设计有关
MemWrite	功能部件控制	控制数据存储器的写操作，为1时进行写操作，为0时读操作，受时钟驱动

• (7) 单周期处理器性能分析

- 单周期MIPS处理器每个时钟周期执行一条指令， $CPI=1$ ；看似性能很好，但是由于时钟周期由执行速度最慢的指令决定；因此单周期处理器的时钟周期不能太短（主频不能太高），从而导致程序执行效率较低。

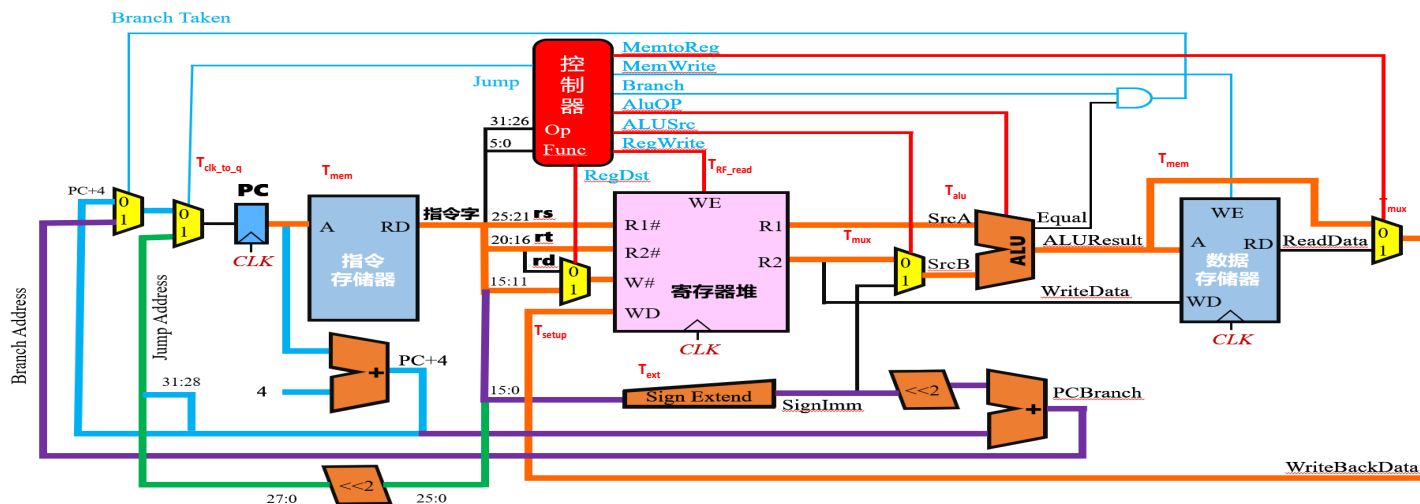


图6.26 lw指令数据通路的关键延迟

- 通常lw指令的执行时间最长，其关键延迟如图6.26所示，包括：

- ① 程序计数器PC的触发器延迟： $T_{clk_to_q}$
- ② 指令存储器的访存延迟： T_{mem}
- ③ 寄存器堆的读延迟： T_{RF_read}
- ④ 符号扩展单元的延迟： T_{ext}
- ⑤ SrcB端的多路选择器的延迟： T_{mux}
- ⑥ 运算器的延迟： T_{alu}
- ⑦ 数据存储器的访存延迟： T_{mem}
- ⑧ 数据存储器输出端的多路选择器的延迟： T_{mux}
- ⑨ 写入寄存器堆的寄存器建立时间： T_{setup}

数据通路中的关键路径
(即最长路径)

所以，单周期MIPS处理器的最小时钟周期为：

$$T_{\min_clk} = T_{clk_to_q} + T_{mem} + \max(T_{RF_read}, T_{ext} + T_{mux}) + T_{alu} + T_{mem} + T_{mux} + T_{setup};$$

寄存器读与符号扩展后送MUX属于并行通路，作为操作数进入ALU，因而从关键路径角度，要选择延迟时间最长的，即MAX

通常： $T_{RF_read} > T_{ext} + T_{mux}$

因此： $T_{\min_clk} = T_{clk_to_q} + 2T_{mem} + T_{RF_read} + T_{alu} + T_{mux} + T_{setup}$

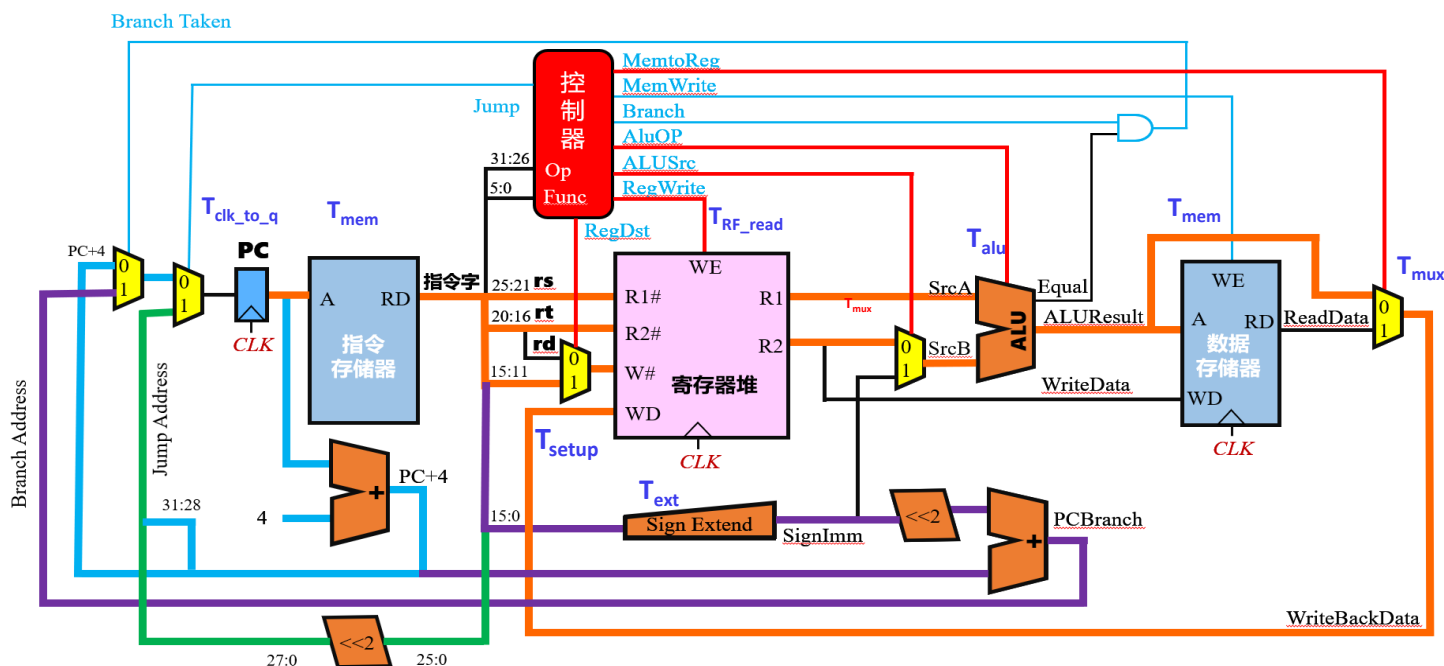


图6.26 lw指令数据通路的关键延迟

- 例6.2：求例6.1中基准测试程序SPECINT2000在单周期MIPS处理器上运行的CPI。假设程序的指令数目为1000亿条，CPU采用65nm CMOS工艺实现，各功能部件的时间延迟如表6.8所示，求该计算机最大时钟频率，以及基准测试程序的执行时间。

表6.8 各功能部件的时间延迟

功能部件	参数	延迟	功能部件	参数	延迟
寄存器延迟	$T_{\text{clk_to_q}}$	30ps	运算器ALU	T_{alu}	200ps
存储器读	T_{mem}	250ps	多路选择器	T_{mux}	25ps
寄存器堆读	$T_{\text{RF_read}}$	150ps	寄存器建立时间	T_{setup}	20ps

- 解：
 - 单周期MIPS处理器的最小时钟周期： $T_{\text{min_clk}} = T_{\text{clk_to_q}} + 2T_{\text{mem}} + T_{\text{RF_read}} + T_{\text{alu}} + T_{\text{mux}} + T_{\text{setup}} = 30 + 2 \times 250 + 150 + 200 + 25 + 20 = 925\text{ps}$
 - 因此，最大时钟频率为： $T_{\text{max_freq}} = 1/925\text{ps} = 1.08\text{GHz}$
 - 单周期处理器的CPI=1
 - 基准测试程序的执行时间为： $T_{\text{total}} = \text{指令条数} \times \text{CPI} \times T_{\text{min_clk}} = 1000 \times 10^8 \times 1 \times 925 \times 10^{-12} = 92.5\text{s}$

比较：

单总线处理器：

$$\text{CPI} = 0.25 \times 9 + 0.1 \times 9 + 0.11 \times 9 + 0.02 \times 7 + 0.52 \times 7 = 7.92$$

$$\text{最小时钟周期: } T_{\text{min_clk}} = T_{\text{clk_to_q}} + \max(T_{\text{alu}}, T_{\text{mem}}) + T_{\text{setup}} = 30 + \max(200, 250) + 20 = 300\text{ps}$$

$$\text{最大时钟频率为: } T_{\text{max_freq}} = 1/300\text{ps} = 3.33\text{GHz}$$

$$\text{基准测试程序的执行时间为: } T_{\text{total}} = \text{指令条数} \times \text{CPI} \times T_{\text{min_clk}} = 1000 \times 10^8 \times 7.92 \times 300 \times 10^{-12} = 237.6\text{s}$$

- 单周期MIPS处理器存在3个问题：

- ① 第一是时钟周期长，程序执行效率不高。

- ② 第二是硬件实现效率不高，例如，为了在ALU运算的同时计算指令顺序地址和分支目标地址，增设了2个加法器，而加法器的电路是相对占用晶圆面积较多的电路。

- ③ 第三是指令存储器和数据存储器分离结构与实际存储系统并不相符。

- 现代计算机已不再采用单周期方式设计处理器，而是采用多周期处理器，或者更高级的指令流水线（第7章）。

– 3、多周期处理器典型数据通路

• (1) 多周期数据通路及控制信号

- **单周期处理器**，一条指令的执行只需要一个时钟周期；**多周期处理器**，一条执行的执行需要多个时钟周期。
- 图6.8的**单总线结构**的计算机，本质上就是一种多周期处理器；例如执行lw指令需要9个时钟周期、执行addi指令需要7个时钟周期（图6.14）。

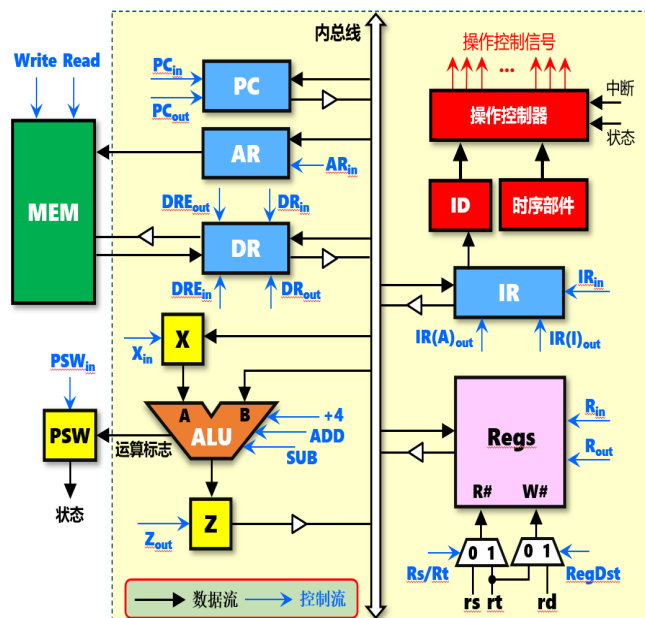


图6.8 单总线结构的计算机框图

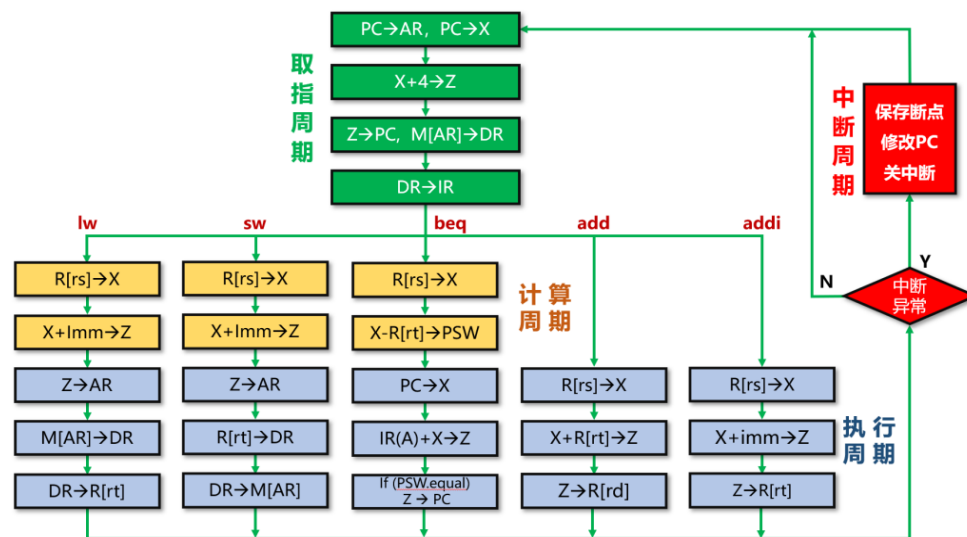


图6.14 单总线结构数据通路指令方框图

- 图6.27是由单周期MIPS处理器的数据通路（图6.25），改造而来的采用专用通路的多周期MIPS处理器的数据通路。

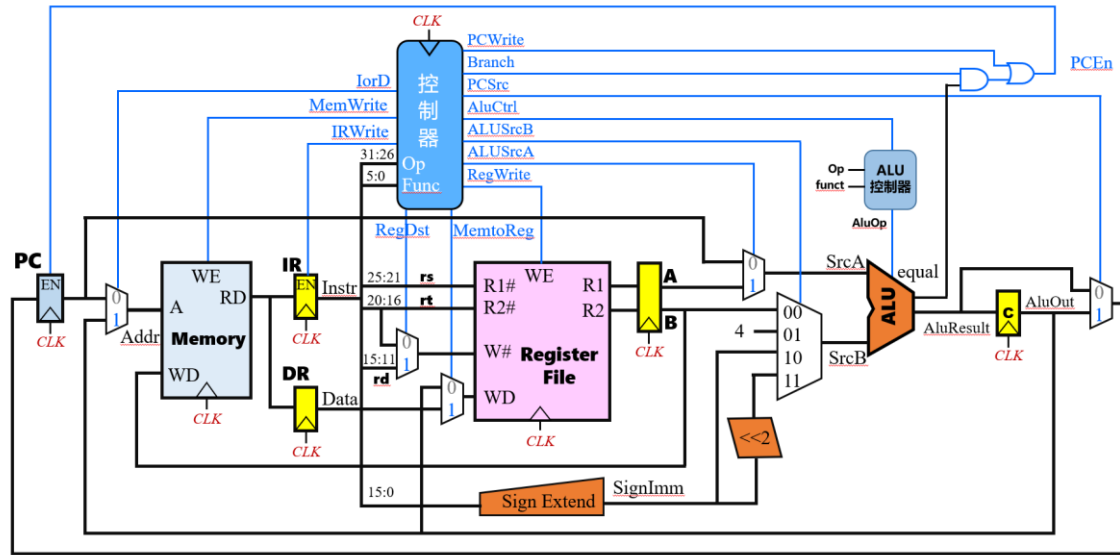


图6.27 多周期MIPS处理器的数据通路

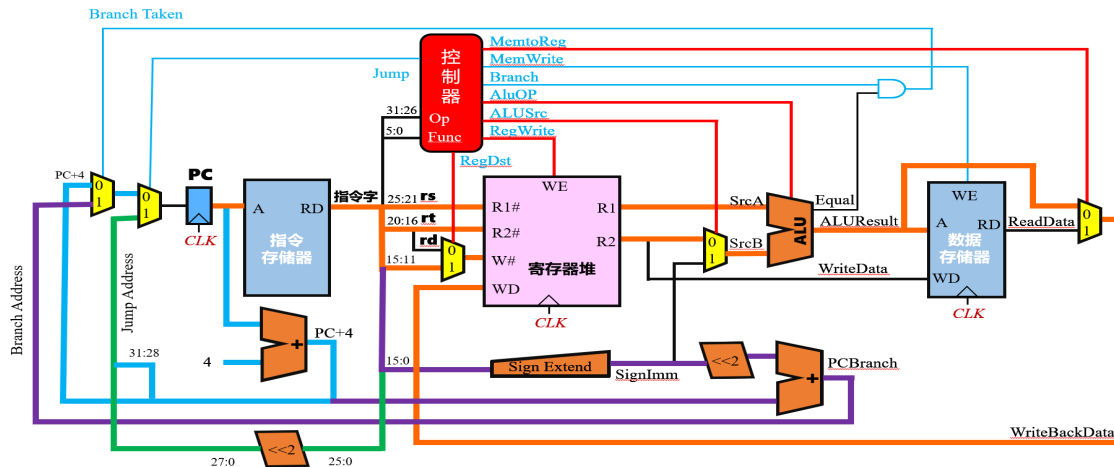


图6.25 单周期MIPS处理器的数据通路

— 多周期处理器结构与单周期处理器结构有如下的不同：

- 不再区分指令存储器和数据存储器，指令和数据保存的同一个存储器中（现代的计算机就是这样，这个存储器就是内存条）。
- 部分功能单元，如ALU、寄存器堆可在一条指令执行过程的不同时钟周期中多次使用，从而不需要额外设置加法器（图6.27中，只有一个ALU，没有另外的2个加法器）。
- 主要功能部件输出端都增加了一些附加寄存器（5个），方便暂存当前时钟周期加工处理的数据给后续时钟周期使用：
 - ① 指令寄存器IR：用于暂存从存储器中读出的指令，需要设置专门的写使能控制信号Irwrite。
 - ② 数据寄存器DR：用于暂存从存储器中读出的数据，不需要设置写使能控制信号。
 - ③ 寄存器A、寄存器B：用于暂存从寄存器堆读出的数据，不需要设置写使能控制信号。
 - ④ 寄存器C：用于暂存ALU的运算结果，不需要设置写使能控制信号。
- 程序计数器PC增加了使能端PCEn，由写操作控制信号PCwrite通过一定的逻辑进行控制。
- 增加和扩展了部分多路选择器（6个）：
 - ① 存储器前的多路选择器用于区分是指令地址还是数据地址。
 - ② 寄存器堆前的多路选择器（W#）用于确定寄存器堆的写入编号W#是rt还是rd。
 - ③ 寄存器堆前的多路选择器（WD）用于确定寄存器堆的写入数据WD是来自ALU还是来自存储器。
 - ④ ALU前的多路选择器（SrcA）用于确定ALU的输入A是来自PC还是来自寄存器A。
 - ⑤ ALU前的多路选择器（SrcB，4选1）用于确定ALU的输入B是来自寄存器B（ $R[rs]+R[rt] \rightarrow R[rd]$ ）、来自常数4（ $PC+4 \rightarrow PC$ ）、来自符号扩展输出（ $M[R[rs] + \text{SignExt}(\text{imm16})] \rightarrow R[rt]$ ）、来自符号扩展输出左移2位（ $PC+4 + \text{SignExt}(\text{imm16}) \ll 2 \rightarrow PC$ ）。
 - ⑥ ALU后的多路选择器用于确定程序计数器PC的输入是来自“PC+4”还是来自分支目标地址（ $PC+4 + \text{SignExt}(\text{imm16}) \ll 2; \{(PC+4)_{31:28}, \text{Address} \ll 2\} \rightarrow PC$ ）。
- 增加了ALU控制器，其输入为AluCtrl、Op、funct，输出为AluOP；当AluCtrl为0时，ALU做加法运算（相当于单周期处理器中的2个单独的加法器）；AluCtrl为1时，ALU的运算功能由Op和funct决定。

表6.10 多周期MIPS处理器的控制信号及功能

控制信号	信号分类	功能说明
Branch	指令译码信号	分支指令译码信号，beq、bne指令需要产生类似的译码信号
RegDst	多路选择控制	写入目的寄存器的选择，为1时写入rd寄存器，为0时写入rt寄存器
AluSrcA	多路选择控制	控制ALU的第一输入，为0时输入PC值，为1时输入寄存器A的值
AluSrcB	多路选择控制	控制ALU的第二输入，值为03时分别输入B值、常量4、立即数、偏移地址
AluCtrl	多路选择控制	ALU控制器选择控制信号，为0时，ALU做加法运算，1为1时，ALU的运算功能由OP和funct决定
MenToReg	多路选择控制	为0时写入暂存在C寄存器中的ALU运算结果，为1时写入DR寄存器的值
PCSrc	多路选择控制	控制程序计数器PC的输入，为0时输入PC+4，为1时输入分支目标地址
IorD	多路选择控制	控制存储器地址输入，为0时输入指令地址，为1时输入寄存器C中的数据地址
RegWrite	功能部件控制	控制寄存器堆写操作，为1时数据需要写入指定的寄存器，受时钟驱动
MemWrite	功能部件控制	控制数据存储器的写操作，为1时进行写操作，为0时为读操作，受时钟驱动
PCWrite	功能部件控制	程序计数器PC的使能信号，为1时数据写入程序计数器PC，受时钟驱动
IRWrite	功能部件控制	指令寄存器IR的使能信号，为1时数据写入指令寄存器IR，受时钟驱动

- (2) 多周期处理器中指令执行过程分解

- ①时钟周期的确定

- 多周期处理器是将指令的执行分解为若干步骤，每个时钟周期执行一个步骤，执行一条指令需要若干个时钟周期。
 - 通常在指令执行过程中**存储器访问、寄存器堆访问、ALU操作**等，都是相对比较费时的操作，因此时钟周期至少要大于这3个操作中耗时最长的操作所需的时间。

- ②并行操作及控制

- 基于专用通路结构的多周期处理器为指令的执行提供了良好的**并行性**。
 - 例如，**指令译码**（在控制器中完成）和**从寄存器堆中取数据暂存到寄存器A、B**的操作，可以同时进行。
 - 例如，可以**利用ALU提前计算分支目标地址并暂存到寄存器C**中，即使指令最后未实现分支跳转（条件不成立），这样做对指令的执行也没有影响；如果指令实现分支跳转（条件成立），提前计算分支目标地址将有利于缩短分支指令的执行时间。

• (3) 取指令阶段数据通路

- 多周期MIPS处理器指令周期的第一个时钟周期**T1**是**取指令阶段**，共包括**两条并发**的数据通路（图6.28）；这两条数据通路没有冲突，可以在一个时钟周期内并发（同时完成）：
 - ① $M[PC] \rightarrow IR$ ；利用程序计数器PC访存指令存储器，将指令字送入IR中锁存（图中的**紫色通路**）。
 - ② $PC+4 \rightarrow PC$ ；PC自增，修改为下一条顺序指令的地址（图中的**橙色通路**）。
- 第一个数据通路（图中的**紫色通路**）的控制信号： $lorD=0$ （PC $\rightarrow$ 存储器）， $MemWrite=0$ （存储器读）， $IRWrite=1$ （存储器 $\rightarrow$ IR）。
- 第二个数据通路（图中的**橙色通路**）的控制信号： $AluSrcA=0$ （PC $\rightarrow$ ALU的A端）， $AluSrcB=1$ （常数4 $\rightarrow$ ALU的B端）， $AluCtrl=0$ （ALU做加法运算，即PC+4）， $PCSrc=0$ （ALU运算结果（即PC+4） $\rightarrow$ PC输入口）， $PCWrite=1$ （ $PCEn=1$ ， $PC+4 \rightarrow PC$ ）。

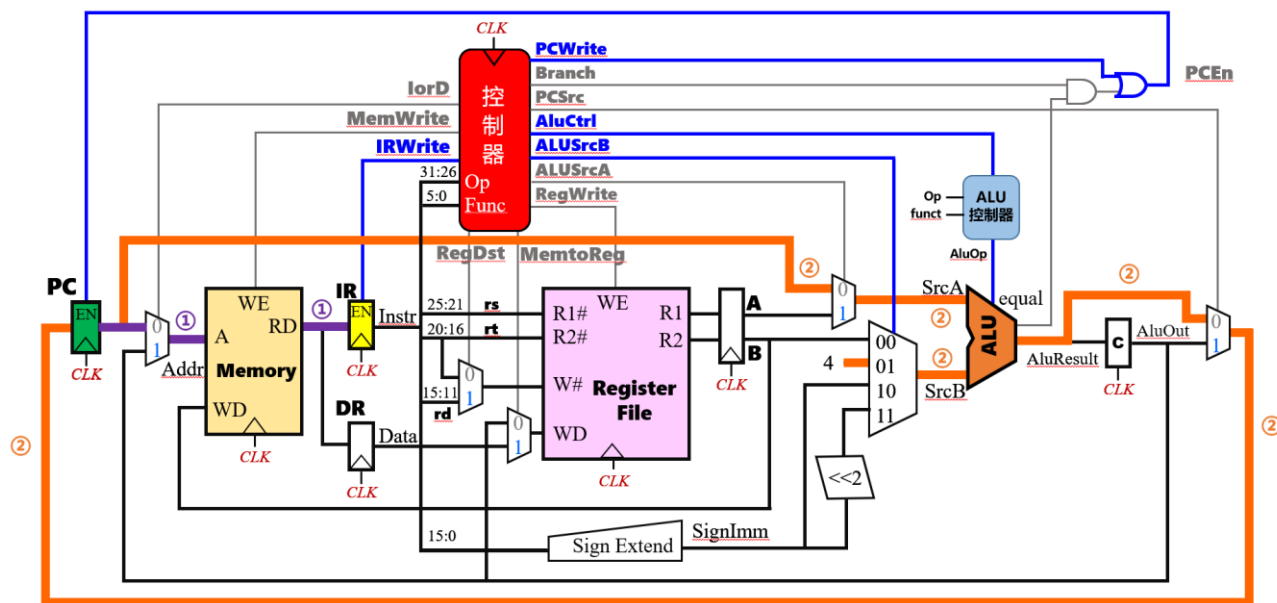


图6.28 多周期MIPS处理器取指令阶段的数据通路

• (4) 译码/取数阶段数据通路

- 多周期MIPS处理器指令周期的第二个时钟周期T2是译码/取数阶段，共包括三条并发的数据通路（图6.29）；这三条数据通路没有冲突，可以在一个时钟周期内并发（同时完成）：
 - ① IR的Op和funct -> 控制器；指令译码，控制器根据IR中的Op和funct字段进行译码，生成指令译码信号（图中的绿色通路）。
 - ② R[rs] -> A; R[rt] -> B；取操作数，将IR中的rs、rt字段送寄存器堆的R1#、R2#，寄存器堆的输出送A、B寄存器中暂存（图中的橙色通路）。
 - ③ $PC + 4 + \text{SignExt}(\text{imm16}) \ll 2 \rightarrow C$ ；提前计算分支目标地址，并将结果暂存在C寄存器中（图中的紫色通路）。
- 第一个数据通路（图中的绿色通路）没有控制信号；第二个数据通路（图中的橙色通路）也没有控制信号（RegWrite是控制寄存器堆的写操作）。
- 第三个数据通路（图中的紫色通路）的控制信号：AluSrcA=0（PC+4 -> ALU的A端），AluSrcB=3（ $\text{SignExt}(\text{imm16}) \ll 2 \rightarrow$ ALU的B端），AluCtrl=0（ALU做加法运算，即 $PC + 4 + \text{SignExt}(\text{imm16}) \ll 2 \rightarrow C$ ）。

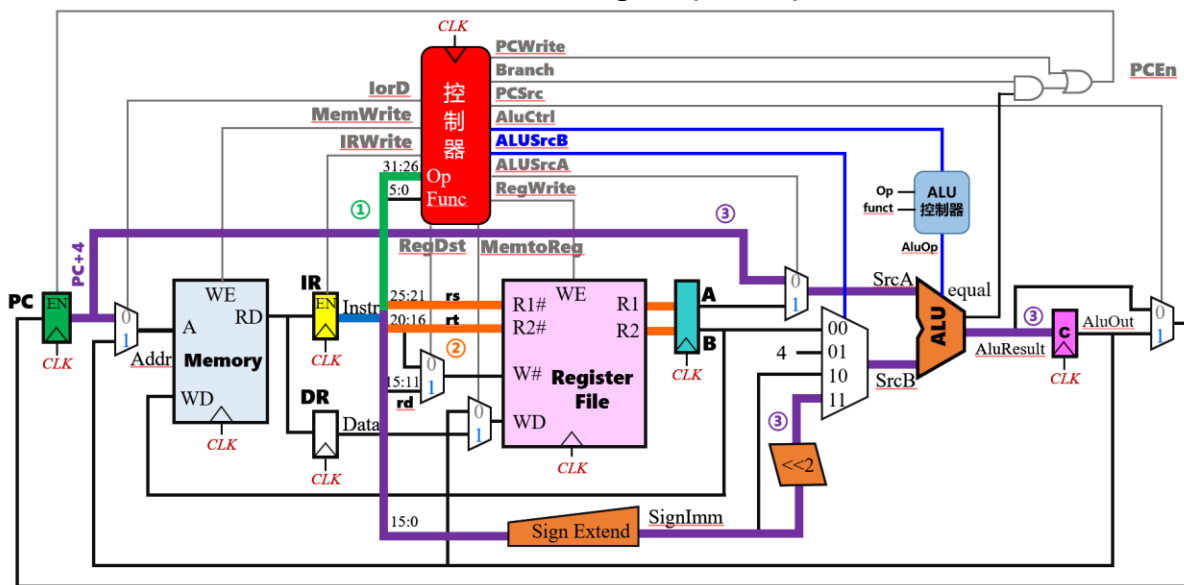


图6.29 多周期MIPS处理器译码/取数阶段的数据通路

• (5) R型算术逻辑运算指令执行阶段的数据通路

- MIPS的算术逻辑运算指令属于R型指令，例如加法指令：**add rd,rs,rt**；RTL功能描述是： **$R[rs] + R[rt] \rightarrow R[rd]$** 。
- R型算术逻辑运算指令执行阶段需要**2个时钟周期（T3、T4）**，每个时钟周期各对应一条数据通路（图6.30）；这两条数据通路不是并发执行，而是先后执行的：
 - ① **$A \text{ op } B \rightarrow C$** ；数据运算，结果送C寄存器，具体执行什么运算由IR中的Op和funct字段译码决定（图中的**绿色通路**，时钟周期T3内执行）。
 - ② **$C \rightarrow R[rd]$** ；结果写回，将运算结果从寄存器C，写回到寄存器堆的rd寄存器中（图中的**红色通路**，时钟周期T4内执行）。
- 第一个数据通路（图中的**绿色通路**）的控制信号：**AluSrcA=1**（寄存器A $\rightarrow$ ALU的A端），**AluSrcB=0**（寄存器B $\rightarrow$ ALU的B端），**AluCtrl=1**（ALU控制器根据IR的Op和funct产生AluOp信号，决定ALU做什么运算，运算结果 **$A \text{ op } B \rightarrow C$** ）。
- 第二个数据通路（图中的**红色通路**）的控制信号：**RegDst=1**（rd $\rightarrow$ 寄存器堆的W#端），**MemToReg=0**（寄存器C $\rightarrow$ 寄存器堆的WD端），**RegWrite=1**（寄存器堆写，即： **$C \rightarrow R[rd]$** ）。

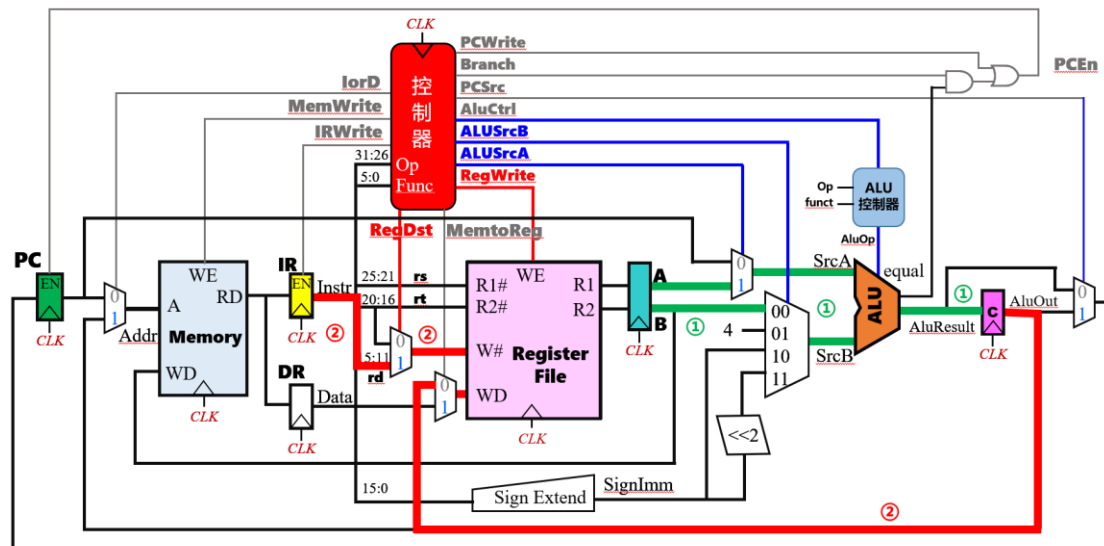


图6.30 R型算术逻辑运算指令执行阶段的数据通路

• (6) I型算术逻辑运算指令执行阶段的数据通路

- MIPS的I型指令除lw、sw指令外，还包括运算指令，例如立即数加法指令：**addi rt,rs,imm**；RTL功能描述是： **$R[rs] + \text{SignExt}(imm) \rightarrow R[rt]$** 。
- I型算术逻辑运算指令执行阶段也需要**2个时钟周期（T3、T4）**，每个时钟周期各对应一条数据通路（图6.31）；这两条数据通路也不是并发执行，而是先后执行的：
 - ① **$A \text{ op } \text{SignExt}(imm) \rightarrow C$** ；数据运算，A和立即数进行运算，结果送C寄存器，具体执行什么运算由IR中的Op和funct字段译码决定（图中的**绿色通路**，时钟周期T3内执行）。
 - ② **$C \rightarrow R[rt]$** ；结果写回，将运算结果从寄存器C，写回到寄存器堆的rd寄存器中（图中的**红色通路**，时钟周期T4内执行）。
- 第一个数据通路（图中的**绿色通路**）的控制信号：**AluSrcA=1**（寄存器A -> ALU的A端），**AluSrcB=2**（ **$\text{SignExt}(imm) \rightarrow \text{ALU的B端}$** ），**AluCtrl=1**（ALU控制器根据IR的Op和funct产生AluOp信号，决定ALU做什么运算，运算结果 **$A \text{ op } \text{SignExt}(imm) \rightarrow C$** ）。
- 第二个数据通路（图中的**红色通路**）的控制信号：**RegDst=0**（rt -> 寄存器堆的W#端），**MemToReg=0**（寄存器C -> 寄存器堆的WD端），**RegWrite=1**（寄存器堆写，即： **$C \rightarrow R[rt]$** ）。

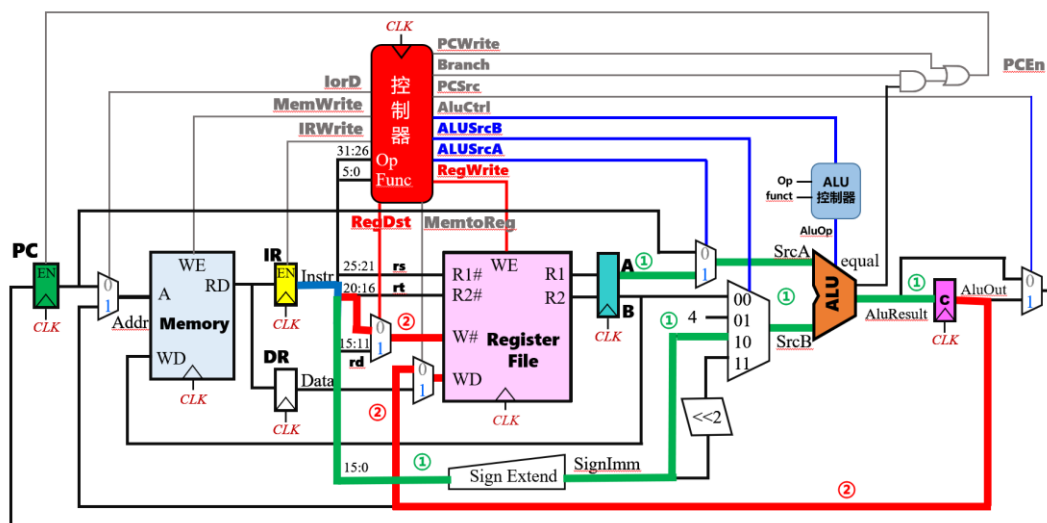


图6.31 I型算术逻辑运算指令执行阶段的数据通路

• (7) lw指令执行阶段的数据通路

- lw指令属于I型指令：**lw rt,imm(rs)**；RTL功能描述是：**Mem[R[rs] + SignExt(imm)] -> R[rt]**。
- lw指令执行阶段需要**3个时钟周期 (T3、T4、T5)**，每个时钟周期各对应一条数据通路（图6.32）；这三条数据通路也不是并发执行，而是先后执行的：
 - ① **A + SignExt(imm) -> C**；计算访存地址，A和立即数相加，结果送C寄存器（图中的**绿色通路**，时钟周期T3内执行）。
 - ② **M[C] -> DR**；访存，以C寄存器的内容作为访存地址，取出的数送DR（图中的**红色通路**，时钟周期T4内执行）。
 - ③ **DR -> R[rt]**；结果写回，将DR中的数据写入寄存器堆的rd寄存器中（图中的**紫色通路**，时钟周期T5内执行）。
- 第一个数据通路（图中的**绿色通路**）的控制信号：**AluSrcA=1**（寄存器A -> ALU的A端），**AluSrcB=2**（**SignExt(imm) -> ALU的B端**），**AluCtrl=0**（ALU进行加法运算，运算结果**A + SignExt(imm) -> C**）。
- 第二个数据通路（图中的**红色通路**）的控制信号：**lorD=1**（C寄存器 -> 存储器的地址端A），**MemWrite=0**（存储器读，**M[C] -> DR**）。
- 第三个数据通路（图中的**紫色通路**）的控制信号：**RegDst=0**（rt -> 寄存器堆的W#端），**MemToReg=1**（DR -> 寄存器堆的WD端），**RegWrite=1**（寄存器堆写，即：**DR -> R[rt]**）。

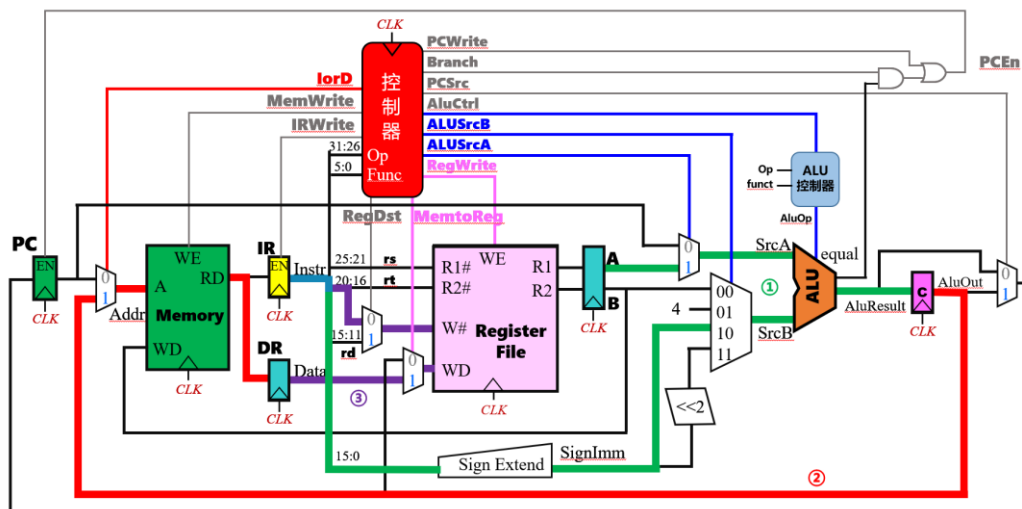


图6.32 lw指令执行阶段的数据通路

• (8) sw指令执行阶段的数据通路

- sw指令也属于I型指令：**sw rt,imm(rs)**；RTL功能描述是： **$R[rt] \rightarrow \text{Mem}[R[rs] + \text{SignExt}(imm)]$** 。
- sw指令执行阶段需要**2个时钟周期（T3、T4）**，每个时钟周期各对应一条数据通路（图6.33）；这两条数据通路也不是并发执行，而是先后执行的：
 - $A + \text{SignExt}(imm) \rightarrow C$ ；计算访存地址，A和立即数相加，结果送C寄存器（图中的**绿色通路**，时钟周期T3内执行）。
 - $B \rightarrow M[C]$ ；存储器写，将B寄存器的内容（即 **$R[rt]$** ）写入以C寄存器内容为地址的存储器中（图中的**红色通路**，时钟周期T4内执行）。
- 第一个数据通路（图中的**绿色通路**）的控制信号：**AluSrcA=1**（寄存器A $\rightarrow$ ALU的A端），**AluSrcB=2**（ **$\text{SignExt}(imm) \rightarrow$ ALU的B端**），**AluCtrl=0**（ALU进行加法运算，运算结果 **$A + \text{SignExt}(imm) \rightarrow C$** ）。
- 第二个数据通路（图中的**红色通路**）的控制信号：**lorD=1**（C寄存器 $\rightarrow$ 存储器的地址端A），**MemWrite=1**（存储器写， **$B \rightarrow M[C]$** ）。

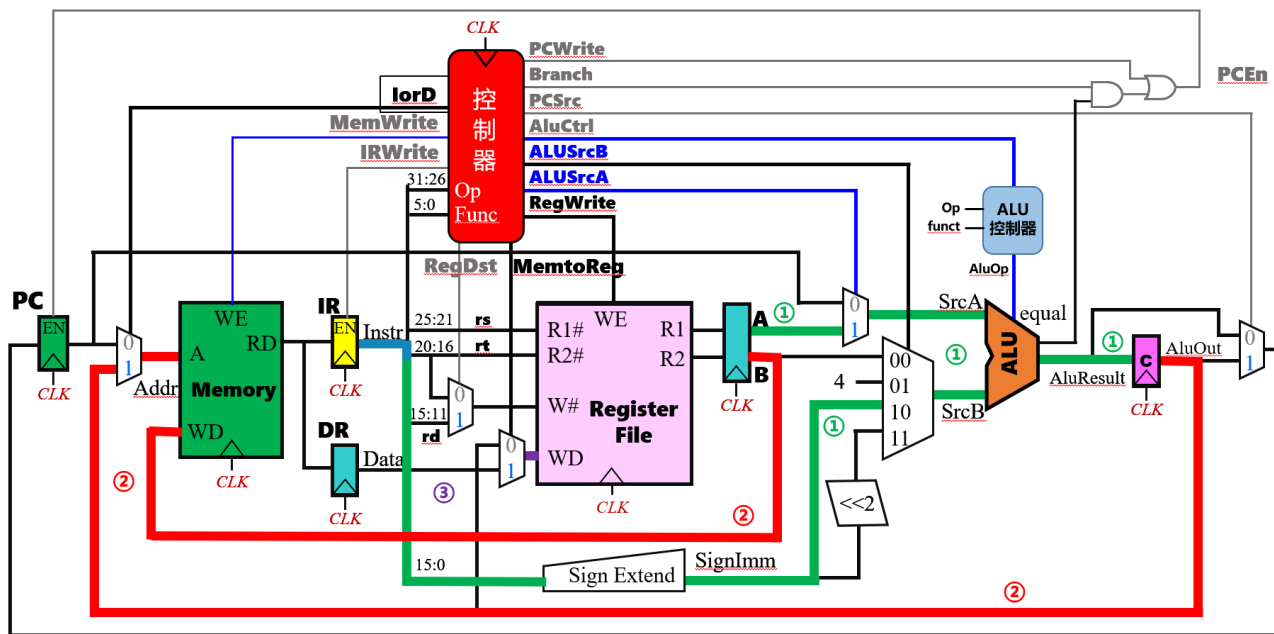


图6.33 sw指令执行阶段的数据通路

图6.34 beq指令执行阶段的数据通路

- 图6.35：多周期MIPS处理器数据通路的指令图。
- 取指周期包括取指令阶段和译码/取数阶段；lw指令需要5个时钟周期，sw、add、addi指令需要4个时钟周期，beq指令需要3个时钟周期。

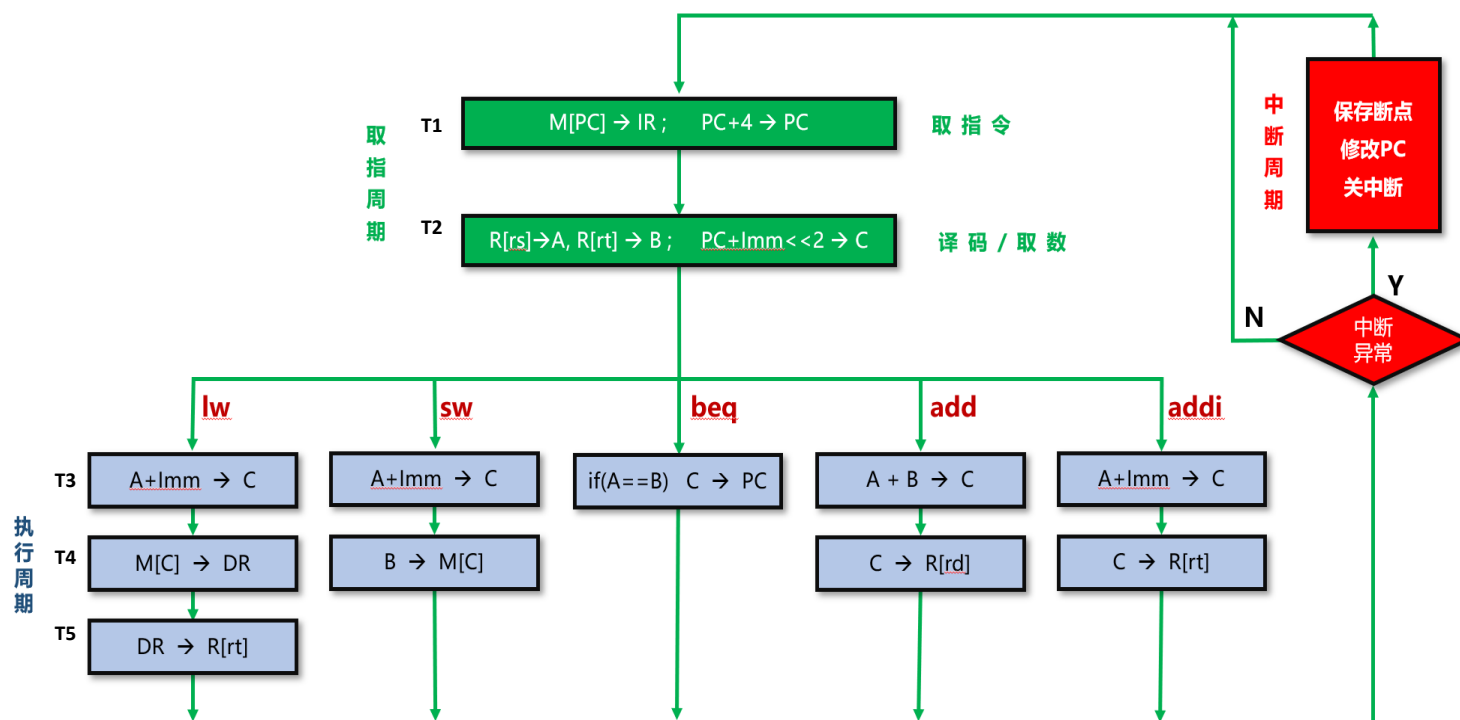


图6.35 多周期MIPS处理器数据通路的指令图

表6.11 MIPS指令多周期数据通路及控制信号

指令类型	时钟周期	操作	控制信号（非零值）
取指令	T1	M[PC] -> IR ; PC+4 -> PC	IRWrite=1; AluSrcB=1; PCWrite=1
	T2	R[rs] -> A, R[rt] -> B ; PC+Imm<<2 -> C	AluSrcB=3
R型运算	T3	A + B -> C	AluSrcA=1; AluCtrl=1
	T4	C -> R[rd]	RegDst=1; RegWrite=1
I型运算	T3	A+Imm -> C	AluSrcA=1; AluSrcB=2; AluCtrl=1
	T4	C -> R[rt]	RegWrite=1
lw指令	T3	A+Imm -> C	AluSrcA=1; AluSrcB=2
	T4	M[C] -> DR	lorD=1
	T5	DR -> R[rt]	MemToReg=1; RegWrite=1
sw指令	T3	A+Imm -> C	AluSrcA=1; AluSrcB=2
	T4	B -> M[C]	lorD=1; MemWrite=1
beq指令	T3	if(A==B) C -> PC	AluSrcA=1; PCSrc=1; Branch=1

- 例6.3：对图6.27给出的多周期MIPS处理器数据通路进行适当改造，使得它能支持无条件分支指令j address。

解：

- j指令属于J型指令，其指令格式为：j address；RTL功能描述是： $\{(PC+4)_{31:28}, address \ll 2\} \rightarrow PC$ 。
- 参考图6.24单周期MIPS处理器j指令的数据通路，需要对图6.27的多周期MIPS处理器的数据通路进行如下的改造（改造后的数据通路见下一页）：

- ① 增加一个左移部件将address（26位）左移2位得到28位地址；增加一个分线器将PC+4的高4位分离出来；增加一个拼接器将PC+4的高4位和address左移2位后的28位地址，拼接为32位地址。
 - ② 将图6.27最右边的2路选择器改为4路选择器，其控制信号PCSrc改为2位；PCSrc=00，选择ALU的输出（PC+4）；PCSrc=01，选择C寄存器的输出（PC+4+SignExt(imm)<<2）；PCSrc=10，选择拼接器的输出（ $\{(PC+4)_{31:28}, Address \ll 2\}$ ）。
 - ③ 增加j指令的译码输出信号Jump，将图6.27中的左上角的2输入或门改为3输入或门；当PCWrite=1时，PCEn=1；当Jump=1时，PCEn=1；当Branch=1且equal=1时，PCEn=1。
- j指令的执行周期也只需要一个时钟周期。

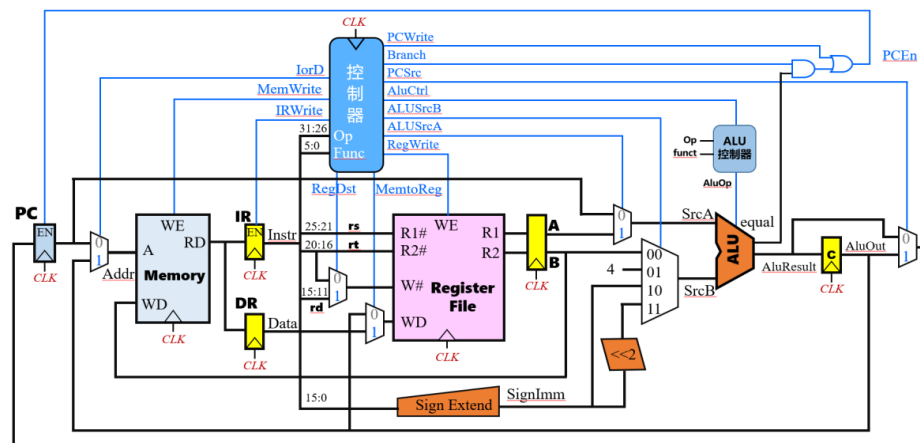
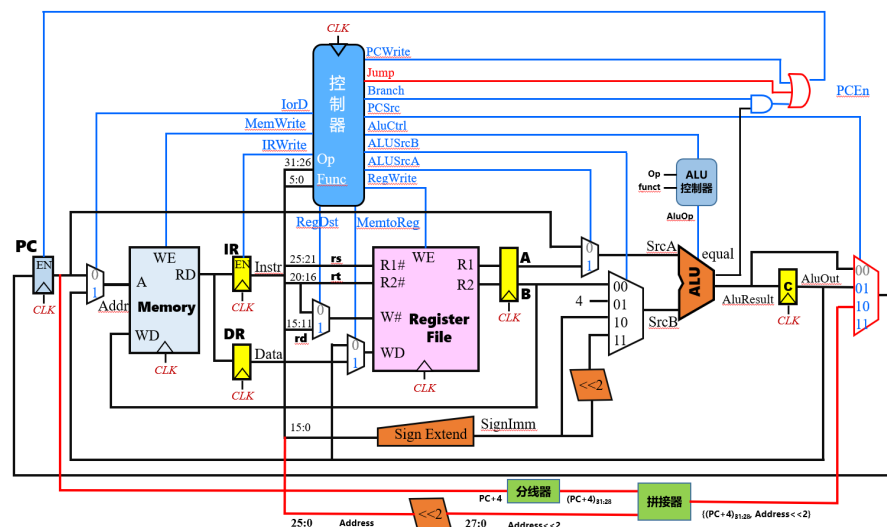
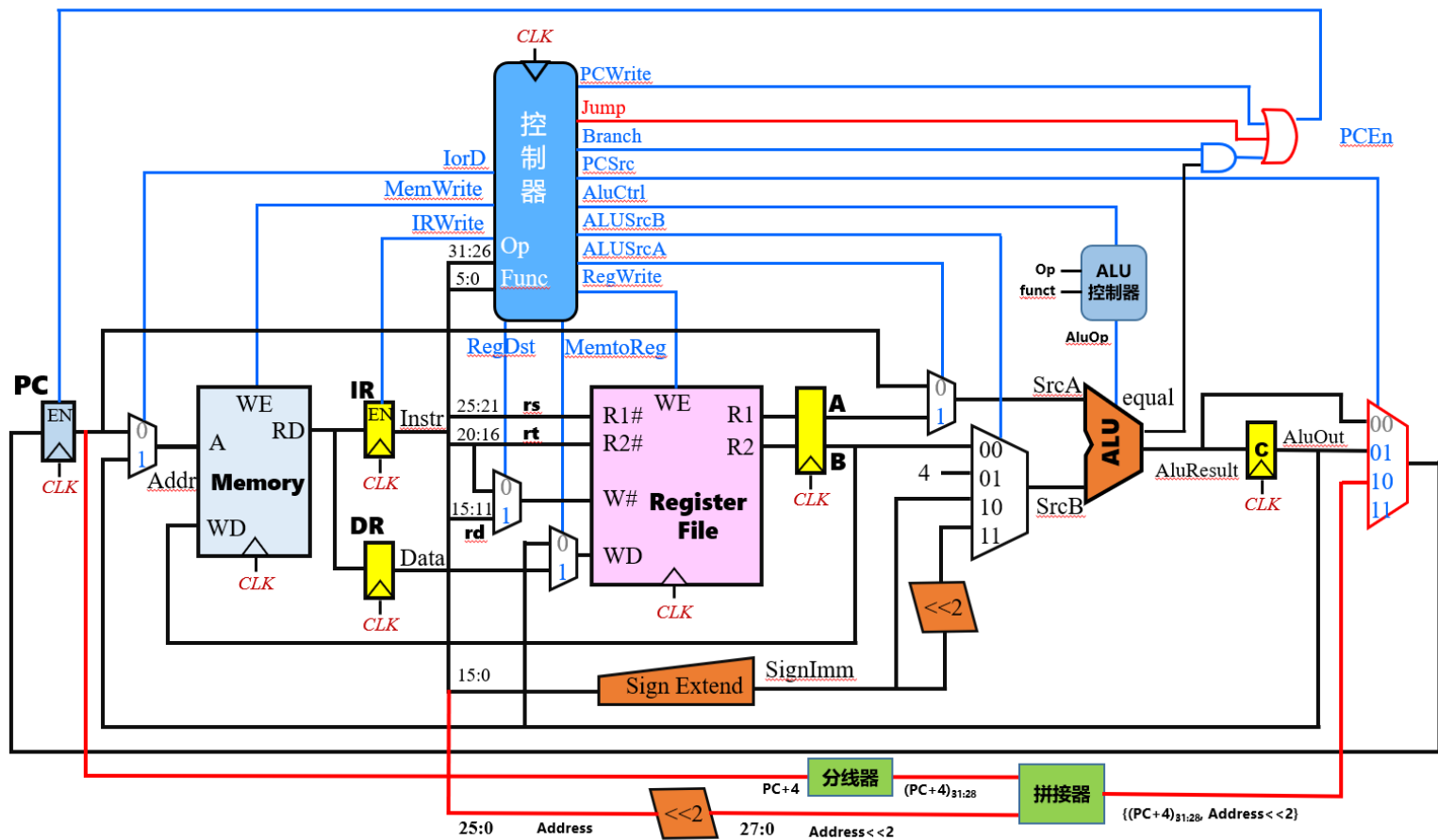


图6.27 多周期MIPS处理器的数据通路



图（例6.3） 支持j指令的多周期MIPS处理器的数据通路



图（例6.3） 支持j指令的多周期MIPS处理器的数据通路

• (10) 多周期处理器性能分析

- 设计多周期MIPS处理器的**时钟周期**时要遵循一个基本原则：指令执行的每个时钟周期只能包含**存储器访问**、**寄存器堆访问**及**ALU操作**等3个慢速操作中的一个。
- 前面的取指阶段、译码/取数阶段、执行阶段（R型算术逻辑运算指令、I型算术逻辑运算指令、lw指令、sw指令、条件分支指令beq）的数据通路中，有两条关键路径（**时间最长的路径**）：

- ① 取指阶段的“PC+4 -> PC”数据通路（图6.28）
- ② lw指令执行阶段的“M[C] -> DR”数据通路（图6.32）

- 所以，多周期MIPS处理器的最小时钟周期为： $T_{\min_clk} = T_{clk_to_q} + T_{mux} + \max(T_{alu} + T_{mux}, T_{mem}) + T_{setup}$

- 其中：

- ① $T_{clk_to_q}$ 为程序计数器的触发器延迟
- ② T_{mux} 为多路选择器的延迟
- ③ T_{alu} 为运算器的延迟
- ④ T_{mem} 为存储器的访存延迟
- ⑤ T_{setup} 为写入寄存器堆的寄存器建立时间

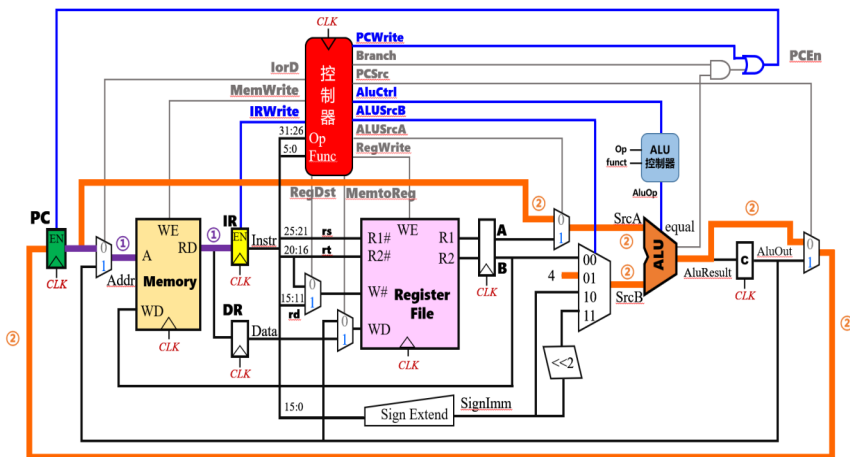


图6.28 多周期MIPS处理器取指令阶段的数据通路

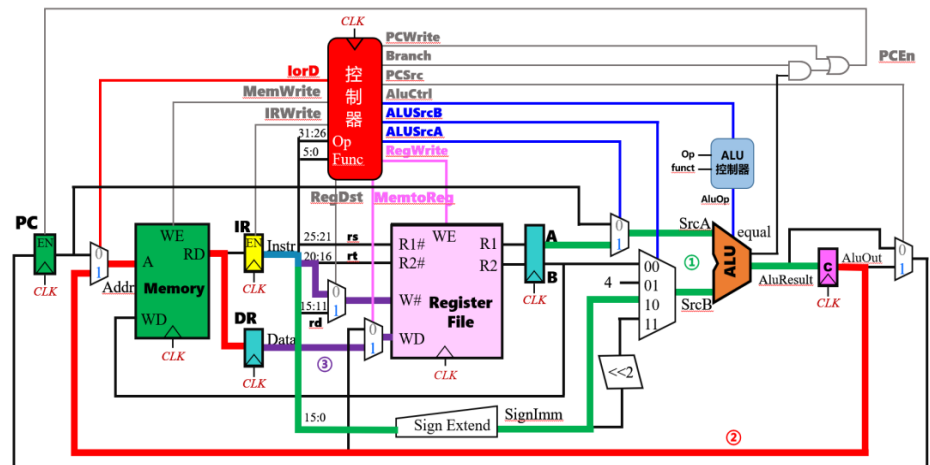


图6.32 lw指令执行阶段的数据通路

- 例6.4：求例6.1中基准测试程序SPECINT2000在**多周期MIPS处理器**上运行的CPI（SPECINT2000基准测试程序包含的取数据指令、存数据指令、条件分支指令、跳转指令、R型算术逻辑运算指令比例分别为：25%、10%、11%、2%、52%）。假设程序的指令数目为1000亿条，CPU采用65nm CMOS工艺实现，各功能部件的时间延迟如表6.8所示，求该计算机最大时钟频率，以及基准测试程序的执行时间。

表6.8 各功能部件的时间延迟

功能部件	参数	延迟	功能部件	参数	延迟
寄存器延迟	$T_{\text{clk_to_q}}$	30ps	运算器ALU	T_{alu}	200ps
存储器读	T_{mem}	250ps	多路选择器	T_{mux}	25ps
寄存器堆读	$T_{\text{RF_read}}$	150ps	寄存器建立时间	T_{setup}	20ps

- 解：
 - 在多周期MIPS处理器上，执行取数据指令、存数据指令、条件分支指令、跳转指令、R型算术逻辑运算指令的时钟周期数分别为：5、4、3、3（根据例6.3，j指令执行阶段只需要1个时钟周期）、4。
 - 因此，**CPI** = $0.25 \times 5 + 0.1 \times 4 + 0.11 \times 3 + 0.02 \times 3 + 0.52 \times 4 = 4.12$
 - 多周期MIPS处理器的**最小时钟周期**： $T_{\text{min_clk}} = T_{\text{clk_to_q}} + T_{\text{mux}} + \max(T_{\text{alu}} + T_{\text{mux}}, T_{\text{mem}}) + T_{\text{setup}} = 30 + 25 + \max(200 + 25, 250) + 20 = 325\text{ps}$
 - 因此，最大时钟频率为： $T_{\text{max_freq}} = 1/325\text{ps} = 3.08\text{GHz}$
 - 基准测试程序的执行时间为： $T_{\text{total}} = \text{指令条数} \times \text{CPI} \times T_{\text{min_clk}} = 1000 \times 10^8 \times 4.12 \times 325 \times 10^{-12} = 133.9\text{s}$

为何多周期执行时间133.9s > 单周期92.5s?

三种数据通路的性能比较：性能比较从优到劣：单周期>多周期>单总线；

1. 单周期MIPS处理器：

单周期处理器的CPI = **1**

多周期中的访存指令LW拖慢了时钟周期，LW被分成5个步骤，但是最慢的步骤进行了时钟同步，时钟周期并不是单周期的1/5，即 $925/5=185\text{ps}$ ，Lw 指令周期为 $5*325\text{ps}=1625\text{ps}>925\text{ps}$ （单周期的CPI=1，故而单条指令执行时间为 $925*1=925$ ）

最小时钟周期： $T_{\min_clk} = T_{clk_to_q} + 2T_{mem} + T_{RF_read} + T_{alu} + T_{mux} + T_{setup} = 30 + 2 \times 250 + 150 + 200 + 25 + 20 = \mathbf{925\text{ps}}$

最大时钟频率为： $T_{\max_freq} = 1/925\text{ps} = 1.08\text{GHz}$

基准测试程序的执行时间为： $T_{\text{total}} = \text{指令条数} \times \text{CPI} \times T_{\min_clk} = 1000 \times 10^8 \times \mathbf{1 \times 925} \times 10^{-12} = 92.5\text{s}$

2. 多周期MIPS处理器：

CPI = $0.25 \times 5 + 0.1 \times 4 + 0.11 \times 3 + 0.02 \times 3 + 0.52 \times 4 = \mathbf{4.12}$
LW SW beg add addi

最小时钟周期： $T_{\min_clk} = T_{clk_to_q} + T_{mux} + \max(T_{alu} + T_{mux}, T_{mem}) + T_{setup} = 30 + 25 + \max(200 + 25, 250) + 20 = \mathbf{325\text{ps}}$

因此，最大时钟频率为： $T_{\max_freq} = 1/325\text{ps} = 3.08\text{GHz}$

基准测试程序的执行时间为： $T_{\text{total}} = \text{指令条数} \times \text{CPI} \times T_{\min_clk} = 1000 \times 10^8 \times \mathbf{4.12 \times 325} \times 10^{-12} = 133.9\text{s}$

3. 单总线处理器：

CPI = $0.25 \times 9 + 0.1 \times 9 + 0.11 \times 9 + 0.02 \times 7 + 0.52 \times 7 = \mathbf{7.92}$

最小时钟周期： $T_{\min_clk} = T_{clk_to_q} + \max(T_{alu}, T_{mem}) + T_{setup} = 30 + \max(200, 250) + 20 = 300\text{ps}$

最大时钟频率为： $T_{\max_freq} = 1/300\text{ps} = 3.33\text{GHz}$

基准测试程序的执行时间为： $T_{\text{total}} = \text{指令条数} \times \text{CPI} \times T_{\min_clk} = 1000 \times 10^8 \times \mathbf{7.92 \times 300} \times 10^{-12} = 237.6\text{s}$

单总线结构计算机

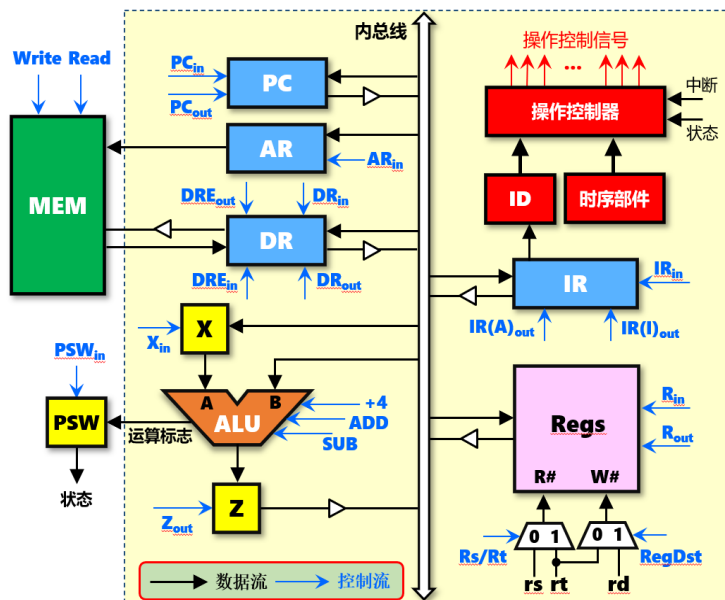


图6.8 单总线结构的计算机框图

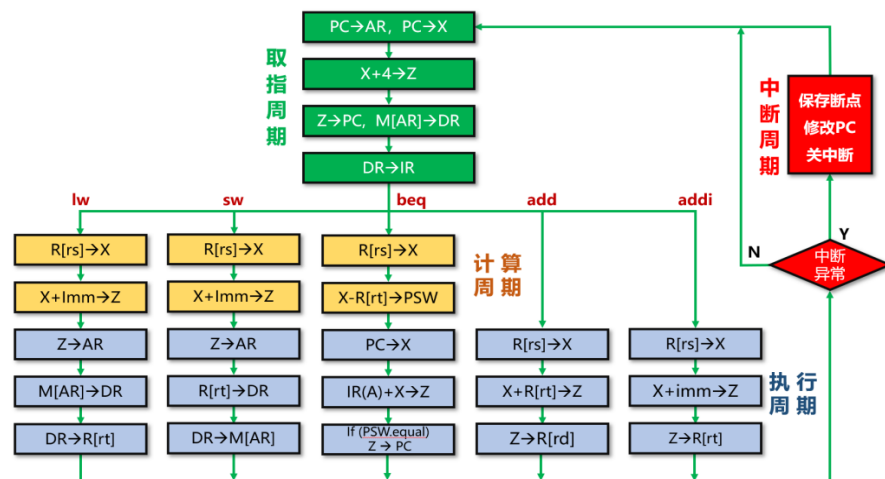


图6.14 单总线结构数据通路指令方框图

指令执行时间

取数指令（lw）：9个时钟周期

存数指令（sw）：9个时钟周期

条件分支指令（beq）：9个时钟周期

R型算术逻辑运算指令（add）：7个时钟周期

I型算术逻辑运算指令（addi）：7个时钟周期

跳转指令（j）：7个时钟周期

$$\text{最小时钟周期} = T_{\min_clk} = T_{clk_to_q} + \max(T_{alu}, T_{mem}) + T_{setup} = 30 + \max(200, 250) + 20 = 300ps$$

单周期MIPS处理器

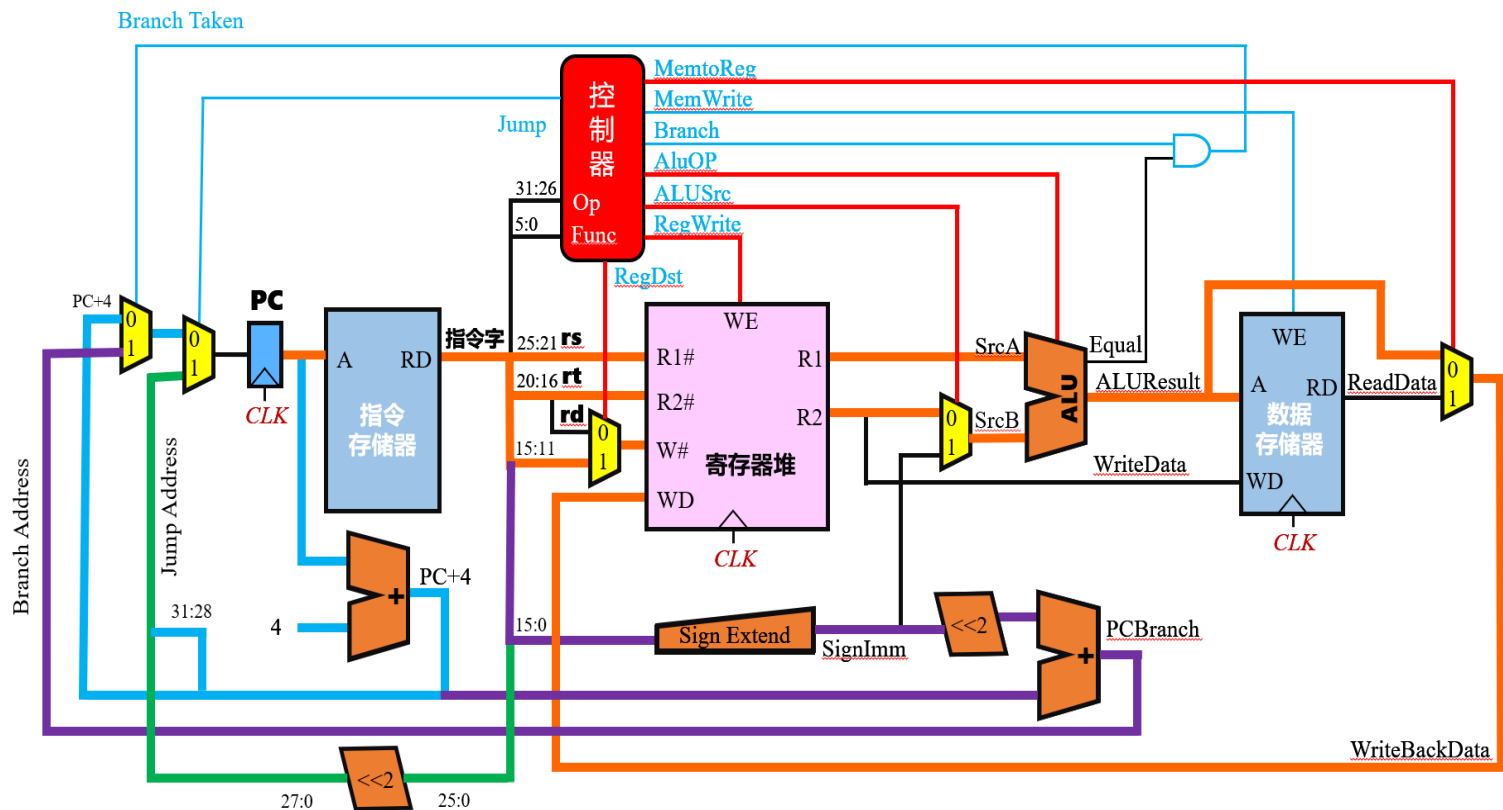


图6.25 单周期MIPS处理器的数据通路

指令执行时间

所有指令：1个时钟周期

$$\text{最小时钟周期} = T_{\min_clk} = T_{clk_to_q} + 2T_{mem} + T_{RF_read} + T_{alu} + T_{mux} + T_{setup} = 30 + 2 \times 250 + 150 + 200 + 25 + 20 = 925ps$$

多周期MIPS处理器

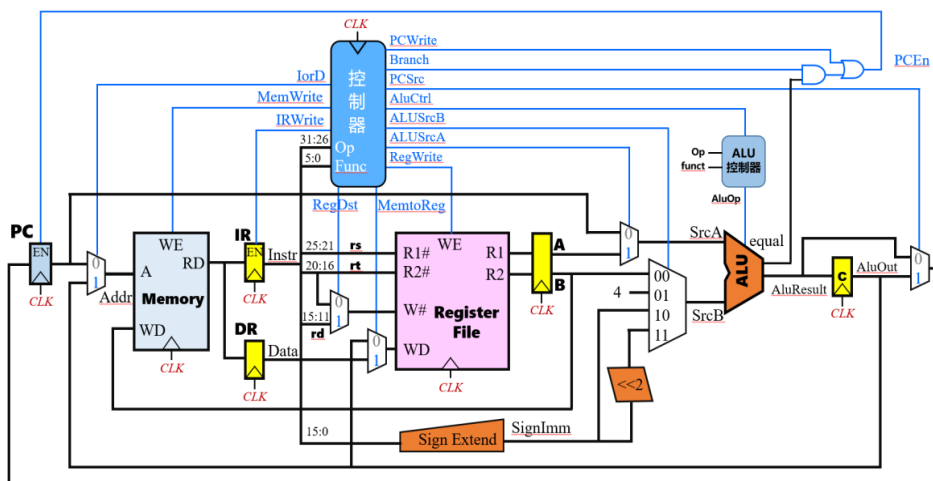


图6.27 多周期MIPS处理器的数据通路

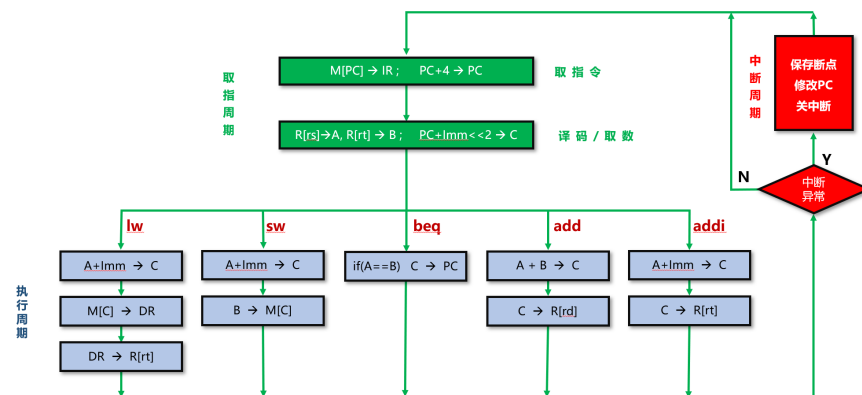


图6.35 多周期MIPS处理器数据通路的指令图

指令执行时间

取数指令（lw）：5个时钟周期

存数指令（sw）：4个时钟周期

条件分支指令（beq）：3个时钟周期

R型算术逻辑运算指令（add）：4个时钟周期

I型算术逻辑运算指令（addi）：4个时钟周期

跳转指令（j）：3个时钟周期

$$\text{最小时钟周期} = T_{\min_clk} = T_{clk_to_q} + T_{mux} + \max(T_{alu} + T_{mux}, T_{mem}) + T_{setup} = 30 + 25 + \max(200 + 25, 250) + 20 = 325ps$$

6.4 时序与控制

• 6.4.1 中央处理器的时序

- **单周期处理器**：所有指令都在1个时钟周期内完成；控制信号同时产生，**没有先后顺序**。
- **单总线结构的处理器**（图6.14）：1个指令周期包含若干个机器周期（取指周期、计算周期和执行周期），1个机器周期又包含若干个时钟周期（取指周期、计算周期、执行周期分别包含4个、2个、3个时钟周期）；控制信号应有**严格的时间先后顺序**，才能保证指令的正确执行。
- **多周期处理器**（图6.35）：与单总线结构的处理器类似，控制信号应有**严格的时间先后顺序**，才能保证指令的正确执行。

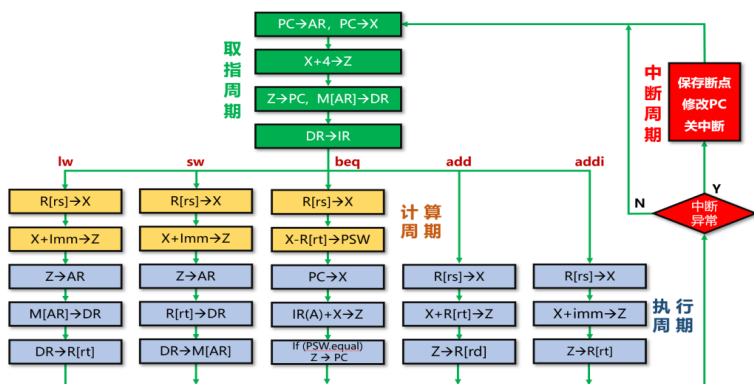


图6.14 单总线结构数据通路指令方框图

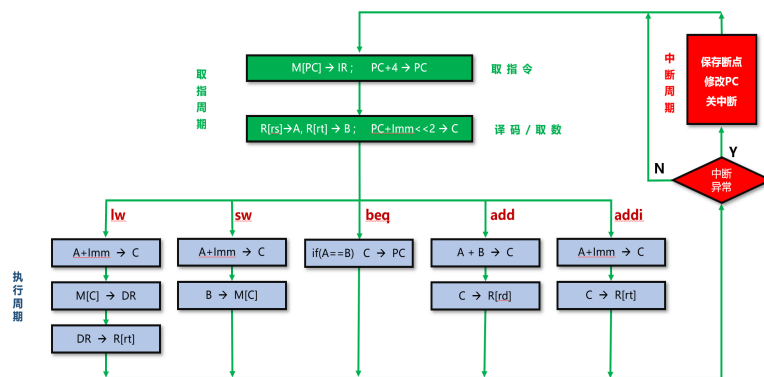


图6.35 多周期MIPS处理器数据通路的指令图

6.4.1 中央处理器的时序

- 传统的计算机采用**三级时序系统**（图6.3）：**指令周期**、**机器周期**、**时钟周期（节拍脉冲）**。

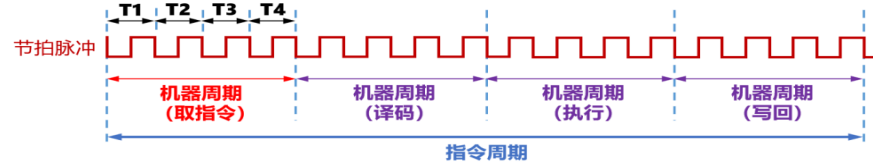


图6.3 指令周期、机器周期与时钟周期的关系

- 现代计算机不再设置节拍脉冲，一个时钟周期就是一个节拍，称为**现代时序系统**

- 图6.36：**三级时序系统示意图**。

- 状态周期**：表示当前指令周期处于哪个机器周期，例如**M1**为取指令周期、**M2**为指令译码周期。
- 节拍电位**：表示当前机器周期处于哪个节拍，例如1个状态周期包含4个节拍（**T1**、**T2**、**T3**、**T4**）。
- 节拍脉冲**：一个节拍电位内设置一个或多个节拍脉冲，例如1个节拍电位内设置2个节拍脉冲（**P1**、**P2**）。

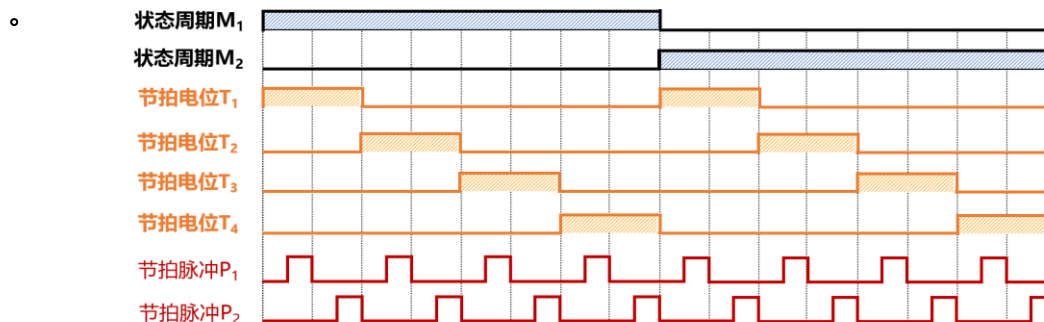


图6.36 三级时序系统示意图

6.4.2 控制方式

对控制器产生控制信号进行时间控制的方式就是控制方式

多级时序的控制方式包括同步控制、异步控制以及联合控制等三种方式

— 1、同步控制方式

- 同步控制方式又称为**固定时序**方式，选取各部件中**最长的操作时间**作为统一的时间间隔标准（时钟周期）。
 - （1）定长指令周期
 - 每条指令包含的机器周期数是固定的（例如1个指令周期=3个机器周期：**取指周期、计算周期、执行周期**）。
 - 每个机器周期包含的时钟周期数也是固定的（例如1个机器周期=4个时钟周期：**T₁、T₂、T₃、T₄**）。
 - 即每条指令的执行都需要12个时钟周期（ $3 \times 4 = 12$ ），图6.37。

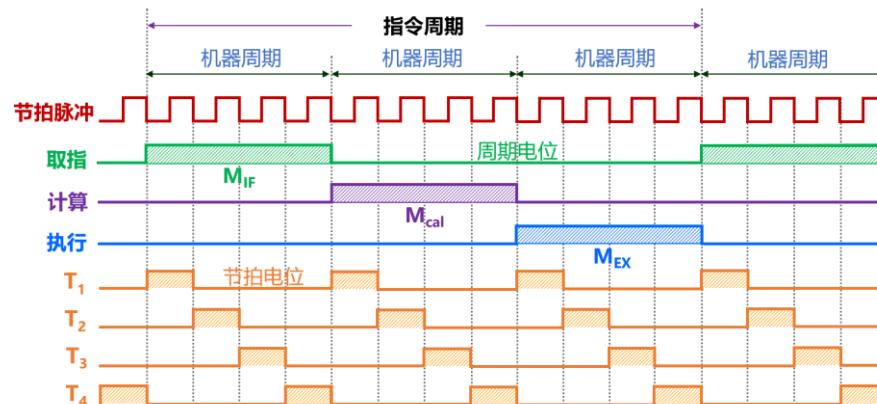


图6.37 定长指令周期的三级时序

6.4.2 控制方式

- (2) 变长指令周期
 - 不同的指令包含的机器周期数是不同的；例如单总线结构的计算机（图6.14）**lw**、**sw**、**beq**指令包含3个机器周期：取指周期、计算周期、执行周期，**add**、**addi**指令包含2个机器周期：取指周期、执行周期。
 - 图6.38中第1个指令周期包含3个机器周期： M_{if} 、 M_{cal} 、 M_{ex} ，第2个指令周期包含2个机器周期： M_{if} 、 M_{ex} 。
 - 不同的机器周期包含的时钟周期数也是不同的，例如**取指指令**包含4个时钟周期，**计算周期**包含2个时钟周期，**执行周期**包含3个时钟周期。
 - 执行**lw**、**sw**、**beq**指令需要9个时钟周期，执行**add**、**addi**指令需要7个时钟周期。
 - 教材中P219的“（2）机器周期固定，节拍数不固定”和“（3）中央与局部控制相结合”都属于变长指令周期。

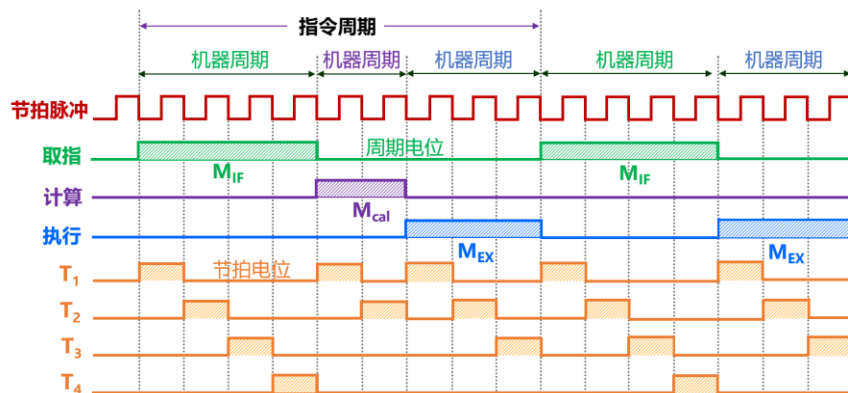


图6.38 变长指令周期的三级时序

6.4.2 控制方式

– 2、异步控制方式

- 异步控制方式没有统一时钟信号，没有固定的机器周期和节拍。
- 各功能部件及操作的时序采用应答机制实现，控制部件发出操作控制信号给功能部件后，必须等到功能部件发出应答信号，才能开始下一步的操作。

– 3、联合控制方式

- 同步控制和异步控制相结合的方式。
- 对大多数操作控制序列采用机器周期、节拍电位进行同步控制；而对少数时间难以确定的操作可以采用异步控制方式。

6.4.3 时序发生器

- 时序发生器用于产生图6.37（定长指令周期）和图6.38（变长指令周期）中的状态周期（取指周期 M_{if} 、计算周期 M_{cal} 、执行周期 M_{ex} ）与节拍电位（ T_1 、 T_2 、 T_3 、 T_4 ）。
- 时序发生器的输入包括：时钟信号CLK、指令译码信号、反馈信号（如程序状态字寄存器PSW/PSR的输出、中断请求信号等）、启动/停止/复位信号等。
- 时序发生器的输出：状态周期、节拍电位。

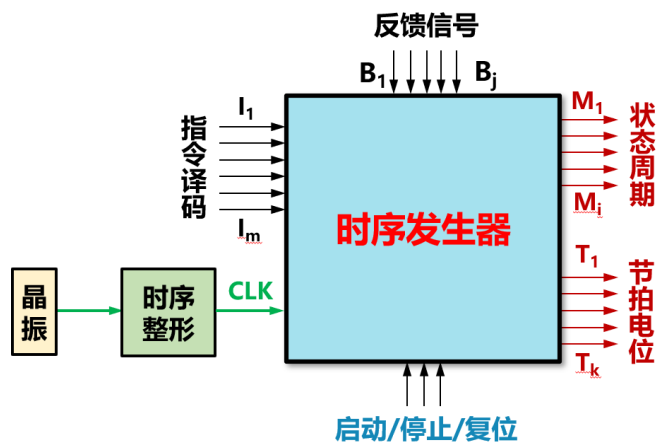


图6.39 时序发生器工作原理

6.4.3 时序发生器

— (1) 定长指令周期三级时序状态机

- 定长指令周期三级时序的状态周期 (M_{if} 、 M_{cal} 、 M_{ex}) 和节拍电位 (T_1 、 T_2 、 T_3 、 T_4) 只与时钟信号CLK有关，与指令译码、反馈信号无关。
- 其时序发生器可以采用有限状态机实现 (图6.40)，每条指令都有12个状态，其中 $s_0 \sim s_3$ 为取指周期， $s_4 \sim s_7$ 为计算周期， $s_8 \sim s_{11}$ 为执行周期。

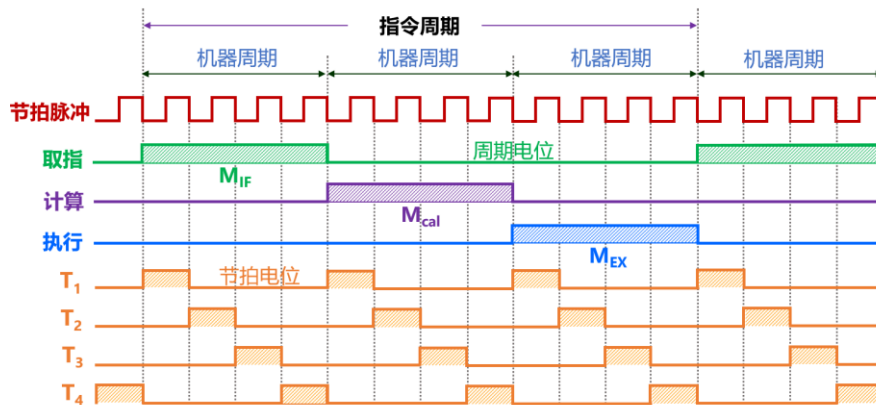


图6.37 定长指令周期的三级时序

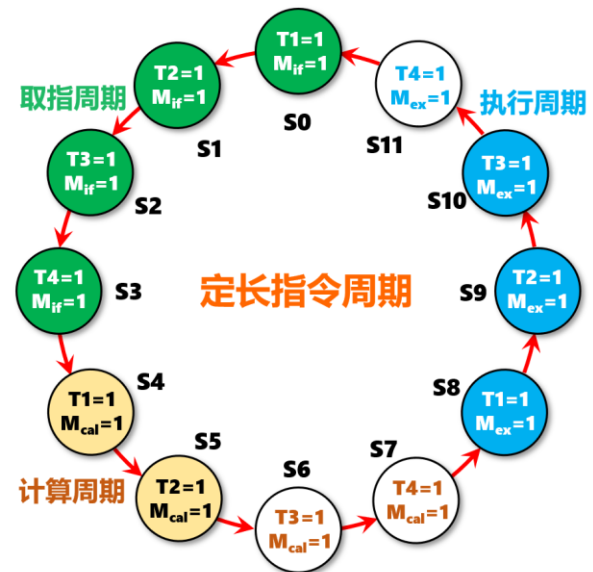


图6.40 定长指令周期三级时序状态机

6.4.3 时序发生器

— (2) 变长指令周期三级时序状态机

- 变长指令周期三级时序的状态周期 (M_{if} 、 M_{cal} 、 M_{ex}) 和节拍电位 (T_1 、 T_2 、 T_3 、 T_4) 除与时钟信号CLK有关, 还与指令译码、反馈信号有关。
- 其时序发生器也可以采用有限状态机实现 (图6.41, 图中没有给出反馈信号), 共有9个状态, 其中 $S_0 \sim S_3$ 为取指周期, $S_4 \sim S_5$ 为计算周期, $S_6 \sim S_8$ 为执行周期。
- lw、sw、beq指令有9个状态, add、addi指令只有7个状态 (S_0 - S_3 - S_6 - S_8)。

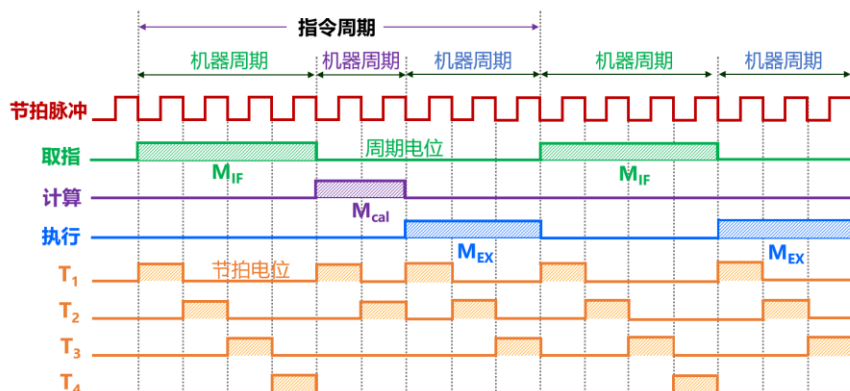


图6.38 变长指令周期的三级时序

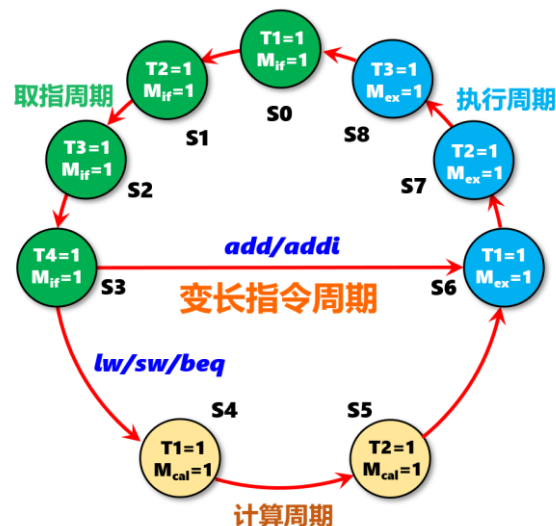


图6.41 变长指令周期三级时序状态机

- 表6.12: 变长指令周期时序发生器的状态转换表（输入为现态，输出为次态），例如：
 - 现在的状态为 $S_0 (M_{if}, T_1)$ ，对于所有的指令，下一个状态都是 $S_1 (M_{if}, T_2)$ ；即有限状态机的现态= S_0 ，输入=任意指令，则次态= S_1 。
 - 现在的状态为 $S_3 (M_{if}, T_4)$ ，对于add或addi指令，下一个状态是 $S_6 (M_{ex}, T_1)$ ；对于lw或sw或beq指令，下一个状态是 $S_4 (M_{cal}, T_1)$ ；即有限状态机的现态= S_3 ，输入=add/addi，则次态= S_6 ，输入=lw/sw/beq，则次态= S_4 。
- 图6.42: 时序发生器内部实现逻辑（上图：定长指令周期，下图：变长指令周期）；图中的FSM为有限状态机（Finite State Machine）。

表6.12 时序发生器状态转换表（变长指令周期）

现态	输入/次态			周期、节拍电位输出
	任意指令	add/addi	lw/sw/beq	
$S_0 (0000)$	$S_1 (0001)$			$M_{if}=T_1=1$
$S_1 (0001)$	$S_2 (0010)$			$M_{if}=T_2=1$
$S_2 (0010)$	$S_3 (0011)$			$M_{if}=T_3=1$
$S_3 (0011)$		$S_6 (0110)$	$S_4 (0100)$	$M_{if}=T_4=1$
$S_4 (0100)$	$S_5 (0101)$			$M_{cal}=T_1=1$
$S_5 (0101)$	$S_6 (0110)$			$M_{cal}=T_2=1$
$S_6 (0110)$	$S_7 (0111)$			$M_{ex}=T_1=1$
$S_7 (0111)$	$S_8 (1000)$			$M_{ex}=T_2=1$
$S_8 (1000)$	$S_0 (0000)$			$M_{ex}=T_3=1$

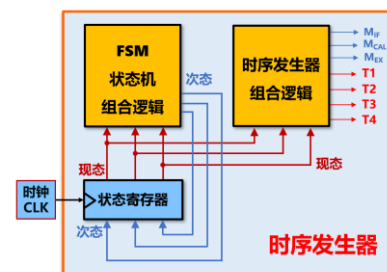


图6.42 时序发生器内部实现逻辑（定长指令周期）

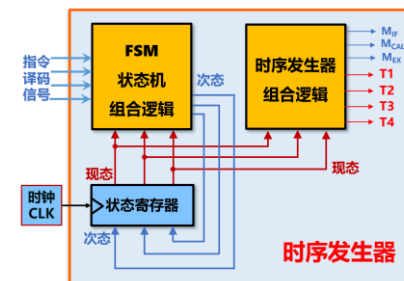
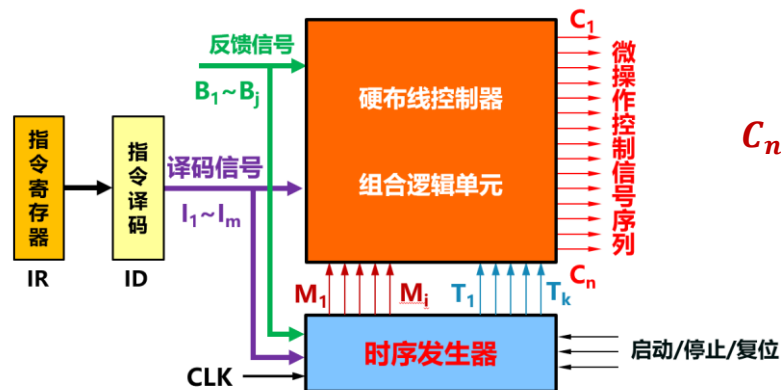


图6.42 时序发生器内部实现逻辑（变长指令周期）

6.5 硬布线控制器

• 6.5.1 三级时序硬布线控制器

- 硬布线控制器也称为**组合逻辑控制器**；控制器由组合逻辑电路实现，控制器**结构复杂**，但是**速度较快**；**RISC**处理器通常采用硬布线控制器。
- 图6.43：传统三级时序硬布线控制器模型，图中的时序发生器见图6.42。
- 硬布线控制器组合逻辑单元的**输入**为：指令译码器的输出、反馈信号、时序发生器的输出；**输出**为微操作控制信号序列（即表6.1单总线结构计算机、表6.9单周期处理器、表6.10多周期处理器的**控制信号**）。



$$C_n = \sum_{m,i,k,j} (I_m \cdot M_i \cdot T_k \cdot B_j)$$

图6.43 传统三级时序硬布线控制器模型

6.5.1 三级时序硬布线控制器

— 硬布线控制器的一般设计流程：

① 分析指令执行的数据通路，画出指令周期流程图，列出各节拍的控制信号。

② 设计时序发生器：根据机器周期、节拍划分构建状态图（图6.40：定长指令周期的状态图，图6.41：变长指令周期的状态图）。

③ 找出同一微操作控制信号产生条件。

$$C_n = \sum_{m,i,k,j} (I_m \cdot M_i \cdot T_k \cdot B_j)$$

④ 写出各微操作控制信号的逻辑表达式。

⑤ 利用组合逻辑电路（逻辑门、PLA、ROM）实现；PLA：Programmable Logic Arrays，可编程逻辑阵列。

6.5.2 三级时序硬布线控制器设计

– 1、单总线结构处理器硬布线控制器

- 图6.8为单总线结构的计算机框图；表6.1为单总线结构计算机的22个控制信号；图6.14为单总线结构计算机5条指令（lw、sw、beq、add、addi）的执行流程图。
- 以 X_{in} 、 Z_{out} 、 PC_{in} 控制信号的实现为例，其他17个控制信号的实现方式类似。

表6.1 控制信号及其功能

序号	控制信号	功能说明
1	PC_{in}	控制PC接收来自内总线的数据，需配合时钟控制
2	PC_{out}	控制PC向内总线输出数据
3	AR_{in}	控制AR接收来自内总线的数据，需配合时钟控制
4	DR_{in}	控制DR接收来自内总线的数据，需配合时钟控制
5	DR_{out}	控制DR向内总线输出数据
6	DRE_{in}	控制DR接收从主存读出的数据，需配合时钟控制
7	DRE_{out}	控制DR向主存输出数据，以便最后将该数据写入主存
8	X_{in}	控制暂存器X接收来自内总线的数据，需配合时钟控制
9	+4	将ALU的A端口的数据加4输出
10	ADD	控制ALU执行加法，实现A端口和B端口的两个数相加
11	SUB	控制ALU执行加法，实现A端口和B端口的两个数相减
12	PSW_{in}	控制程序状态字寄存器PSW接收ALU的运算状态，需配合时钟控制
13	Z_{out}	控制暂存器Z向内总线输出数据，注意暂存器Z没有输入控制，每个时钟都会锁存新值
14	IR_{in}	控制IR接收来自内总线的数据，需配合时钟控制
15	$IR(A)_{out}$	控制IR中的分支目标地址输出到内总线，指令字中的立即数要转换成目标地址，需要相应逻辑
16	$IR(I)_{out}$	控制IR中的立即数输出到内总线，指令字中的立即数符号扩展为32位才能输出
17	Write	存储器写命令，需配合时钟控制
18	Read	存储器读命令
19	R_{in}	控制寄存器堆接收来自内总线的数据，写入W#端口对应的寄存器中，需配合时钟控制
20	R_{out}	控制寄存器堆输出指定编号R#寄存器的数据，该寄存器堆为单端口输出
21	Rs/Rt	控制多路选择器选择送入R#的寄存器编号，为0时送入指令字中的rs字段，为1时送入rt
22	RegDst	控制多路选择器选择送入W#的寄存器编号，为0时送入指令字中的rt字段，为1时送入rd

6.5.2 三级时序硬布线控制器设计

- （1）分析表6.3～表6.7的**控制信号** x_{in} ：

① 所有指令的取指周期的 T_1 节拍都要发出 x_{in} 信号： $M_{if} \cdot T_1$

② 所有指令的计算指令的 T_1 节拍都要发出 x_{in} 信号： $M_{cal} \cdot T_1$

③ beq、addi、add指令的执行指令的 T_1 节拍都要发出 x_{in} 信号： $M_{ex} \cdot T_1 \cdot (beq + addi + add)$

— 因此，控制信号 x_{in} 的逻辑表达式为： $x_{in} = M_{if} \cdot T_1 + M_{cal} \cdot T_1 + M_{ex} \cdot T_1 \cdot (beq + addi + add)$

表6.3 lw指令操作流程及控制信号

周期	节拍	控制信号
取指周期 M_{if}	T_1	$PC_{out}=AR_{in}=X_{in}=1$
	T_2	+4=1
	T_3	$Z_{out}=PC_{in}=1$ $Read=DRE_{in}=1$
	T_4	$DR_{out}=IR_{in}=1$
计算周期 M_{cal}	T_1	$R_{out}=X_{in}=1$
	T_2	$IR(I)_{out}=ADD=1$
执行周期 M_{ex}	T_1	$Z_{out}=AR_{in}=1$
	T_2	$Read=DRE_{in}=1$
	T_3	$DR_{out}=R_{in}=1$

表6.4 sw指令操作流程及控制信号

周期	节拍	控制信号
取指周期 M_{if}	T_1	$PC_{out}=AR_{in}=X_{in}=1$
	T_2	+4=1
	T_3	$Z_{out}=PC_{in}=1$ $Read=DRE_{in}=1$
	T_4	$DR_{out}=IR_{in}=1$
计算周期 M_{cal}	T_1	$R_{out}=X_{in}=1$
	T_2	$IR(I)_{out}=ADD=1$
执行周期 M_{ex}	T_1	$Z_{out}=AR_{in}=1$
	T_2	$R_{out}=Rs/Rt=DR_{in}=1$
	T_3	$DRE_{out}=Write=1$

表6.5 beq指令操作流程及控制信号

周期	节拍	控制信号
取指周期 M_{if}	T_1	$PC_{out}=AR_{in}=X_{in}=1$
	T_2	+4=1
	T_3	$Z_{out}=PC_{in}=1$ $Read=DRE_{in}=1$
	T_4	$DR_{out}=IR_{in}=1$
计算周期 M_{cal}	T_1	$R_{out}=X_{in}=1$
	T_2	$R_{out}=Rs/Rt=SUB=PSW_{in}=1$
执行周期 M_{ex}	T_1	$PC_{out}=X_{in}=1$
	T_2	$IR(A)_{out}=ADD=1$
	T_3	$Z_{out}=1$ $PC_{in}=PSW_{equal}$

表6.6 addi指令操作流程及控制信号

周期	节拍	控制信号
取指周期 M_{if}	T_1	$PC_{out}=AR_{in}=X_{in}=1$
	T_2	+4=1
	T_3	$Z_{out}=PC_{in}=1$ $Read=DRE_{in}=1$
	T_4	$DR_{out}=IR_{in}=1$
执行周期 M_{ex}	T_1	$R_{out}=X_{in}=1$
	T_2	$IR(I)_{out}=ADD=1$
	T_3	$Z_{out}=R_{in}=1$

表6.7 add指令操作流程及控制信号

周期	节拍	控制信号
取指周期 M_{if}	T_1	$PC_{out}=AR_{in}=X_{in}=1$
	T_2	+4=1
	T_3	$Z_{out}=PC_{in}=1$ $Read=DRE_{in}=1$
	T_4	$DR_{out}=IR_{in}=1$
执行周期 M_{ex}	T_1	$R_{out}=X_{in}=1$
	T_2	$R_{out}=Rs/Rt=ADD=1$
	T_3	$Z_{out}=RegDst=R_{in}=1$

6.5.2 三级时序硬布线控制器设计

• (2) 分析表6.3 ~ 表6.7的控制信号 Z_{out} :

① 所有指令的取指周期的 T_3 节拍 ($Z \rightarrow PC$) 都要发出 Z_{out} 信号: $M_{if} \cdot T_3$

② lw 、 sw 指令的执行周期的 T_1 节拍 ($Z \rightarrow AR$) 都要发出 Z_{out} 信号: $M_{ex} \cdot T_1 \cdot (lw + sw)$

③ Beq ($Z \rightarrow PC$)、 $addi$ ($Z \rightarrow R[rt]$)、 add ($Z \rightarrow R[rd]$) 指令的执行周期的 T_3 节拍都要发出 Z_{out} 信号: $M_{ex} \cdot T_3 \cdot (beq + addi + add)$

— 因此, 控制信号 Z_{out} 的逻辑表达式为: $Z_{out} = M_{if} \cdot T_3 + M_{ex} \cdot T_1 \cdot (lw + sw) + M_{ex} \cdot T_3 \cdot (beq + addi + add)$

表6.3 lw 指令操作流程及控制信号

周期	节拍	控制信号
取指周期 M_{if}	T_1	$PC_{out} = AR_{in} = X_{in} = 1$
	T_2	$+4=1$
	T_3	$Z_{out} = PC_{in} = 1$ $Read = DRE_{in} = 1$
	T_4	$DR_{out} = IR_{in} = 1$
计算周期 M_{cal}	T_1	$R_{out} = X_{in} = 1$
	T_2	$IR(I)_{out} = ADD = 1$
执行周期 M_{ex}	T_1	$Z_{out} = AR_{in} = 1$
	T_2	$Read = DRE_{in} = 1$
	T_3	$DR_{out} = R_{in} = 1$

表6.4 sw 指令操作流程及控制信号

周期	节拍	控制信号
取指周期 M_{if}	T_1	$PC_{out} = AR_{in} = X_{in} = 1$
	T_2	$+4=1$
	T_3	$Z_{out} = PC_{in} = 1$ $Read = DRE_{in} = 1$
	T_4	$DR_{out} = IR_{in} = 1$
计算周期 M_{cal}	T_1	$R_{out} = X_{in} = 1$
	T_2	$IR(I)_{out} = ADD = 1$
执行周期 M_{ex}	T_1	$Z_{out} = AR_{in} = 1$
	T_2	$R_{out} = Rs/Rt = DR_{in} = 1$
	T_3	$DR_{out} = Write = 1$

表6.5 beq 指令操作流程及控制信号

周期	节拍	控制信号
取指周期 M_{if}	T_1	$PC_{out} = AR_{in} = X_{in} = 1$
	T_2	$+4=1$
	T_3	$Z_{out} = PC_{in} = 1$ $Read = DRE_{in} = 1$
	T_4	$DR_{out} = IR_{in} = 1$
计算周期 M_{cal}	T_1	$R_{out} = X_{in} = 1$
	T_2	$R_{out} = Rs/Rt = SUB = PSW_{in} = 1$
执行周期 M_{ex}	T_1	$PC_{out} = X_{in} = 1$
	T_2	$IR(A)_{out} = ADD = 1$
	T_3	$Z_{out} = 1$ $PC_{in} = PSW_{equal}$

表6.6 $addi$ 指令操作流程及控制信号

周期	节拍	控制信号
取指周期 M_{if}	T_1	$PC_{out} = AR_{in} = X_{in} = 1$
	T_2	$+4=1$
	T_3	$Z_{out} = PC_{in} = 1$ $Read = DRE_{in} = 1$
	T_4	$DR_{out} = IR_{in} = 1$
执行周期 M_{ex}	T_1	$R_{out} = X_{in} = 1$
	T_2	$IR(I)_{out} = ADD = 1$
	T_3	$Z_{out} = R_{in} = 1$

表6.7 add 指令操作流程及控制信号

周期	节拍	控制信号
取指周期 M_{if}	T_1	$PC_{out} = AR_{in} = X_{in} = 1$
	T_2	$+4=1$
	T_3	$Z_{out} = PC_{in} = 1$ $Read = DRE_{in} = 1$
	T_4	$DR_{out} = IR_{in} = 1$
执行周期 M_{ex}	T_1	$R_{out} = X_{in} = 1$
	T_2	$R_{out} = Rs/Rt = ADD = 1$
	T_3	$Z_{out} = RegDst = R_{in} = 1$

6.5.2 三级时序硬布线控制器设计

- （3）分析表6.3～表6.7的**控制信号PC_{in}**：

① 所有指令的取指周期的T₃节拍都要发出PC_{in}信号： $M_{if} \cdot T_3$

② 当PSW的equal为1时，beq指令的执行周期的T₃节拍（分支目标地址送入PC）要发出PC_{in}信号： $M_{ex} \cdot T_3 \cdot beq \cdot equal$

— 因此，控制信号PC_{in}的逻辑表达式为： $PC_{in} = M_{if} \cdot T_3 + M_{ex} \cdot T_3 \cdot beq \cdot equal$

表6.3 lw指令操作流程及控制信号

周期	节拍	控制信号
取指周期 M_{if}	T ₁	$PC_{out}=AR_{in}=X_{in}=1$
	T ₂	+4=1
	T ₃	$Z_{out}=PC_{in}=1$ $Read=DRE_{in}=1$
	T ₄	$DR_{out}=IR_{in}=1$
计算周期 M_{cal}	T ₁	$R_{out}=X_{in}=1$
	T ₂	$IR(I)_{out}=ADD=1$
执行周期 M_{ex}	T ₁	$Z_{out}=AR_{in}=1$ $Read=DRE_{in}=1$
	T ₂	$DR_{out}=R_{in}=1$

表6.4 sw指令操作流程及控制信号

周期	节拍	控制信号
取指周期 M_{if}	T ₁	$PC_{out}=AR_{in}=X_{in}=1$
	T ₂	+4=1
	T ₃	$Z_{out}=PC_{in}=1$ $Read=DRE_{in}=1$
	T ₄	$DR_{out}=IR_{in}=1$
计算周期 M_{cal}	T ₁	$R_{out}=X_{in}=1$
	T ₂	$IR(I)_{out}=ADD=1$
执行周期 M_{ex}	T ₁	$Z_{out}=AR_{in}=1$
	T ₂	$R_{out}=Rs/Rt=DR_{in}=1$ $DR_{out}=Write=1$

表6.5 beq指令操作流程及控制信号

周期	节拍	控制信号
取指周期 M_{if}	T ₁	$PC_{out}=AR_{in}=X_{in}=1$
	T ₂	+4=1
	T ₃	$Z_{out}=PC_{in}=1$ $Read=DRE_{in}=1$
	T ₄	$DR_{out}=IR_{in}=1$
计算周期 M_{cal}	T ₁	$R_{out}=X_{in}=1$
	T ₂	$R_{out}=Rs/Rt=SUB=PSW_{in}=1$
执行周期 M_{ex}	T ₁	$PC_{out}=X_{in}=1$
	T ₂	$IR(A)_{out}=ADD=1$
	T ₃	$Z_{out}=1$ $PC_{in}=PSW_{equal}$

表6.6 addi指令操作流程及控制信号

周期	节拍	控制信号
取指周期 M_{if}	T ₁	$PC_{out}=AR_{in}=X_{in}=1$
	T ₂	+4=1
	T ₃	$Z_{out}=PC_{in}=1$ $Read=DRE_{in}=1$
	T ₄	$DR_{out}=IR_{in}=1$
执行周期 M_{ex}	T ₁	$R_{out}=X_{in}=1$
	T ₂	$IR(I)_{out}=ADD=1$
	T ₃	$Z_{out}=R_{in}=1$

表6.7 add指令操作流程及控制信号

周期	节拍	控制信号
取指周期 M_{if}	T ₁	$PC_{out}=AR_{in}=X_{in}=1$
	T ₂	+4=1
	T ₃	$Z_{out}=PC_{in}=1$ $Read=DRE_{in}=1$
	T ₄	$DR_{out}=IR_{in}=1$
执行周期 M_{ex}	T ₁	$R_{out}=X_{in}=1$
	T ₂	$R_{out}=Rs/Rt=ADD=1$
	T ₃	$Z_{out}=R_{out}Dst=R_{in}=1$

—2、单周期MIPS处理器硬布线控制器

- 图6.25为单周期MIPS处理器的数据通路。
- 表6.9为单周期MIPS处理器的8个控制信号。
- 由于单周期处理器每条指令都在1个时钟周期内执行完成，没有状态周期（机器周期，即取指指令、计算周期、执行周期）和节拍电位（即 T_1 、 T_2 、 T_3 、 T_4 ）。
- 因此，单周期处理器的控制信号只与指令译码信号有关。

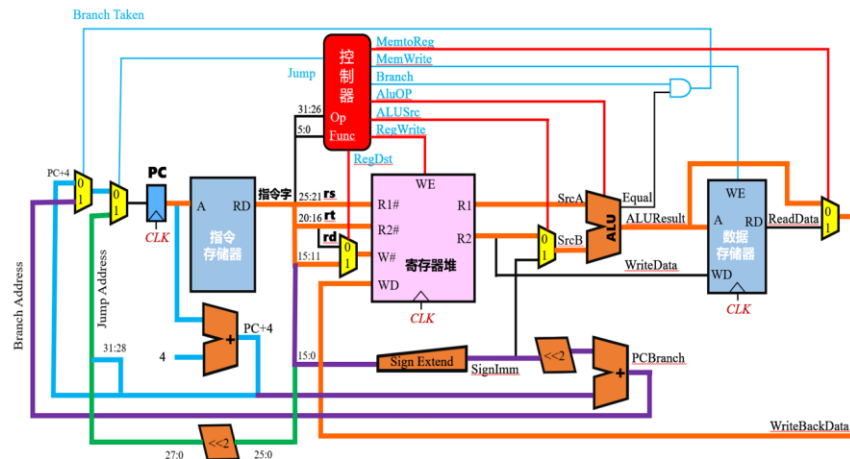


图6.25 单周期MIPS处理器的数据通路

表6.9 单周期MIPS处理器的控制信号及功能

控制信号	信号分类	功能说明
Jump	指令译码信号	j 、 jal 等无条件分支指令时生成
Branch	指令译码信号	分支指令译码信号， beq 、 bne 指令需要产生类似的译码信号
RegDst	多路选择控制	写入目的寄存器的选择，为 1 时写入 rd 寄存器，为 0 时写入 rt 寄存器
AluSrc	多路选择控制	控制 ALU 的第二输入，为 0 时输入 R2 （ rt 寄存器），为 1 时输入扩展后的立即数
MenToReg	多路选择控制	为 1 时将从数据存储器读出的数据写回寄存器堆，为 0 时将 ALU 的运算结果写回寄存器堆
RegWrite	功能部件控制	控制寄存器堆写操作，为 1 时数据需要写入指定的寄存器，受时钟驱动
AluOp	功能部件控制	控制 ALU 进行不同的运算，具体取指和位宽（如 AluOP 为4位）与 ALU 的设计有关
MemWrite	功能部件控制	控制数据存储器的写操作，为 1 时进行写操作，为 0 时进行读操作，受时钟驱动

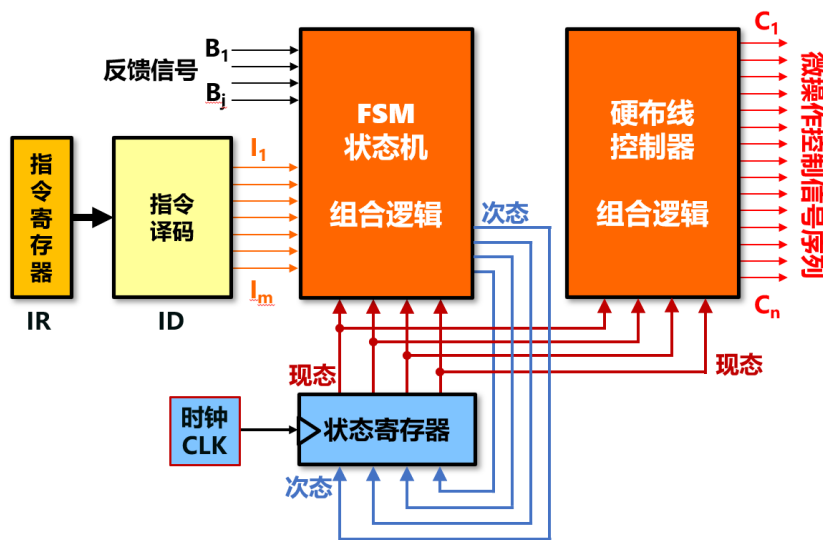
- 假设该单周期MIPS处理器需用支持add、sub、and、addi、andi、lw、sw、beq、bne、j、jal等**11条指令**；通过分析前面的图6.17、图6.18、图6.19、图6.20、图6.21、图6.22、图6.24、图6.25，可以知道（表6.13）：
 - Jump控制信号由j、jal两条无条件转移指令的译码信号生成，即：**Jump = j + jal**（注：这里的“+”表示逻辑或）
 - Branch控制信号由beq、bne两条条件转移指令的译码信号生成，即：**Branch = beq + bne**
 - RegDst控制信号由add、sub、and三条R型运算类指令的译码信号生成，即：**RegDst = add + sub + and**
 - AluSrc控制信号由addi、andi（**andi rt,rs,imm**）两条I型运算类指令的译码信号生成，即：**AluSrc = addi + andi**
 - MemToReg控制信号由lw取数指令的译码信号生成，即：**MemToReg = lw**
 - RegWrite控制信号由add、sub、and、addi、andi、lw六条需用写回寄存器的指令的译码信号生成，即：
RegWrite = add + sub + and + addi + andi + lw
 - AluOp控制信号由add、sub、and、addi、andi五条运算类指令的译码信号生成，通常**AluOp为4位**，可以对应**16种运算功能**，即：**AluOp = f(add、sub、and、addi、andi)**
 - MemWrite控制信号由sw存数指令的译码信号生成，即：**MemWrite = sw**

表6.13 单周期MIPS控制器逻辑表达式

序号	控制信号逻辑表达式	功能说明
1	Jump = j+jal	j、jal等无条件分支指令时生成
2	Branch = beq+bne	分支指令译码信号，beq、bne指令需要产生类似的译码信号
3	RegDst = add + sub + and	写入目的寄存器的选择，为1时写入rd寄存器，为0时写入rt寄存器
4	AluSrc = addi + andi	控制ALU的第二输入，为0时输入R2（rt寄存器），为1时输入扩展后的立即数
5	MenToReg = lw	为1时将从数据存储器读出的数据写回寄存器堆，为0时将ALU的运算结果写回寄存器堆
6	RegWrite = add + sub + and + addi + andi + lw	控制寄存器堆写操作，为1时数据需要写入指定的寄存器，受时钟驱动
7	AluOp = f(add、sub、and、addi、andi)	控制ALU进行不同的运算，具体取指和位宽（如AluOP为4位）与ALU的设计有关
8	MemWrite = sw	控制数据存储器的写操作，为1时进行写操作，为0时间读操作，受时钟驱动

6.5.3 现代时序硬布线控制器

- 现代计算机已经不再使用多级时序系统，指令执行过程中的定时信号就是基本时钟，**一个时钟周期就是一个节拍**。
- 图6.44：现代时序硬布线控制器模型；图中FSM状态机的**输入**为现态（当前的状态）、指令译码信号、反馈信号，**输出**为次态（下一个状态）。
- 硬布线控制器的**输入**为现态，**输出**为微操作控制信号序列（控制信号）。



$$C_n = f(\text{现态})$$

图6.44 现代时序硬布线控制器模型

6.5.3 现代时序硬布线控制器

— 现代时序硬布线控制器的一般设计流程：

① 分析数据通路，画出指令周期流程图，确定各节拍（时钟周期）的控制信号。

② 绘制指令执行状态转换图。

③ 构建状态转换表。

④ 实现有限状态机FSM组合逻辑。

$$C_n = f(\text{现态})$$

⑤ 实现硬布线控制器组合逻辑电路。

6.5.4 现代时序硬布线控制器设计

– 1、单总线结构处理器硬布线控制器

- 根据图6.14的单总线结构计算机指令执行流程，可以得到图6.45的指令执行状态转换图，共有**25个状态**（对应图6.14中的25个长方形框）。

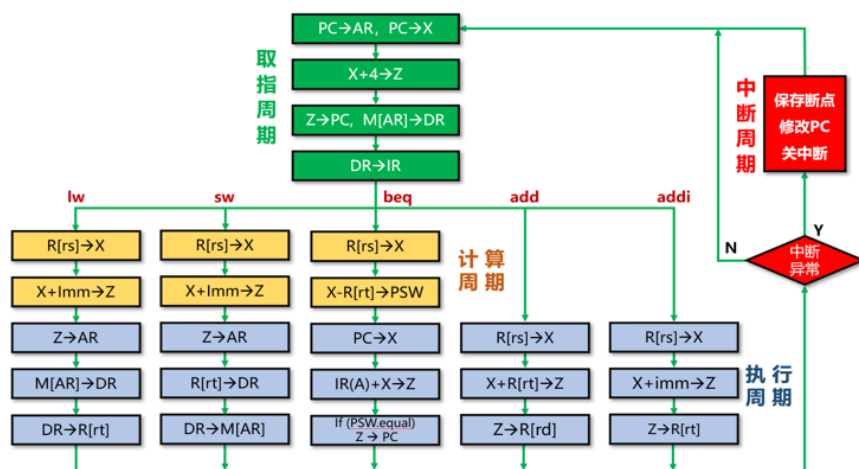


图6.14 单总线结构数据通路指令方框图

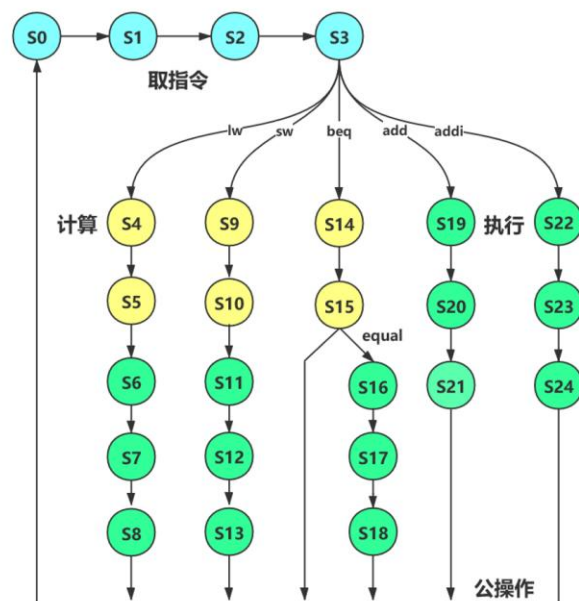


图6.45 指令执行状态转换图（单总线结构）

- 表6.14：图6.45对应的状态转换表；表中增加了1个状态 S_{26} （中断响应周期状态）；在每条指令的执行周期结束时，如果有中断请求，则进入中断响应周期。

表6.14 现代时序指令状态转换表（单总线结构）

现态	输入/次态								操作控制信号输出
	xxx	lw	sw	beq	add	addi	equal	有中断	
S_0	S_1								$PC_{out}=AR_{in}=X_{in}=1$
S_1	S_2								+4=1
S_2	S_3								
S_3		S_4	S_9	S_{14}	S_{19}	S_{22}			
S_4	S_5								
S_5	S_6								
S_6	S_7								
S_7	S_8								
S_8	S_0							S_{26}	
S_9	S_{10}								
S_{10}	S_{11}								
S_{11}	S_{12}								
S_{12}	S_{13}								
S_{13}	S_0							S_{26}	
S_{14}	S_{15}								
S_{15}							S_{16}		
S_{15}	S_0							S_{26}	
S_{16}	S_{17}								
S_{17}	S_{18}								
S_{18}	S_0							S_{26}	
S_{19}	S_{20}								
S_{20}	S_{21}								
S_{21}	S_0							S_{26}	
S_{22}	S_{23}								
S_{23}	S_{24}								
S_{24}	S_0							S_{26}	$Z_{out}=R_{in}=1$

– 2、多周期MIPS处理器硬布线控制器

- 例6.5：请采用现代时序设计图6.27所示的多周期MIPS处理器数据通路中的**硬布线控制器**，要求支持表6.2所示的典型MIPS32指令（前5条：lw、sw、beq、add、addi，图6.35）。

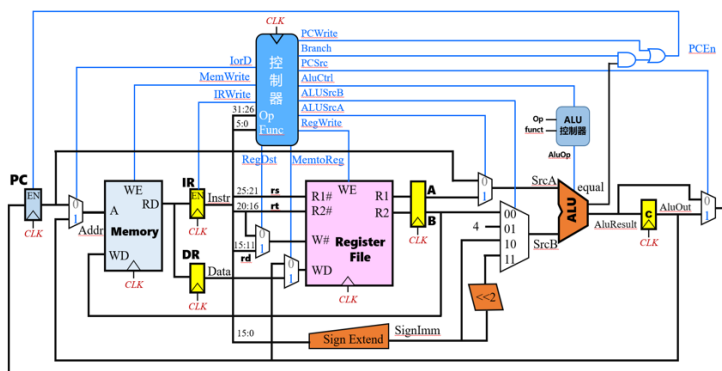


图6.27 多周期MIPS处理器的数据通路

表6.2 典型MIPS 32指令

序号	指令	汇编代码	指令类型	RTL功能说明	指令功能
1	lw	lw rt,imm(rs)	I型	$R[rt] \leftarrow M[R[rs] + \text{SignExt}(imm)]$	取数指令：寄存器 ← 存储器
2	sw	sw rt,imm(rs)	I型	$M[R[rs] + \text{SignExt}(imm)] \leftarrow R[rt]$	存取指令：存储器 ← 寄存器
3	beq	beq rs,rt,imm	I型	$\text{if}(R[rs] == R[rt]) \quad PC \leftarrow PC + 4 + \text{SignExt}(imm) \ll 2$	条件分支指令：如果rs = rt，则跳转
4	addi	addi rt,rs,imm	I型	$R[rt] \leftarrow R[rs] + \text{SignExt}(imm)$	加法指令：寄存器 ← 寄存器 + 立即数
5	add	add rd,rs,rt	R型	$R[rd] \leftarrow R[rs] + R[rt]$	加法指令：寄存器 ← 寄存器 + 寄存器
6	slt	slt rd,rs,rt	R型	$R[rd] \leftarrow (R[rs] < R[rt]) ? 1 : 0$	比较指令：如果rs < rt，则置rd=1；否则，置rd=0
7	j	j imm26	J型	$PC \leftarrow \{(PC+4)_{31:26}, imm26 \ll 2\}$	无条件转移指令

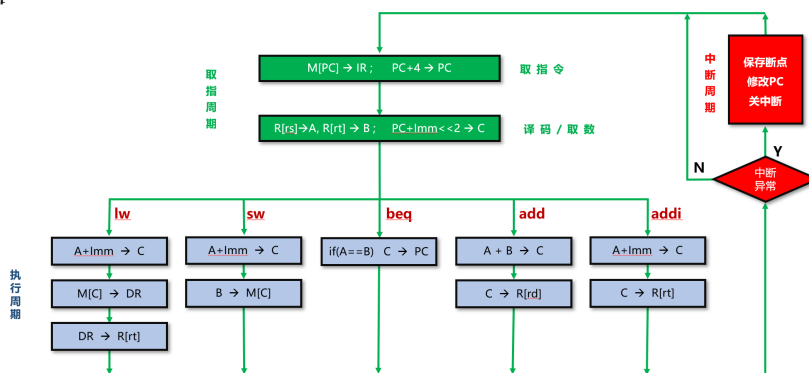


图6.35 多周期MIPS处理器数据通路的指令图

• 解:

- 根据图6.35的多周期MIPS处理器指令执行流程,可以得到图6.46的指令执行状态转换图; 共有12个状态 ($S_0 \sim S_{11}$), 对应图6.35中的12个长方形框。

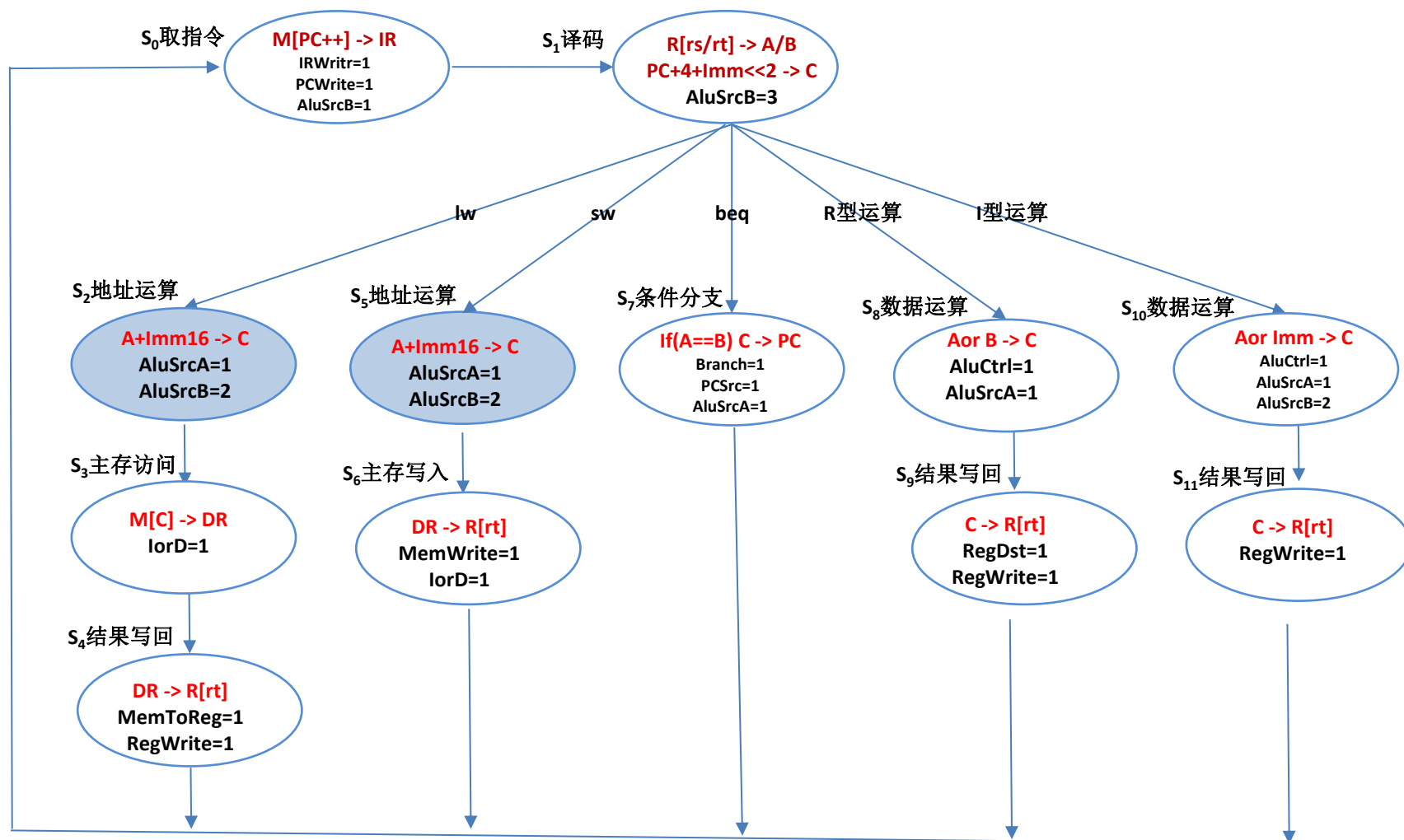


图6.46 多周期MIPS指令执行状态转换图

- 表6.15：图6.46对应的指令状态转换表（12个状态： $S_0 \sim S_{11}$ ）。

表6.15 多周期MIPS指令状态转换表

现态	输入/次态						操作控制信号输出
	xxx	lw	sw	beq	add	addi	
S_0	S_1						IRWrite=PCWrite=1 AluSrcB=1
S_1		S_2	S_5	S_7	S_8	S_{10}	AluSrcB=3
S_2	S_3						
S_3	S_4						
S_4	S_0						
S_5	S_6						
S_6	S_0						
S_7	S_0						
S_8	S_9						
S_9	S_0						
S_{10}	S_{11}						
S_{11}	S_0						RegWrite=1

6.6 微程序控制器

• 6.6.1 微程序控制的基本概念

- 微程序控制的基本思想：仿照程序设计的基本方法，将实现指令功能所需要的控制信号（即微操作控制信号），按照一定的规则编码成**微指令**，存放在**控制存储器**（简称**控存**，通常采用**ROM**存储器）中。
- 一条指令对应一段**微程序**（若干条微指令），执行指令的过程就是执行微程序的过程。
- 微程序控制器的设计采用了存储技术和程序设计技术，相比硬布线控制器中的**硬件时序**，微程序控制器是一种**软件时序**。
- 微程序控制器使复杂的控制逻辑得到**简化**。

6.6.1 微程序控制的基本概念

– 1、微命令与微操作

- 控制器发出的控制信号称为**微命令**，也称为**微操作**。
- 微操作可分为相容性和互斥性两种：**相容性微操作**是指能在一个时钟周期内同时执行的微操作；**互斥性微操作**是指不能在同一个时钟周期内并行执行的微操作。
 - 例如，单总线结构计算机（图6.8）中的存储器读写控制信号（Read、Write）、所有向内总线输出的控制信号（ PC_{out} 、 DR_{out} 、 Z_{out} 、 R_{out} 、 $IR(A)_{out}$ 、 $IR(I)_{out}$ ）、运算器控制信号（ADD、+4、SUB）、DR寄存器的两个输入控制信号（ DR_{in} 、 DRE_{in} ）都属于**互斥性微操作**。
- 从内总线向寄存器锁存数据的使能信号（ PC_{in} 、 AR_{in} 、 DR_{in} 、 X_{in} 、 R_{in} 、 IR_{in} ）、DR寄存器的两个输出控制信号（ DR_{out} 、 DRE_{out} ）都属于**相容性微操作**。
- 互斥性的微操作不能出现在同一条微指令中，只有相容性的微操作才能出现在同一条微指令中。

6.6.1 微程序控制的基本概念

— 2、微指令与微程序

- 在一个时钟周期内，一组实现一定操作功能的相容性微操作称为**微指令**。
- 图6.4的单总线结构计算机的微指令格式如图6.47，微指令包括操作控制字段和顺序控制字段。
- **操作控制字段**对应每一个控制信号（微命令、微操作），图6.8的单总线结构计算机有22个控制信号。
- **顺序控制字段**包括**判断测试字段**（P0、P1）和**下地址字段**（**下址字段**）；测试位P0（ P_{IR} ）用于取指周期结束，判断微程序进入到哪一条指令的计算周期或执行周期；测试位P1（ P_{equal} ）用于**beq**指令判断是否相等（**equal**）。
- 通常将取指周期的微指令编制成一段公共的微程序，称为**取指微程序**。

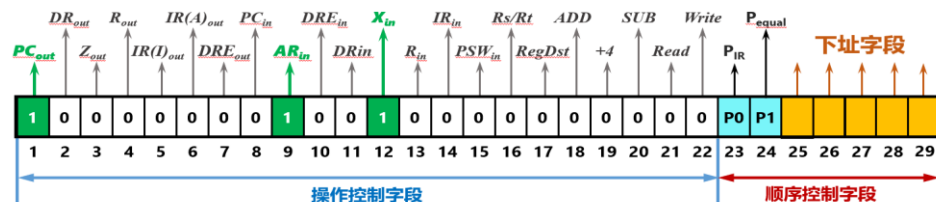


图6.47 微指令格式示例（单总线结构）

6.6.1 微程序控制的基本概念

— 3、微指令周期

- 将取出并执行一条微指令所需的时间定义为**微指令周期**（简称微周期）。
- 通常1个微指令周期对应1个机器周期；1个微指令周期包含4个节拍， T_1 节拍用于取微指令， $T_2 \sim T_4$ 节拍用于执行微指令；如图6.48所示。

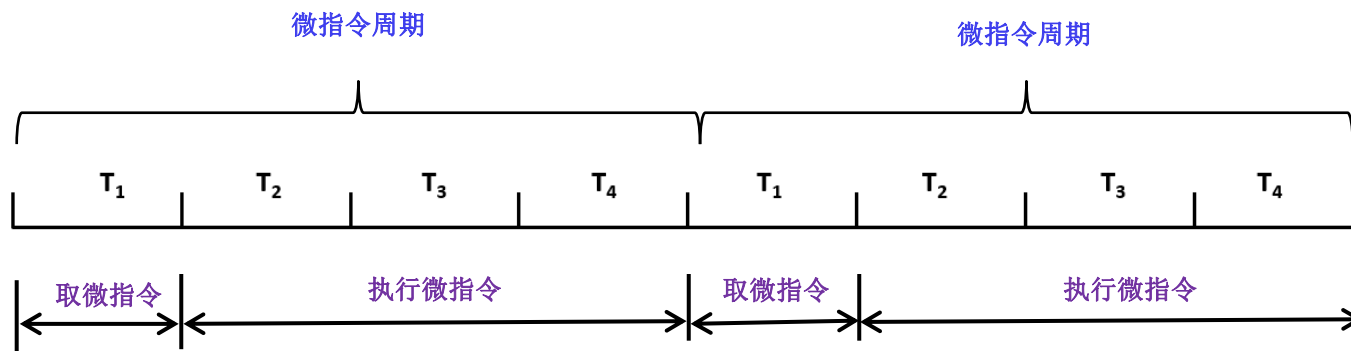


图6.48 微指令周期

6.6.2 微程序控制器组成原理

— 1、微程序控制器组成

- 图6.49：微程序控制器的组成框图，主要包括控制存储器（控存）、微地址寄存器（ μAR ）、地址转移逻辑等。

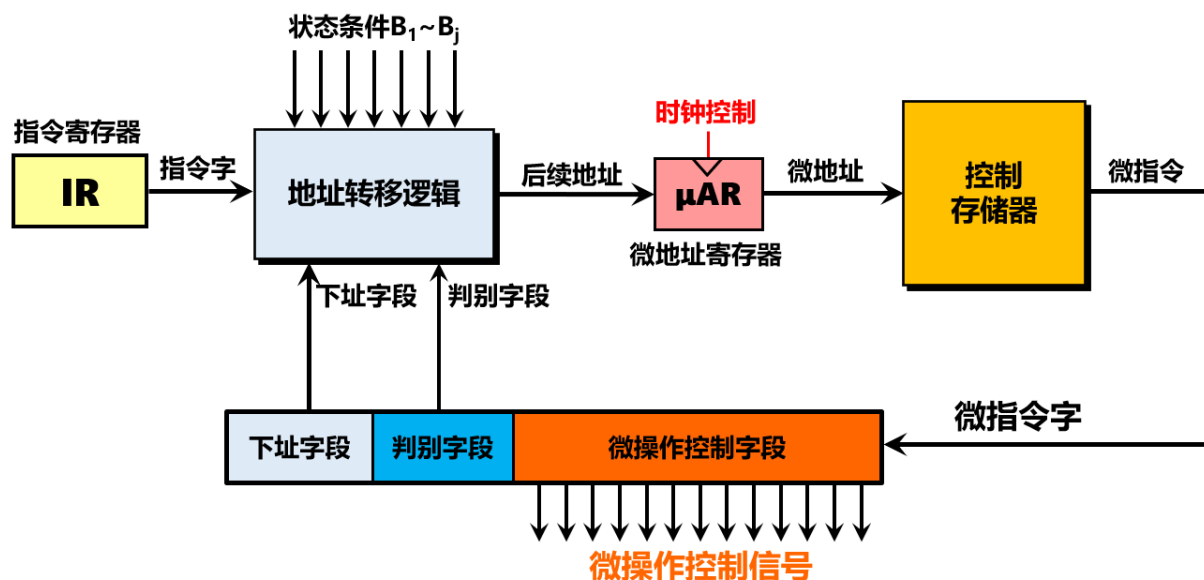


图6.49 微程序控制器组成框图

6.6.2 微程序控制器组成原理

- (1) 控制存储器
 - 控制存储器用于存放全部指令的所有微程序，控制存储器通常采用ROM。
 - 控制存储器的长度等于微指令的长度；控制存储器的容量等于所有指令的微程序包含的微指令数量，其取决于指令系统。
- (2) 微地址寄存器
 - 微地址寄存器 (μAR) 为控制存储器提供微地址（微指令的地址）。
 - 初始化时（上电，按Reset键） $\mu AR=0$ ；因此，控制存储器0号单元的微指令应该是取指微程序的第1条微指令。
 - 微地址寄存器 (μAR) 的输入为地址转移逻辑的输出（后续地址）， μAR 受时钟信号CLK的控制。
- (2) 地址转移逻辑
 - 地址转移逻辑用于产生后续地址（下一条微指令的地址）。
 - 地址转移逻辑的输入为指令译码信号（指令字）、状态条件（如PSW的equal、中断请求信号等）、微指令的判断测试字段、微指令的下址字段。

6.6.2 微程序控制器组成原理

– 2、后续微地址的形成

- 后续微地址的形成方法有两种：下址字段法和计数器法。
- **下址字段法**：在微指令中设置专门的**下址字段**，该方法又称为**断定法**；图6.49的微程序控制器就是采用下址字段法。
- **计数器法**：其思路与程序计数器PC相同，后续微指令的地址由 $\mu AR+1$ 得到，如图6.50所示；微指令不再包括下址字段部分，从而减少微指令的长度，减少控制存储器的容量开销。
- 采用计数器法时，需要在判别测试字段增加一个**结束微指令判断测试位** P_{end} ，用于判断当前微指令是不是该指令的最后一条微指令，如果是则转到取指微程序的第1条微指令。

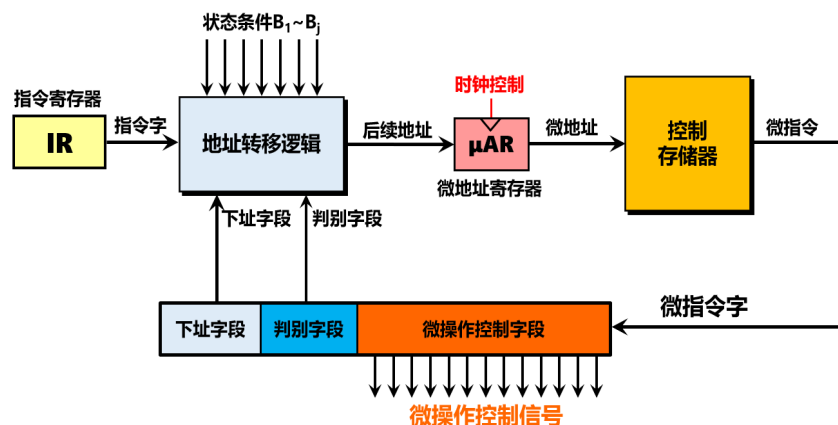


图6.49 微程序控制器组成框图（下址字段法）

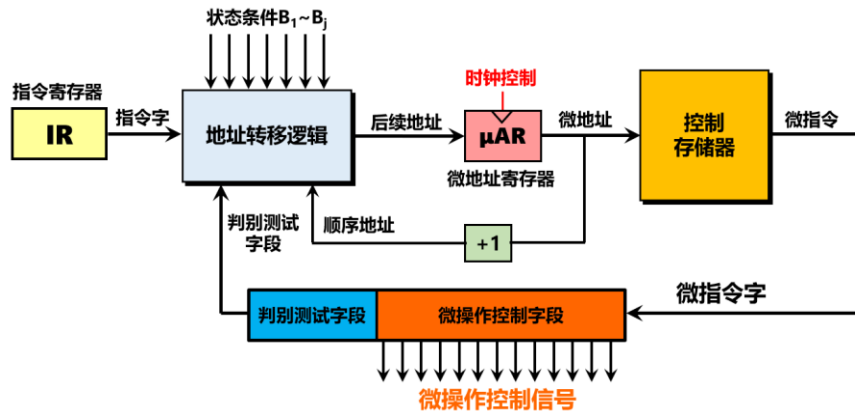


图6.50 微程序控制器组成框图（计数器法）

– 3、地址转移逻辑

- (1) 下址字段法地址转移逻辑

- 地址转移逻辑根据指令字（指令译码信号）、状态反馈条件、判别测试字段、下址字段，给出后续微地址；图6.51为采用下址字段法的地址转移逻辑实现方法。
- **微程序入口查找 $P_0=1$** ：将IR的指令字（指令译码信号）直接转换成微程序入口地址。
- **条件判别测试**：用于比较判别测试字段的 $P_1 \sim P_j$ 与状态反馈条件中的 $B_1 \sim B_j$ 是否相符，如果相符，则转到相应的微程序分支地址处（例如beq指令微程序的 P_1 （ P_{equal} ）判断测试位，当PSW中的equal=1时，表示条件成立）。
- 判别测试字段中的 P_0 （ P_{IR} ）用于指令译码测试， $P_0=1$ 时，微程序的后续地址由微程序入口查找给定（即每条指令的计算周期或执行周期对应的微指令入口地址）。

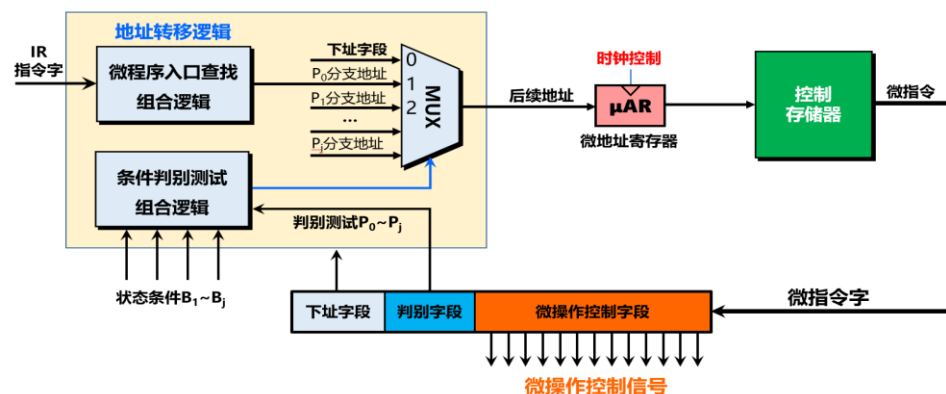


图6.51 地址转移逻辑（下址字段法）

- (2) 计数器法地址转移逻辑

- 图6.52为采用计数器法的地址转移逻辑实现方法。
- 计数器法微指令中没有下址字段，用顺序地址代替下址字段。
- 此外，微指令的判别测试字段中需要增加一个结束微指令判断测试位 P_2 (P_{end})，用于判断当前微指令是不是该指令的最后一微指令，如果是则转到取指微程序的第1条微指令的地址（取指微程序地址）。

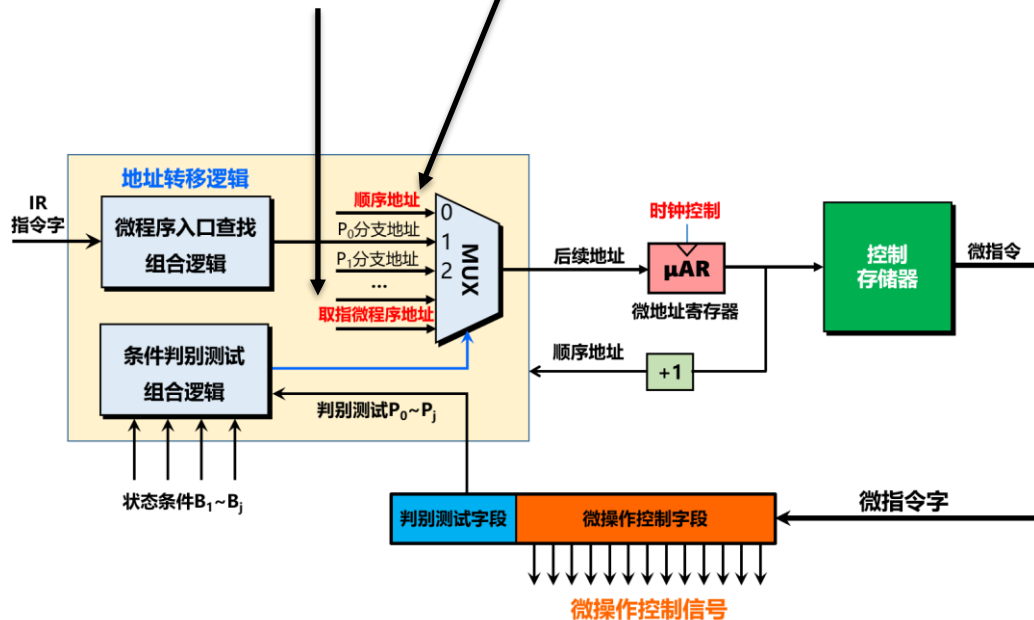


图6.52 地址转移逻辑（计数器法）

- 当指令系统规模较大、判别测试条件较多时，采用组合逻辑实现图6.51与图6.52中的微程序入口查找和条件判别测试，将变得较为复杂，也不便于扩展。
- 此时，可以采用**ROM**实现微程序入口查找和条件判别测试，如图6.53所示；该图为采用**计数器法**的地址转移逻辑，如果采用下址字段法，实现方法类似。
- 图6.53中，多路选择器MUX的输入有4个：
 - ① **顺序地址**：由 $\mu AR+1$ 得到；如果采用下址字段法，则为下址字段。
 - ② **P_0 分支地址**：以指令字（指令译码信号）为地址，从ROM1中得到，为每条指令的计算周期或执行周期对应的微指令**入口地址**。
 - ③ **0**：微程序的起始地址，即：**取指微程序的入口**，上电或按Reset键后， $\mu AR=0$ 。
 - ④ **分支地址**：为 P_1 、 P_2 、...、 P_{end} 判别测试位对应的分支地址。

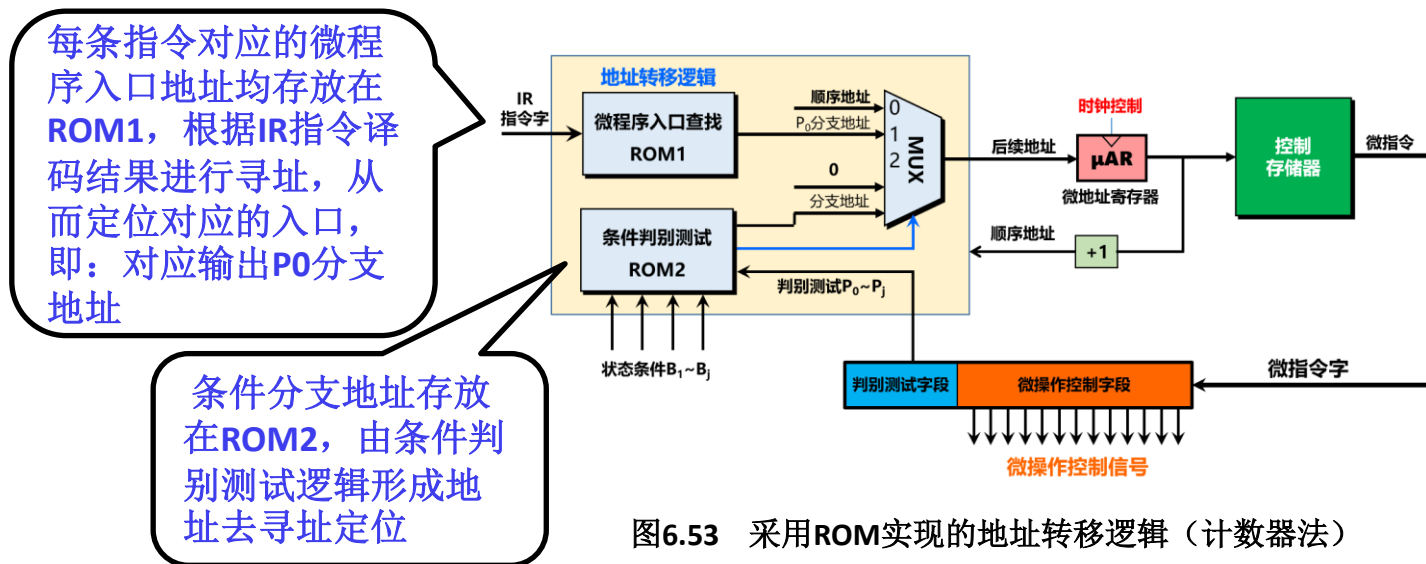


图6.53 采用ROM实现的地址转移逻辑（计数器法）

6.6.3 微程序控制器设计

– 对图6.8单总线结构计算机的控制器采用微程序方法进行设计。

– 1、指令周期、数据通路设计

- 前面已经完成（6.3.2小节）单总线结构计算机的数据通路和指令周期的设计，具体见图6.8（数据通路）和图6.14（指令周期）。

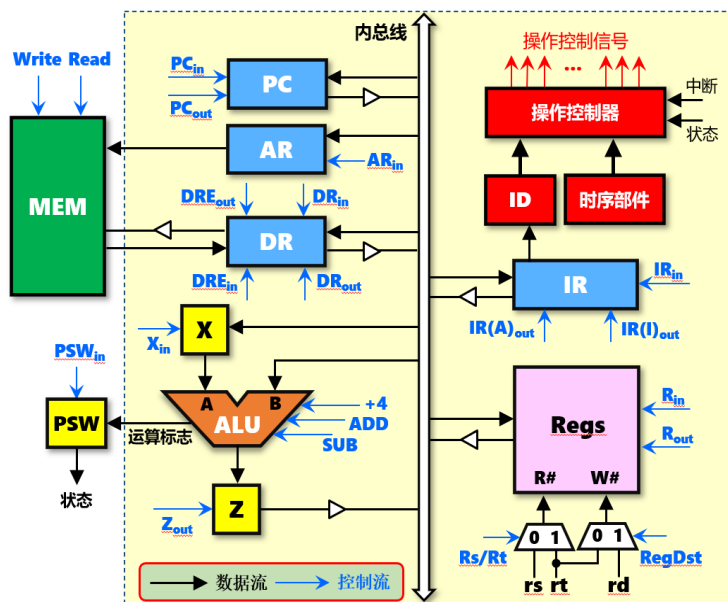


图6.8 单总线结构的计算机框图

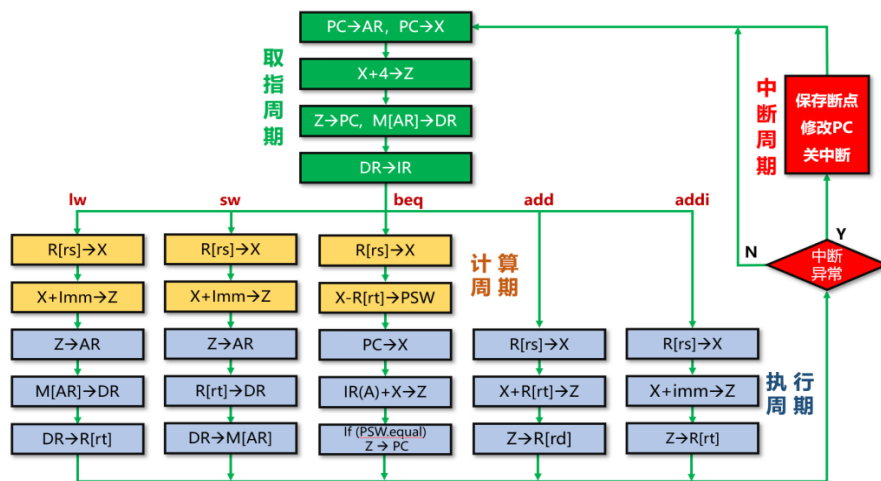


图6.14 单总线结构数据通路指令方框图

– 2、微指令设计

- 图6.8单总线结构计算机有22个控制信号（表6.1），因此微指令的**操作控制字段**需要**22位**。
- 采用下址字段法形成后续微地址，微指令格式如图6.54；**判别测试字段**有**2位**，分别是 P_0 （ P_{IR} ）和 P_1 （ P_{equal} ）；暂定下址字段有**5位**，可以支持**32条**微指令。

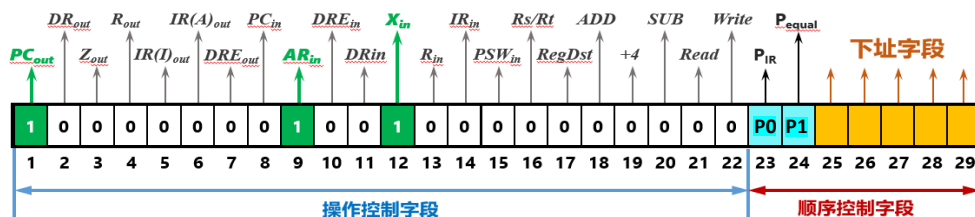


图6.54 微指令格式（下址字段法）

- 采用计数器法形成后续微地址，微指令格式如图6.55；**判别测试字段**有**3位**，增加1位 P_2 （ P_{end} ）；没有下址字段。

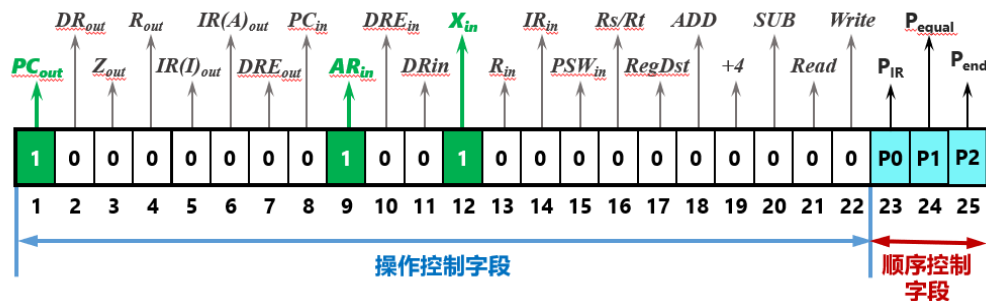


图6.55 微指令格式（计数器法）

– 3、微程序设计

- 采用下址字段法设计图6.8单总线结构计算机的微程序。

• （1）取指微程序

- 根据表6.3，取指周期需要4条微指令，具体如图6.56所示。
- 第1条微指令（微地址=00000）对应节拍T₁；22位操作控制字段只有3个是1，其余为0，对应表6.3中取指周期T₁节拍的3个控制信号；判断测试字段的2位（P_{IR}、P_{equal}）全部为0，即不需要进行条件测试；下址字段为00001，表示接下去执行第2条微指令（微地址=00001）。
- 第2条、第3条微指令与第1条微指令类似，分别对应表6.3中取指周期的节拍T₂和T₃。
- 第4条微指令对应节拍T₄，该微指令的判断测试位P_{IR}=1，表示要根据指令字（指令译码信号）来确定微指令的后续地址；因此，该微指令的下址字段为不确定值（xxxxx）。

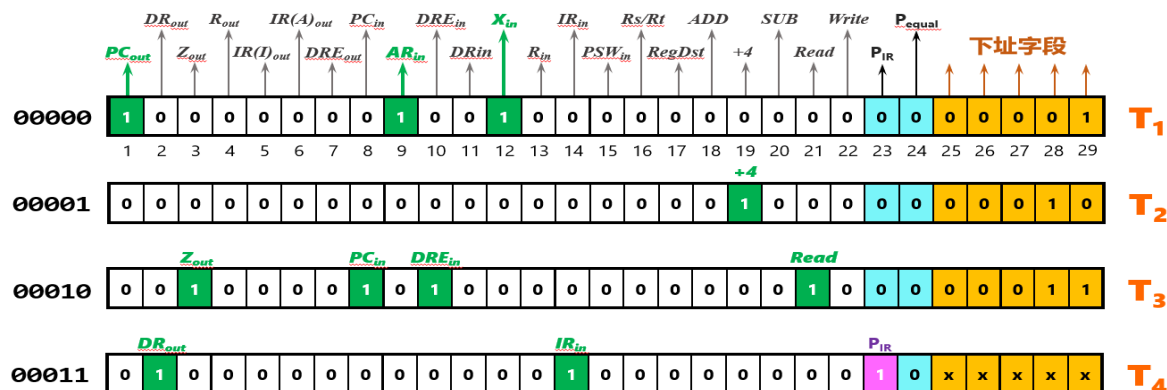


图6.56 取指微程序（下址字段法）

表6.3 lw指令操作流程及控制信号（仅给出值为1的控制信号）

周期	节拍	操作	控制信号
取指周期 M _{if}	T ₁	PC -> AR; PC -> X	PC _{out} =AR _{in} =X _{in} =1
	T ₂	X+4 -> Z	+4=1
	T ₃	Z -> PC; M[AR] -> DR	Z _{out} =PC _{in} =1 Read=DRE _{in} =1
	T ₄	DR -> IR	DR _{out} =IR _{in} =1

• (2) lw指令微程序 $lw\ rt,imm(rs); R[rt]=M[R[rs]+imm]$

- 根据表6.3，lw指令的计算周期和执行周期需要5条微指令，具体如图6.57所示。
- 第1条微指令（微地址=00100）对应计算周期的节拍T₁，下址字段=00101。
- 第2条微指令（微地址=00101）对应计算周期的节拍T₂，下址字段=00110。
- 第3条微指令（微地址=00110）对应执行周期的节拍T₁，下址字段=00111。
- 第4条微指令（微地址=00111）对应执行周期的节拍T₂，下址字段=01000。
- 第5条微指令（微地址=01000）对应执行周期的节拍T₃，下址字段=00000，表示进入指令取值周期，接下去执行取指微程序的第1条微指令。

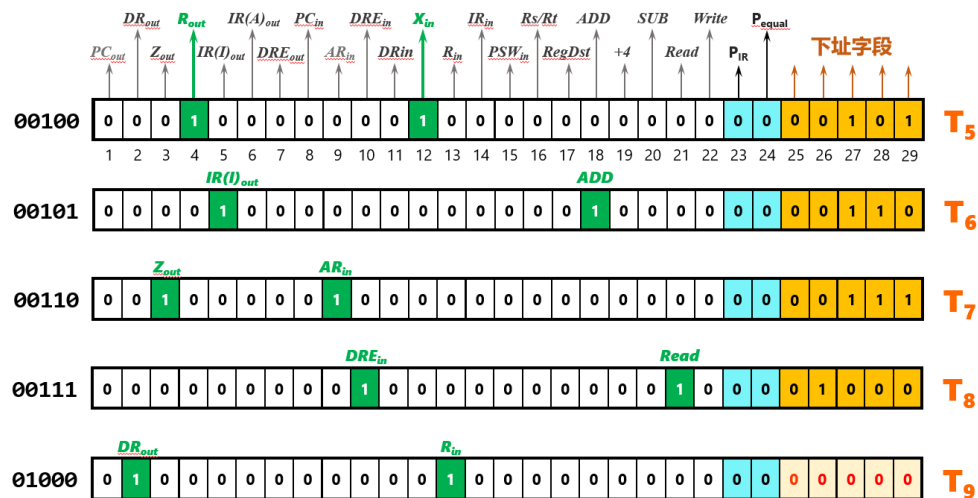


表6.3 lw指令操作流程及控制信号（仅给出值为1的控制信号）

周期	节拍	操作	控制信号
----	----	----	------

计算周期 M_{cal}	T ₁	$R[rs] \rightarrow X$	$R_{out}=X_{in}=1$
	T ₂	$IR(I) + X \rightarrow Z$	$IR(I)_{out}=ADD=1$
执行周期 M_{ex}	T ₁	$Z \rightarrow AR$	$Z_{out}=AR_{in}=1$
	T ₂	$M[AR] \rightarrow DR$	$Read=DRE_{in}=1$
	T ₃	$DR \rightarrow R[rt]$	$DR_{out}=R_{in}=1$

图6.57 lw指令微程序（下址字段法）

• (3) **sw指令微程序** **sw rt,imm(rs); M[R[rs]+imm]=R[rt]**

- 根据表6.4，sw指令的计算周期和执行周期需要5条微指令，具体如图6.58所示。
- 第1条微指令（微地址=01001）对应计算周期的节拍T1，下址字段=01010。
- 第2条微指令（微地址=01010）对应计算周期的节拍T2，下址字段=01011。
- 第3条微指令（微地址=01011）对应执行周期的节拍T1，下址字段=01100。
- 第4条微指令（微地址=01100）对应执行周期的节拍T2，下址字段=01101。
- 第5条微指令（微地址=01101）对应执行周期的节拍T3，下址字段=00000，表示接下去执行取指令程序的第1条微指令。

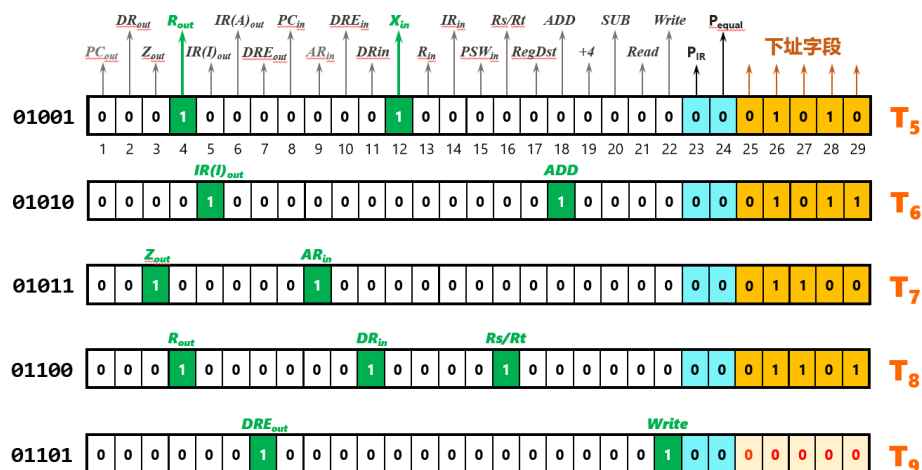


表6.4 sw指令操作流程及控制信号（仅给出值为1的控制信号）

周期	节拍	操作	控制信号
----	----	----	------

计算周期 M_{cal}	T ₁	R[rs] -> X	R _{out} =X _{in} =1
	T ₂	IR(I) + X -> Z	IR(I) _{out} =ADD=1
执行周期 M_{ex}	T ₁	Z -> AR	Z _{out} =AR _{in} =1
	T ₂	R[rt] -> DR	R _{out} =Rs/Rt=DR _{in} =1
	T ₃	DR -> M[AR]	DRE _{out} =Write=1

图6.58 sw指令微程序（下址字段法）

• (4) beq指令微程序 **beq rs, rt, imm ; if (R[rs]==R[rt]) PC=PC+4+imm<<2**

- 根据表6.5，beq指令的计算周期和执行周期需要5条微指令，具体如图6.59所示。
- 第1条微指令（微地址=01110）对应计算周期的节拍T₁，下址字段=01111。
- 第2条微指令（微地址=01111）对应计算周期的节拍T₂，下址字段=10000。
- 第3条微指令（微地址=10000）对应执行周期的节拍T₁，下址字段=10001。
- 第4条微指令（微地址=10001）对应执行周期的节拍T₂，下址字段=10010。
- 第5条微指令（微地址=10010）对应执行周期的节拍T₃，下址字段=00000，表示接下去执行取指令程序的第1条微指令。

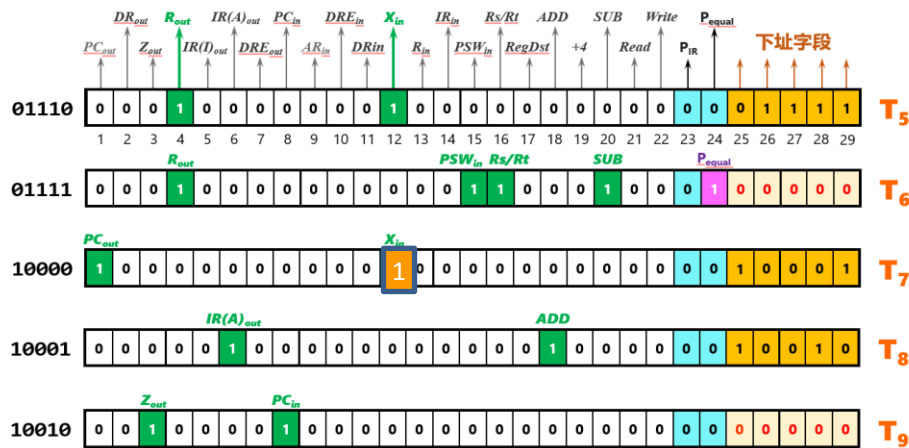


表6.5 beq指令操作流程及控制信号（仅给出值为1的控制信号）

周期	节拍	操作	控制信号
计算周期 M_{cal}	T ₁	R[rs] -> X	$R_{out}=X_{in}=1$
	T ₂	X - R[rt] -> PSW	$R_{out}=Rs/Rt=SUB=PSW_{in}=1$
执行周期 M_{ex}	T ₁	PC -> X	$PC_{out}=X_{in}=1$
	T ₂	IR(A) + X -> Z	$IR(A)_{out}=ADD=1$
	T ₃	if(PSW.equal) Z -> PC	$Z_{out}=1$ $PC_{in}=PSW.equal$

图6.59 beq指令微程序（下址字段法）

- (5) add指令微程序 **add rd,rs,rt R[rd]=R[rs]+R[rt]**
 - 根据表6.7，add指令的执行周期需要3条微指令，具体如图6.60a所示。
 - 第1条微指令（微地址=10011）对应执行周期的节拍T1，下址字段=10100。
 - 第2条微指令（微地址=10100）对应执行周期的节拍T2，下址字段=10101。
 - 第3条微指令（微地址=10101）对应执行周期的节拍T3，下址字段=00000，表示接下去执行取指微程序的第1条微指令。

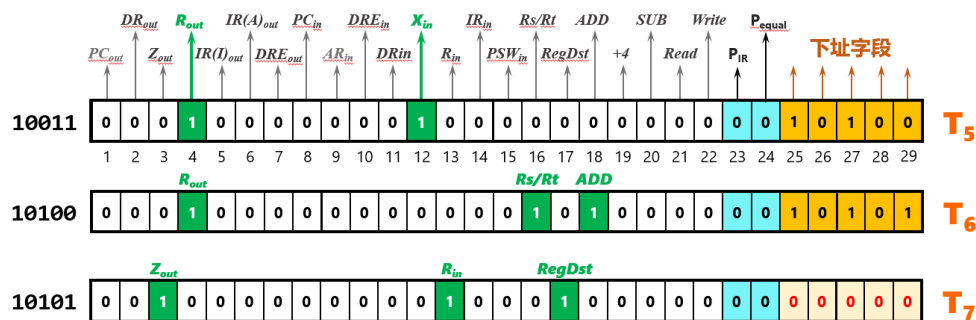


表6.7 add指令操作流程及控制信号（仅给出值为1的控制信号）

周期	节拍	操作	控制信号
----	----	----	------

执行周期 M _{ex}	T ₁	R[rs] -> X	R _{out} =X _{in} =1
	T ₂	R[rt] + X -> Z	R _{out} =Rs/Rt=ADD=1
	T ₃	Z -> R[rd]	Z _{out} =RegDst=R _{in} =1

图6.60a add指令微程序（下址字段法）

- (6) **addi指令微程序** **addi rt,rs,imm; R[rt]=R[rs]+imm**

- 根据表6.6，**addi**指令的执行周期需要**3条微指令**，具体如图6.60b所示。
- 第1条微指令（微地址=**10110**）对应执行周期的节拍**T1**，下址字段=**10111**。
- 第2条微指令（微地址=**10111**）对应执行周期的节拍**T2**，下址字段=**11000**。
- 第3条微指令（微地址=**11000**）对应执行周期的节拍**T3**，下址字段=**00000**，表示接下去执行取指微程序的第1条微指令。

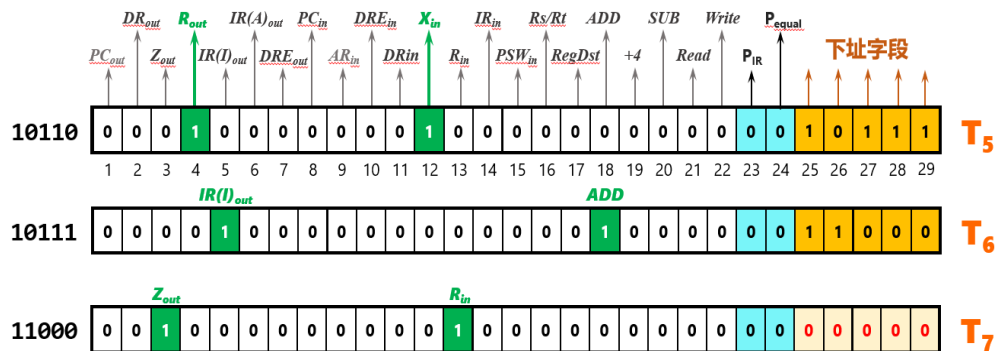


表6.6 **addi**指令操作流程及控制信号（仅给出值为1的控制信号）

周期	节拍	操作	控制信号
执行周期 M_{ex}	T_1	$R[rs] \rightarrow X$	$R_{out}=X_{in}=1$
	T_2	$IR(I) + X \rightarrow Z$	$IR(I)_{out}=ADD=1$
	T_3	$Z \rightarrow R[rt]$	$Z_{out}=R_{in}=1$

图6.60b **addi**指令微程序（下址字段法）

- 综上，单总线结构的计算机5条机器指令共需要25条微指令，每条微指令的长度=22位（操作控制字段）+2位（判别测试字段）+5位（下址字段）=29位，控制存储器的容量为：**25x29位**，图6.61a。

状态	微地址	操作控制字段															顺序控制字段							
0	00000	1						1		1									0	0	0	0	1	取指微程序
1	00001															1			0	0	0	1	0	
2	00010			1				1		1								1	0	0	0	1	1	
3	00011		1											1				1	x	x	x	x	x	
4	00100				1									1					0	0	1	0	1	lw
5	00101					1										1			0	0	1	1	0	
6	00110			1						1									0	0	1	1	1	
7	00111									1								1	0	1	0	0	0	
8	01000		1											1					0	0	0	0	0	sw
9	01001				1									1					0	1	0	1	0	
10	01010					1										1			0	1	0	1	1	
11	01011			1						1									0	1	1	0	0	
12	01100				1								1				1		0	1	1	0	1	beq
13	01101							1										1	0	0	0	0	0	
14	01110				1								1						0	1	1	1	1	
15	01111				1									1	1			1	0	0	0	0	0	
16	10000	1											1						1	0	0	0	1	add
17	10001							1									1		1	0	0	1	0	
18	10010			1						1									0	0	0	0	0	
19	10011				1								1						1	0	1	0	0	
20	10100				1									1			1		1	0	1	0	1	addi
21	10101			1									1				1		0	0	0	0	0	
22	10110				1								1						1	0	1	1	1	
23	10111					1										1			1	1	0	0	0	
24	11000		1										1						0	0	0	0	0	

图6.61 单总线结构计算机的微程序（下址字段法）

- 如果采用计数器法设计微指令，则不需要下址字段，但是必须增加1位判别测试位 P_2 (P_{end})；此时，每条微指令的长度=22位（操作控制字段）+3位（判别测试字段）=25位，控制存储器的容量为：**25x25位**，图6.61b。

状态	微地址	操作控制字段																						顺序控制字段		
																								P_0	P_1	P_{end}
0	00000	1																								
1	00001																									
2	00010																									
3	00011																									
4	00100																									
5	00101																									
6	00110																									
7	00111																									
8	01000																									
9	01001																									
10	01010																									
11	01011																									
12	01100																									
13	01101																									
14	01110																									
15	01111																									
16	10000																									
17	10001																									
18	10010																									
19	10011																									
20	10100																									
21	10101																									
22	10110																									
23	10111																									
24	11000																									

图6.61b 单总线结构计算机的微程序（计数器法）

- 微程序控制器的**设计流程**:
 - ① 分析指令执行的数据通路，列出每条指令的执行操作流程和每一步需要的控制信号。
 - ② 对指令的操作流程进行细化，将每条指令的每个微操作分配到具体的机器周期的各个时间节拍信号上。
 - ③ 以时钟周期为单位构建指令执行状态图。
 - ④ 设计微指令格式、微命令编码方式。
 - ⑤ 根据指令执行状态图编制每条指令的微程序，按照状态机组织微程序并存放到控制存储器中。
 - ⑥ 根据微程序组织方式构建微程序控制器中的地址转移逻辑，微地址寄存器**AR**、控制存储器之间的通路，实现微程序控制器。
- 微程序控制器因为要频繁访问控制存储器，其性能比硬布线控制器**要差**。
- 但是微程序控制器的**设计规整、实现容易**，如果控制存储器采用可写的**ROM**（如EEPROM、Flash ROM），可方便**修改微程序、扩展指令的功能**。
- 微程序控制器适合**CISC**计算机，如x86、IBM S/360、DEC VAX等；硬布线控制器适合**RISC**计算机，如MIPS、ARM、RISC-V等。

6.6.4 微指令及其编码方法

– 1、微命令编码方法

- 微命令的编码方法就是微指令中的**操作控制字段**的表示方法，常见的有：直接表示法、编码表示法、混合表示法。

- (1) 直接表示法

- 微指令操作控制字段的每1位对应1个微命令（微操作，控制信号），“1”表示发出该控制信号，“0”表示不发出该控制信号，图6.62。
- 直接表示法的**优点**是简单，微操作的并行能力强，操作速度快；**缺点**是操作控制字段很长。
- 前面介绍的微程序设计都是采用的直接表示法（图6.47、图6.54、图6.55、图6.56、图6.57、图6.58、图6.59、图6.60、图6.61）。

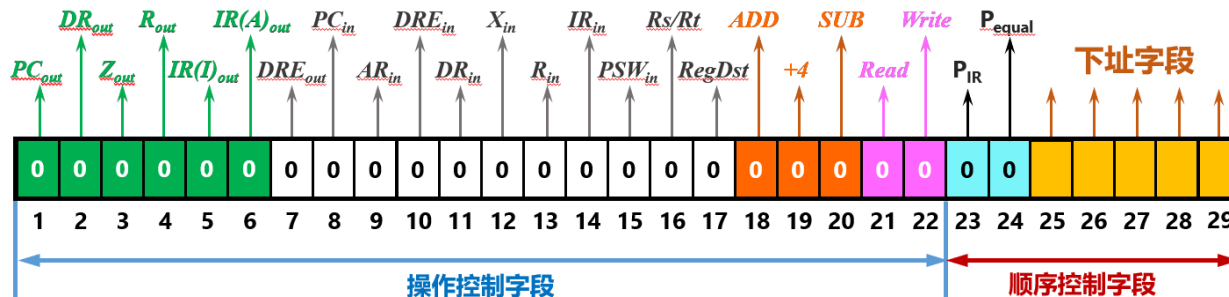


图6.62a 直接表示法

6.6.4 微指令及其编码方法

• (2) 编码表示法

- 编码表示法也称为**字段译码法**，该方法将**互斥性的微操作**（控制信号；例如存储器的读写控制信号Read和Write就是互斥的）分成若干组，1个组对应1个字段，各组内部的控制信号均是**互斥**的，各字段通过译码器生成控制信号，图6.62b。

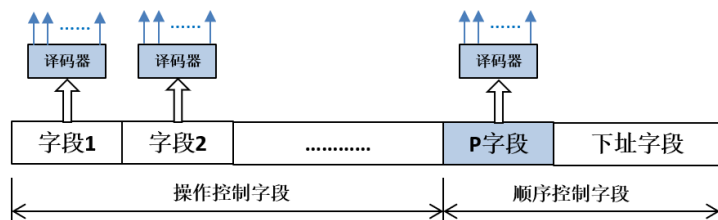


图6.62b 编码表示法

- 编码表示法的**优点**是能有效缩短微指令的字长，**缺点**是译码器会降低微指令的执行速度。

• (3) 混合表示法

- 将**直接表示法**和**编码表示法**混合使用，以便在微指令字长、并行性及执行速度和灵活性等方面进行折中，发挥它们的共同优点，图6.62c。

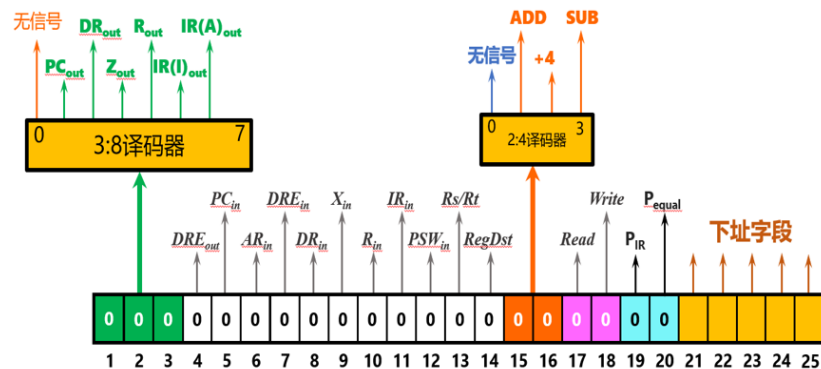


图6.62c 混合表示法

- 例6.6：某微指令为24位字长，采用混合控制法（混合编码法），其中23～15位用直接表示法，14～5位分为A、B、C三组，均采用编码表示法，C组除表示4种控制转移的判别测试P1～P4外，其余用于表示微命令，各字段的位数分配如图6.63所示，请回答如下问题：
 - （1）该格式的微指令最多可表示多少种微命令？
 - （2）一条微指令中可同时出现的微命令最多可以有多少个？
 - （3）控制存储器的最大容量是多少？
- 解：
 - （1）采用直接表示的微命令有9个，A组、B组经过译码后各可表示7个微命令（ $2^3-1=7$ ，8种情况，有1种情况表示不发出任何控制信号），C组用于微命令的个数为11（ $2^4-1-4=11$ ，4个判别测试P1～P4，有1种情况表示不发出任何控制信号）；因此，该微指令最多可表示 $9+7+7+11=34$ 个微命令。
 - （2）采用编码表示法时，每个字段最多只能发出1个微命令（控制信号）；因此，一条微指令中可同时出现的微命令最多为 $9+1+1+1=12$ 个。
 - （3）因为下址字段为5位，微指令最多有 $2^5=32$ 条，微指令的字长=24位；因此，控制存储器的最大容量是 32×24 位。

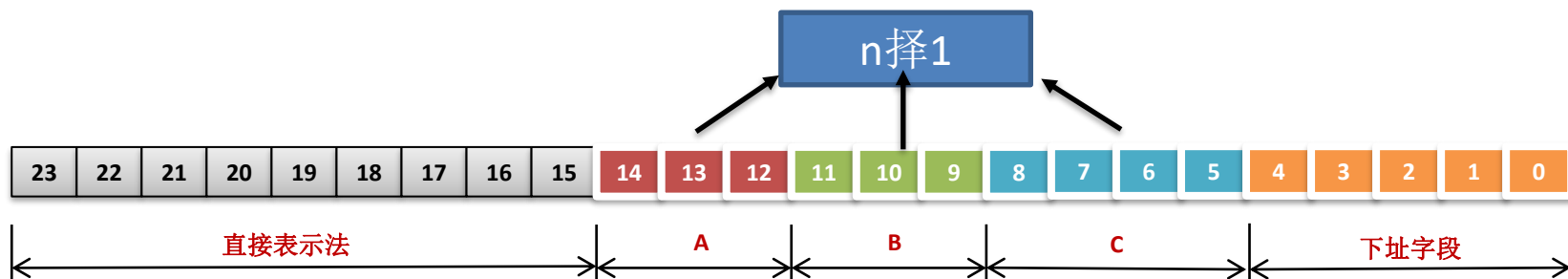


图6.63 各字段的位数分配

- 2、微指令的格式
 - (1) 水平型微指令
 - 在一个微指令周期内，能同时给出多个微命令的微指令，称为水平型微指令；前面的直接表示法、编码表示法、混合表示法都属于水平型微指令，分别称为全水平型、编码水平型和混合水平型微指令。
 - (2) 垂直型微指令
 - 垂直型微指令类似于机器指令的编码方法，一条垂直型微指令包含微操作码字段和地址码字段；表6.16：垂直型微指令格式与功能。
 - 如果采用扩展指令格式设计，表6.16的垂直型微指令最多只需要13位的指令字即可实现：
 - 寄存器地址6位，可以访问32个通用寄存器及其他寄存器（如PC、AR、DR、X、Z、IR等）。
 - 表6.16中“MOV AR,PC”微指令可表示为：0 xxxxxx xxxxxx，13位，xxxxxx表示AR，xxxxxx表示PC。
 - 其他微指令因为是单操作数或零操作数，操作码采用扩展技术，可以满足数量的要求。

表6.16 垂直型微指令格式与功能

类型	垂直型微指令举例	指令格式	功能说明
寄存器传送	MOV AR,PC	双操作数	PC -> AR
运算控制	ADD 4	单操作数	X+4 -> Z
运算控制	ADD DATA	单操作数	X + DATA -> Z
运算控制	SUB DATA	单操作数	X - DATA -> Z
主存传送	LOAD	零操作数	M[AR] -> DR
主存传送	STORE	零操作数	DR -> M[AR]
条件转移	BRANCH P ₀	单操作数	如果P ₀ =1，则根据指令译码信号转到相应指令的计算周期或执行周期的微程序入口
条件转移	BRANCH P _{end}	单操作数	如果P _{end} =1，则转到取指微程序的入口

— 表6.17: lw指令微程序垂直型微指令举例；共需要**13条**微指令，每条微指令的长度为**13位**，微程序容量=13x13位=**169位**。

— 如果采用水平型指令，lw指令微程序的容量为：

① 全水平型微指令（下址字段法）：图6.54，需要**9条**微指令，每条微指令的长度=29位，微程序容量=9x29=261位。

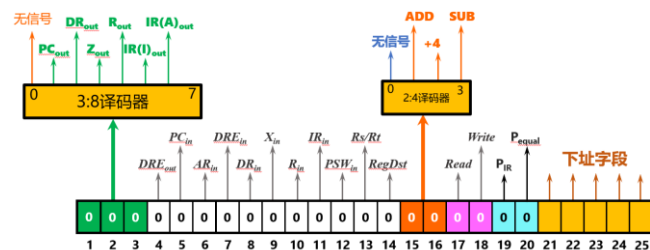
② 全水平型微指令（计数器法）：图6.55，需要9条微指令，每条微指令的长度=25位，微程序容量=9x25=225位。

③ 混合水平型微指令（下址字段法）：右下图，需要9条微指令，每条微指令的长度=25位，微程序容量=9x25=225位。

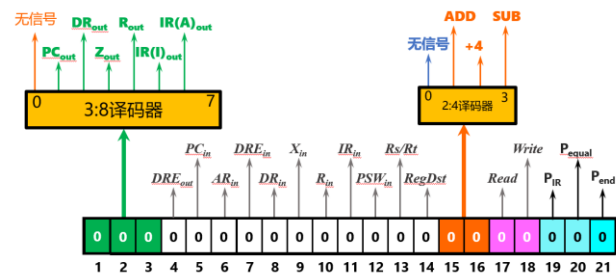
④ 混合水平型微指令（计数器法）：右下图，需要9条微指令，每条微指令的长度=21位，微程序容量=9x21=189位。

表6.17 lw指令微程序垂直型微指令举例

		9条		13条	
周期	节拍	操作	微指令序号	垂直型微指令	控制信号
取指周期 M_{if}	T1	$PC \rightarrow AR; PC \rightarrow X$	C1	MOV AR, PC	PC_{out}, AR_{in}
			C2	MOV X, PC	PC_{out}, X_{in}
	T2	$X+4 \rightarrow Z$	C3	ADD 4	+4
	T3	$Z \rightarrow PC; M[AR] \rightarrow DR$	C4	MOV PC, Z	Z_{out}, PC_{in}
			C5	LOAD	$DRE_{in}, Read$
	T4	$DR \rightarrow IR$	C6	MOV IR, DR	DR_{out}, IR_{in}
			C7	BRANCH P_0	
计算周期 M_{cal}	T1	$R[rs] \rightarrow X$	C8	MOV X, R[rs]	R_{out}, X_{in}
	T2	$IR(I) + X \rightarrow Z$	C9	ADD IR(I)	$IR(I)_{out}, ADD$
执行周期 M_{ex}	T1	$Z \rightarrow AR$	C10	MOV AR, Z	Z_{out}, AR_{in}
	T2	$M[AR] \rightarrow DR$	C11	LOAD	$DRE_{in}, Read$
	T3	$DR \rightarrow R[rt]$	C12	MOV R[rt], DR	DR_{out}, R_{in}
			C13	BRANCH P_{end}	



混合水平型微指令（下址字段法）



混合水平型微指令（计数器法）

- 垂直型微指令的**优点**：每条微指令每个时钟周期只控制1 ~ 2个微操作（只发出1~2个控制信号，表6.17每条微指令的控制信号最多只有**2个**），微指令结构简单规整、字长短、易于编制微程序。
- 垂直型微指令的**缺点**：编制的微程序较长（lw指令的微程序有**13条**微指令），不能充分利用数据通路固有的并行性，几乎没有并行能力，故执行效率低。
- 水平型微指令的**特点**：微程序短（lw指令的微程序有**9条**微指令），执行效率高，能充分利用数据通路固有的并行性，有较高的并行操作能力。
- 水平型微指令是**面向数据通路**的描述，垂直型微指令是**面向操作算法**的描述。
- 垂直型微指令的设计思想在Pentium4和安腾处理器中得到了应用，但是目前基本被淘汰。

6.7 异常与中断处理

• 6.7.1 异常与中断的基本概念

- **异常 (Exception)**：通常是指CPU内部引起的异常事件，也称为**内部中断**、软件中断；异常可以进一步分为：故障、自陷、终止。
 - ① **故障 (Fault)**：通常是由**指令执行引起**的异常，如未定义指令、越权指令、段故障、缺页故障、存储保护违例、数据未对齐、除数为零、浮点溢出、整数溢出等；故障又可分为：可恢复故障（如数据缺页）和不可恢复故障（如未定义指令、越权指令等）。
 - ② **自陷 (Trap)**：是一种事先安排的“异常”事件；如系统调用指令、条件陷阱指令；例如x86的**INT 21H**指令，MIPS的**syscall**指令。
 - ③ **终止 (Abort)**：是指随机出现的使得CPU无法继续执行的**硬件故障**，和具体指令无关；如机器校验错、总线错误、异常处理中再次异常的双错等。
- **中断 (外部中断, Interrupt)**：是指由外部设备向CPU发出的中断请求（如鼠标点击、按键动作等），要求CPU暂停当前正在执行的程序，转去执行为某个外部设备事件服务的中断服务程序，处理完毕后，再返回断点继续执行。
- MIPS处理器将异常和中断都称为“异常”。
- x86处理器将异常和中断都称为“中断”，来自CPU内部的中断称为“内部中断（软件中断）”，来自CPU外部的中断称为“外部中断（硬件中断）”。

6.7.2 异常与中断处理过程

- 当发生中断（或异常）事件时，CPU接收到中断请求，在**指令执行结束时**CPU要进入**中断响应周期**进行响应处理（例外情况：例如产生故障异常的指令并没有执行完毕，但必须立即进入中断响应）。
- **中断响应周期**的主要任务是：关中断、保存断点、中断识别。
- （1）**关中断**：关中断的目的是临时**禁止新的中断请求**，是为了在中断响应周期以及中断服务程序中保护现场操作的完整性，只有这样才能保证中断服务程序执行完成后，能返回断点正确执行。
- （2）**保存断点**：保存断点就是保存将来返回被中断程序的位置；对于已经执行完毕的指令，断点是**下一条指令的位置**；对于缺页故障、段错等执行指令引起的故障异常，由于指令并没有执行，断点就是异常指令的PC值。
- （3）**中断识别**：中断识别的主要任务就是根据当前的中断请求**识别中断来源**，将中断服务程序入口地址送入程序计数器PC。
- 中断响应周期内的操作是由**硬件实现**的，整个中断响应周期是不可被打断的；中断响应周期结束后，CPU将执行中断服务程序，直至中断返回，这部分由软件实现（**中断服务程序**）；因此，整个中断处理过程是软硬件协同实现的。

6.7.3 支持中断的CPU设计

– 1、数据通路升级

- 图6.64为支持中断的单总线结构的计算机框图（图中没有画出ALU和寄存器堆）；与图6.4相比，增加了：

- ① **EPC**: Exception Program Counter, **异常程序计数器**, 用于保存断点；有2个控制信号： EPC_{in} 、 EPC_{out} 。
- ② **IE**: Interrupt Enable, 中断使能位（**中断使能触发器/寄存器**），有2个控制信号：开中断STI、关中断CLI。
- ③ 中断控制（**中断控制逻辑**）：用于进行中断优先级排队；假设有4个外部中断请求信号A、B、C、D，中断控制逻辑对4个中断请求进行优先级排队，输出优先级最高的中断号，并发出中断请求信号，控制信号CtrlInt用于清除当前正在响应的中断请求。
- ④ **多路选择器MUX**：根据中断号，将对应的中断服务程序入口地址（A中断入口、B中断入口、C中断入口、D中断入口）通过内总线送程序计数器PC，控制信号为IntA_{out}。

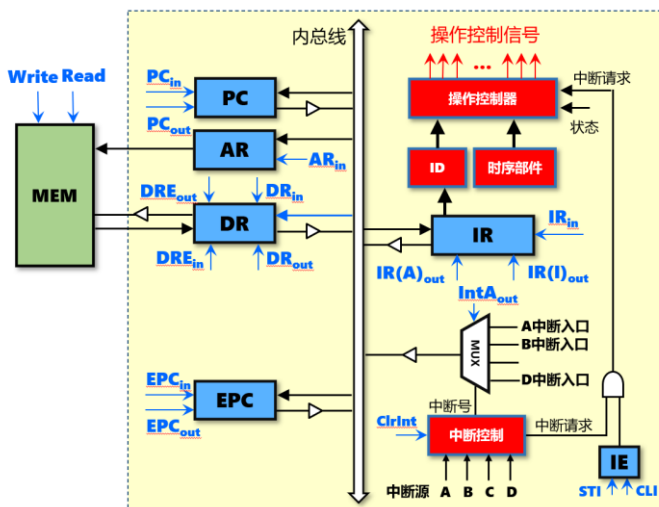


表6.18 中断控制信号及其作用

序号	控制信号	作用说明
1	EPC_{in}	控制EPC接收来自内总线的数据，需配合时钟控制
2	EPC_{out}	控制EPC向内总线输出数据
3	STI	开中断，将中断使能寄存器置1，操作控制器可以接收中断请求
4	CLI	关中断，将中断使能寄存器置0，操作控制器可以接收不到任何中断请求
5	IntA _{out}	将中断服务程序入口地址输出到内总线
6	CtrlInt	清除当前正在响应的中断请求

图6.64 支持中断的单总线结构计算机框图

– 2、操作控制器升级

• （1）三级时序硬布线控制器升级

– 对图6.41的**单总线**结构计算机的**三级时序**状态机（**变长指令周期**）进行升级，使其能支持中断，升级后的状态机如图6.65所示。

– 图6.65中增加了1个**中断响应周期** M_{int} ，包含2个状态 S_9 和 S_{10} ，如表6.19所示：

① 状态 S_9 ：进行关中断和保存断点的操作，将PC值送入EPC寄存器中保存，发出控制信号： $CLI=1$ 、 $PC_{out}=1$ 、 $EPC_{in}=1$ 。

② 状态 S_{10} ：完成中断识别的操作，将要响应的中断服务程序入口地址送PC，发出控制信号： $IntA_{out}=1$ 、 $PC_{in}=1$ 。

– 此外，还有增加1条中断返回指令**eret**；该指令的主要功能是将EPC中保存的断点送回PC，同时开中断；该指令执行周期的操作及控制信号如表6.20所示。

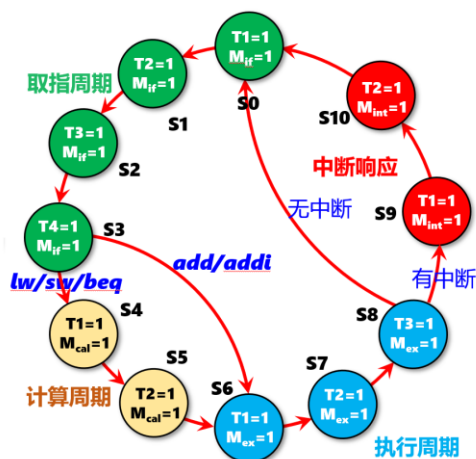


表6.19 中断响应周期的操作及控制信号

周期	节拍	操作	功能说明	控制信号
中断 M_{int}	T1	$0 \rightarrow IE$ $PC \rightarrow EPC$	关中断，同时将PC值送入EPC寄存器中保存	$CLI=PC_{out}=EPC_{in}=1$
	T2	中断程序入口 $\rightarrow PC$	将要响应的中断对应的中断程序入口地址送PC	$IntA_{out}=PC_{in}=1$

表6.20 eret指令执行周期的操作及控制信号

周期	节拍	操作	功能说明	控制信号
中断 M_{int}	T3	$EPC \rightarrow PC$ $1 \rightarrow IE$	恢复断点，将EPC送PC，同时开中断	$EPC_{out}=PC_{in}=STI=ClrInt=1$

图6.65 支持中断的三级时序状态机（变长指令周期）

- (2) 现代时序硬布线控制器升级

- 同样可以对图6.45的**单总线**结构计算机的**现代时序**状态机进行升级，使其能支持中断，升级后的状态机如图6.66所示。
- 图6.66中增加了2个**状态 S_{26} 和 S_{27}** ，用于**中断响应**。
- 此外，也要增加1条中断返回指令**eret**，该指令的执行只需要1个时钟周期 S_{25} 。

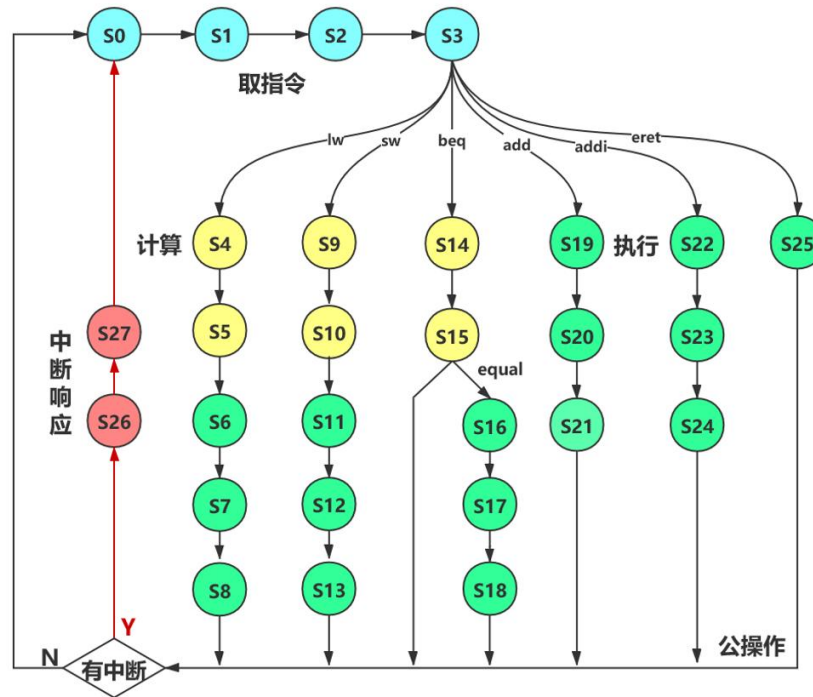


图6.66 支持中断的现代时序状态机

- **(3) 微程序控制器升级**

- 可以用微程序设计方法实现图6.66的支持中断的现代时序状态机：
- 首先，要修改微指令格式，增加6个中断控制信号（表6.18）；图6.67为支持中断的微指令格式（计数器法）。
- 其次，要增加2条中断响应微指令（对应图6.66中的 S_{26} 和 S_{27} ）；增加1条中断返回指令的微指令（对应图6.66中的 S_{25} ）。
- 此外，指令执行完毕要进行中断判断，不管是下址字段法还是计数器法，都要有 P_{end} 测试位，并根据中断测试条件进行分支（如果有中断，则转中期响应周期；如果没有中断则转取指微程序入口）。
- 图6.68为支持中断的微程序控制器原理图（采用下址字段法）：
 - ① 第1种情况： $P_{end}=1$ 、中断请求 $IntR=1$ 、 $P_{equal}=0$ ；进入中断响应周期
 - ② 第2种情况： $P_{end}=1$ 、 $IntR=0$ ；进入 S_0
 - ③ 第3种情况： beq 指令， $P_{end}=1$ 、 $IntR=0$ 、 $P_{equal}=0$ ；进入 S_0
 - ④ 第4种情况： beq 指令， $P_{end}=1$ 、 $IntR=0$ 、 $P_{equal}=1$ ；进入 S_{16}

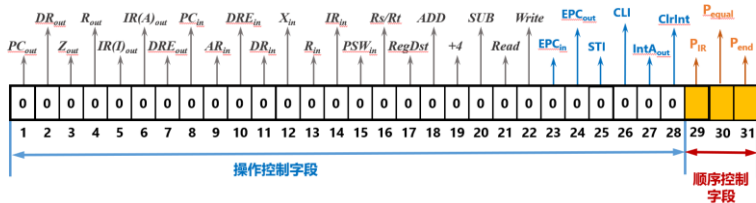


图6.67 支持中断的微指令格式（计数器法）

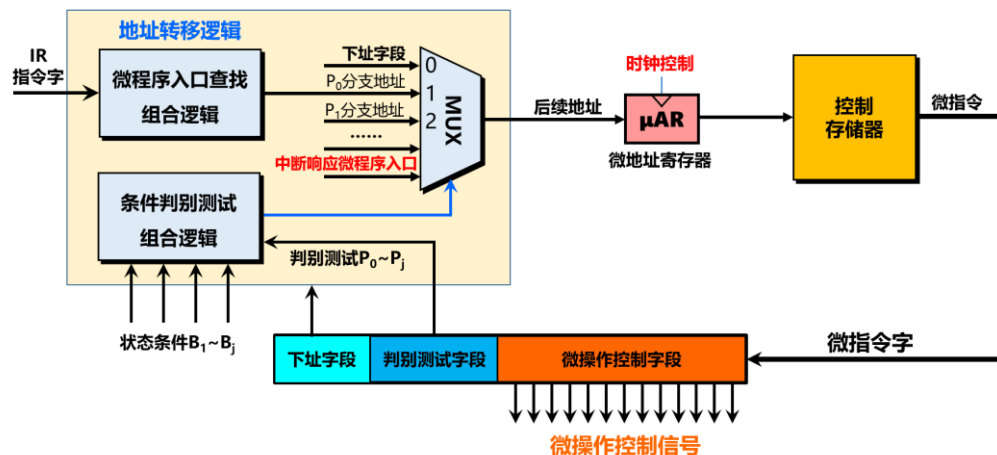


图6.68 支持中断的微程序控制器原理图（下址字段法）

6.7.3 支持中断的CPU设计

— 3、中断服务程序

- 中断机制是软硬件协同的机制，**中断响应周期**内的操作（关中断、保存断点、中断识别）都是由**硬件**实现的，**中断服务程序**则是由**软件**实现的。
- 中断服务程序主要包括4个步骤：
 - ① **保护现场**：将中断服务程序中会被改写的寄存器，通过压栈的方式保存到内存堆栈中，以保证被中断的程序在执行完中断服务程序后，还能正确执行。
 - » `PUSH AX`
 - » `PUSH BX`
 - » `PUSH CX`
 - ② **中断服务**：完成中断处理，这是中断服务程序的核心内容。
 - ③ **恢复现场**：通过出栈的方式，将保存在内存堆栈中的内容送回寄存器。
 - » `POP CX`
 - » `POP BX`
 - » `POP AX`
 - ④ **中断返回**：执行中断返回指令**eret**，返回到断点处。

实践训练（实验5）

- 用Logisim构建单总线结构CPU，要求能支持MIPS32指令，能运行简单的内存冒泡排序程序，尝试利用以下3种方式构建控制器：
 - ① 传统三级时序硬布线控制器
 - ② 现代时序硬布线控制器
 - ③ 微程序控制器
- 尝试为单总线CPU增加简单的中断处理逻辑，使得CPU能响应按键中断
- 用Logisim构建单周期MIPS处理器，要求能支持MIPS32指令，能运行简单的内存冒泡排序程序
- 用Logisim构建多周期MIPS处理器，要求能支持MIPS32指令，能运行简单的内存冒泡排序程序，控制器可以采用硬布线、微程序控制器中的一种